

Hyper-Parameter Optimization: A Review of Algorithms and Applications

Tong Yu

YUTONG01@INSPUR.COM

Department of AI and HPC

Inspur Electronic Information Industry Co., Ltd

1036 Langchao Rd, Jinan, Shandong, China

Hong Zhu

ZHUHONGBJ@INSPUR.COM

Department of AI and HPC

Inspur (Beijing) Electronic Information Industry Co., Ltd

2F, Block C, 2 Xinxu Rd., Shangdi, Haidian Dist, Beijing, China

Editor:

Abstract

Since deep neural networks were developed, they have made huge contributions to peoples everyday lives. Machine learning provides more rational advice than humans are capable of in almost every aspect of daily life. However, despite this achievement, the design and training of neural networks are still challenging and unpredictable procedures that have been alleged to be alchemy. To lower the technical thresholds for common users, automated hyper-parameter optimization (HPO) has become a popular topic in both academic and industrial areas. This paper provides a review of the most essential topics on HPO. The first section introduces the key hyper-parameters related to model training and structure, and discusses their importance and methods to define the value range. Then, the research focuses on major optimization algorithms and their applicability, covering their efficiency and accuracy especially for deep learning networks. This study next reviews major services and tool-kits for HPO, comparing their support for state-of-the-art searching algorithms, feasibility with major deep-learning frameworks, and extensibility for new modules designed by users. The paper concludes with problems that exist when HPO is applied to deep learning, a comparison between optimization algorithms, and prominent approaches for model evaluation with limited computational resources.

Keywords: Hyper-parameter, auto-tuning, deep neural network

1. Introduction

In the past several years, neural network techniques have become ubiquitous and influential in both research and commercial applications. In the past 10 years, neural networks have shown impressive results in image classification (Szegedy et al., 2016; He et al., 2016), objective detection (Girshick, 2015; Redmon et al., 2016), natural language understanding (Hochreiter and Schmidhuber, 1997; Vaswani et al., 2017), and industrial control systems (Abbeel, 2016; Hammond, 2017). However, neural networks are efficient in applications but inefficient when obtaining a model. It is thought to be a brute force method because a network is initialized with a random status and trained to an accurate model with an extremely large dataset. Moreover, researchers must dedicate their efforts to carefully coping

with model design, algorithm design, and corresponding hyper-parameter selection, which means that the application of neural networks comes at a great price. Based on experience is generally the most widely used method, which means a practicable set of hyper-parameters requires researchers to have experience in training neural networks. However, the credibility of empirical values is weakened because of the lack of logical reasoning. In addition, experiences generally provide workable instead of optimal hyper-parameter sets.

The no free lunch theorem suggests that the computational cost for any optimization problem is the same for all problems and no solution offers a shortcut (Wolpert and Macready, 1997; Igel, 2014). A feasible alternative for computational resources is the preliminary knowledge of experts, which is efficient in selecting influential parameters and narrowing down the search space. To save the rare resource of experts experience, automated machine learning (AutoML) has been proposed as a burgeoning technology to design and train neural networks automatically, at the cost of computational resources (Feurer et al., 2015; Katz et al., 2016; Bello et al., 2017; Zoph and Le, 2016; Jin et al., 2018). Hyper-parameter optimization (HPO) is an important component of AutoML in searching for optimum hyper-parameters for neural network structures and the model training process.

Hyper-parameter refers to parameters that cannot be updated during the training of machine learning. They can be involved in building the structure of the model, such as the number of hidden layers and the activation function, or in determining the efficiency and accuracy of model training, such as the learning rate (LR) of stochastic gradient descent (SGD), batch size, and optimizer (hyp). The history of HPO dates back to the early 1990s (Ripley, 1993; King et al., 1995), and the method is widely applied for neural networks with the increasing use of machine learning. HPO can be viewed as the final step of model design and the first step of training a neural network. Considering the influence of hyper-parameters on training accuracy and speed, they must carefully be configured with experience before the training process begins (Rodriguez, 2018). The process of HPO automatically optimizes the hyper-parameters of a machine learning model to remove humans from the loop of a machine learning system. As a trade of human efforts, HPO demands a large amount of computational resources, especially when several hyper-parameters are optimized together. The questions of how to utilize computational resources and design an efficient search space have resulted in various studies on HPO on algorithms and toolkits. Conceptually, HPOs purposes are threefold (Feurer and Hutter, 2019): to reduce the costly menial work of artificial intelligence (AI) experts and lower the threshold of research and development; to improve the accuracy and efficiency of neural network training (Melis et al., 2017); and to make the choice of hyper-parameter set more convincing and the training results more reproducible (Bergstra et al., 2013).

In recent years, HPO has become increasingly necessary because of two rising trends in the development of deep learning models. The first trend is the upscaling of neural networks for enhanced accuracy (Tan and Le, 2019). Empirical studies have indicated that in most cases, more complex machine learning models with deeper and wider layers work better than do those with simple structures (He et al., 2016; Zagoruyko and Komodakis, 2016; Huang et al., 2019). The second trend is to design a tricky lightweight model to provide satisfying accuracy with fewer weights and parameters (Ma et al., 2018; Sandler et al., 2018; Tan et al., 2019). In this case, it is more difficult to adapt empirical values because of the stricter choices of hyper-parameters. Hyper-parameter tuning plays an essential role

in both cases: a model with a complex structure indicates more hyper-parameters to tune, and a model with a carefully designed structure implies that every hyper-parameter must be tuned to a strict range to reproduce the accuracy. For a widely used model, tuning its hyper-parameters by hand is possible because the ability to tune by hand depends on experience and researchers can always borrow knowledge from previous works. This is similar for models at a small scale. However, for a larger model or newly published models, the wide range of hyper-parameter choices requires a great deal of menial work by researchers, as well as much time and computational resources for trial and error.

In addition to research, the industrial application of deep learning is a crucial practice in automobiles, manufacturing, and digital assistants. However, even for trained professional researchers, it is still no easy task to explore and implement a favorable model to solve specific problems. Users with less experience have substantial needs for suggested hyper-parameters and ready-to-use HPO tools. Motivated by both academic needs and practical application, automated hyper-parameter tuning services (Golovin et al., 2017; Amazon, 2018) and toolkits (Liaw et al., 2018; Microsoft, 2018) provide a solution to the limitation of manual deep learning designs.

This study is motivated by the prosperous demand for design and training deep learning network in industry and research. The difficulty in selecting proper parameters for different tasks makes it necessary to summarize existing algorithms and tools. The objective of this research is to conduct a survey on feasible algorithms for HPO, make a comparison on leading tools for HPO tasks, and propose challenges on HPO tasks on deep learning networks. Thus, this remainder of this paper is structured as follows. Section 2 begins with a discussion of key hyper-parameters for building and training neural networks, including their influence on models, potential search spaces, and empirical values or schedules based on previous experience. Section 3 focuses on widely used algorithms in hyper-parameter searching, and these approaches are categorized into searching algorithms and trial schedulers. This section also evaluates the efficiency and applicability of these algorithms for different machine learning models. Section 4 provides an overview of mainstream HPO toolkits and services, compares their pros and cons, and presents some practical and implementation details. Section 5 more comprehensively compares existing HPO methods and highlights the efficient methods of model evaluation, and finally, Section 6 provides the study's conclusions.

The contribution of this study is summarized as follows:

- Hyper-parameters are systematically categorized into structure-related and training-related. The discussion of their importance and empirical strategies are helpful to determine which hyper-parameters are involved in HPO.
- HPO algorithms are analyzed and compared in detail, according to their accuracy, efficiency and scope of application. The analysis on previous studies is not only committed to include state-of-the-art algorithms, but also to clarify limitations on certain scenarios.
- By comparing HPO toolkits, this study gives insights of the design of close-sourced libraries and open-sourced services, and clarifies the targeted users for each of them.

- The potential research direction regarding to existing problems are suggested on algorithms, applications and techniques.

2. Major Hyper-Parameters and Search Space

Considering the computational resources required, hyper-parameters with greater importance receive preferential treatment in the process of HPO. Hyper-parameters with a stronger effect on weights during training are more influential for neural network training (Ng, 2017). It is difficult to quantitatively determine which of the hyper-parameters are the most significant for final accuracy. In general, there are more studies on those with higher importance, and their importance has been decided by previous experience.

Hyper-parameters can be categorized into two groups: those used for training and those used for model design. A proper choice of hyper-parameters related to model training allows neural networks to learn faster and achieve enhanced performance. Currently, the most adopted optimizer for training a deep neural network is stochastic gradient descent (Robbins and Monroe, 1951) with momentum (Qian, 1999) as well as its variants such as AdaGrad (Duchi et al., 2011), RMSprop (Hinton et al., 2012a), and Adam (Kingma and Ba, 2014). In addition to the choice of optimizer, corresponding hyper-parameters (e.g., momentum) are critical for certain networks. During the training process, batch size and LR draw the most attention because they determine the speed of convergence, the tuning of which should always be ensured. Hyper-parameters for model design are more related to the structure of neural networks, the most typical example being the number of hidden layers and width of layers. These hyper-parameters usually measure the models learning capacity, which is determined by the complexity of function (Agrawal, 2018).

This section provides an in-depth discussion on hyper-parameters of great importance for model structures and training, as well as introduce their effect in models and provide suggested values or schedules.

2.1 Learning Rate

LR is a positive scalar that determines the length of step during SGD (Goodfellow et al., 2016). In most cases, the LR must be manually adjusted during model training, and this adjustment is often necessary for enhanced accuracy (Bengio, 2012). An alternative choice of fixed LR is a varied LR over the training process. This method is referred to as the LR schedule or LR decay (Goodfellow et al., 2016). Adaptive LR can be adjusted in response to the performance or structure of the model, and it is supported by a learning algorithm (Smith, 2017; Brownlee, 2019).

Constant LR, as the simplest schedule, is often set as the default by deep learning frameworks (e.g., Keras). It is an important but tricky task to determine a good value for LR. With a proper constant LR, the network is able to be trained to a passable but unsatisfactory accuracy because the initial value could always be overlarge, especially in the final few steps. A small improvement based on the constant value is to set an initial rate as 0.1, for example, adjusting to 0.01 when the accuracy is saturated, and to 0.001 if necessary (nachiket tanksale, 2018).

Linear LR decay is a common choice for researchers who wish to set a schedule. It changes gradually during the training process based on time or step. The basic mathematical

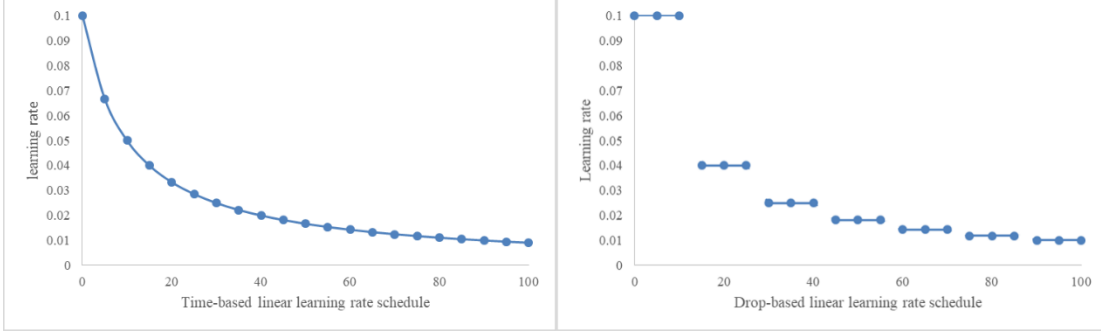


Figure 1: Linear decay of learning rate with time-based (left) and drop-based (right) schedules

function of linear decay is

$$lr = \frac{lr_0}{1 + kt}$$

where lr_0 and k are hyper-parameters for linear learning decay, indicating the initial LR and decay rate. A typical choice of t could be training time or number for self-iteration. If t is training time, the schedule of LR is a continuous change, as shown in Figure 1. If t stands for the number of iterations, the LR drops every few iterations. A typical choice is to drop the LR by half every 10 epochs (Lau, 2017) by 0.1 every 20 epochs (Li et al., 2015). This has the following mathematical form:

$$lr = \frac{lr_0}{1 + decay * selfiteration}$$

Compared with LR itself, the deep learning model is less sensitive to lr_0 and k .

Exponential decay is another widely used schedule (Li et al., 2015). Compared with linear decay, an exponential schedule provides a more drastic decay at the beginning and a gentle decay when approaching convergence (Figure 2). The basic mathematical form of exponential decay is as follows:

$$lr = lr_0 \cdot \exp(-kt)$$

where lr_0 and k are hyper-parameters for exponential learning decay. Similar to linear decay, t could be the number of self-iterations or epochs. If the number of epochs is used, the LR can be expressed with a similar form:

$$lr = lr_0 \cdot floor^{\frac{epoch}{epochsdrop}}$$

In addition to the initial LR and epoch drop rate, $floor$ is another hyper-parameter. Generally, the LR is set to drop every 10 epochs ($epochsdrop=10$) by half ($floor=0.5$), but these can be decided by the model. EfficientNet (Tan and Le, 2019) uses an exponential schedule when trained with ImageNet (Krizhevsky et al., 2012). Its choice for lr_0 , $floor$ and $epochsdrop$ are too tricky to be designed purely by hand. For example, the LR is updated

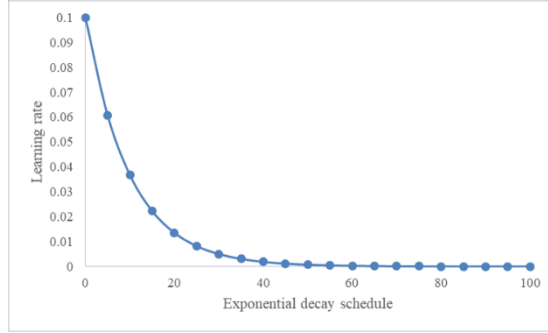


Figure 2: Exponential decay of learning rate

every 2.4 epochs ($epochsdrop=2.4$) by 0.97 ($floor=0.97$). Automatic hyper-parameter tuning is supposed to be applied in this case.

A typical LR schedule may encounter some challenges when applied to a certain model. One problem is that users must determine all hyper-parameters in the schedule in advance of training, which is a task that depends on experience. Another problem is that in the abovementioned schedules, the LR only varies with time or steps, and the same value is applied for all layers in a model. The first problem is typically solved with automated HPO or the cyclical LR method (Smith, 2017). The LR is updated in a triangle rule within a certain bound value, and the bound value is decayed in a certain cyclic schedule (Figure 3).

Regarding the second problem, this study (You et al., 2017) suggested an algorithm

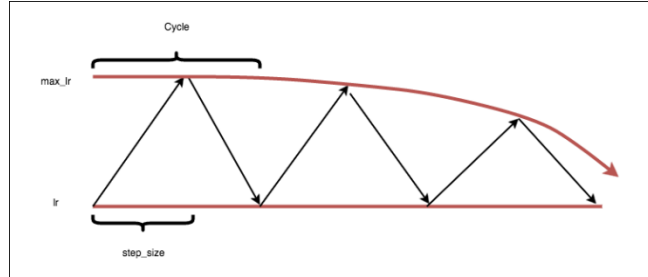


Figure 3: Learning rate decay in a cyclic schedule (source: <https://github.com/bckenstler/CLR>)

based on layer-wise adaptive rate scaling (LARS). In their study, every layer had a local LR related to its weight and gradient. Users decide how to adjust the LR according to the layers and related hyper-parameters.

In reality, selecting the optimal LR or its optimum schedule is a challenge. A small LR leads to slow convergence, whereas a large LR may prevent the model from converging (Figure 4). The LR schedule must be adjusted according to the optimizer algorithm (Lau, 2017). If the LR schedule must be determined in advance, it is recommended to simultaneously tune the hyper-parameters of the LR schedule and corresponding optimizers.

The choice of LR varies with specific tasks, but generally some tips exist that are based

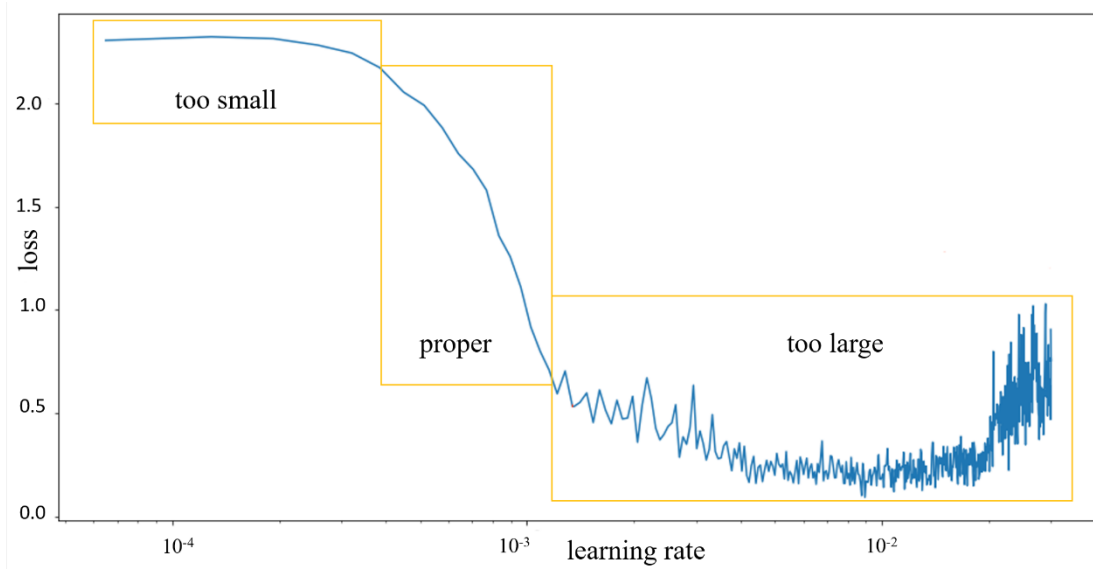


Figure 4: Effect of learning rate (adapted figure from: <https://www.jeremyjordan.me/nn-learning-rate/>)

on our experience. They can be viewed as a general rule for hyper-parameter tuning.

- In practice, it is difficult to decide the importance of a hyper-parameter if one has no experience of it. A sensitivity test is suggested (Hamby, 1994; Breierova and Choudhari, 1996) to ensure the influence of a certain hyper-parameter.
- Initial values of hyper-parameters are influential and must be carefully determined. The initial LR could be a comparatively large value because it will decay during training. In the early stage of training, a large LR will lead to fast convergence with fewer risks (Figure 4)
- Use log scale to update the LR. Thus, exponential decay could be a better choice. An exponential schedule could be applicable for many other tuning hyper-parameters, such as momentum and weight decay.
- Try more schedules. Exponential decay is not always the best choice; it depends on the model and dataset.

2.2 Optimizer

Optimizers, or optimization algorithms, play a critical role in improving accuracy and training speed. Hyper-parameters related to optimizers include the choice of optimizer, mini-batch size, momentum, and beta. Selecting an appropriate optimizer is a tricky task. This section discusses the most widely adopted optimizers (mini-batch gradient descent, RMSprop, and Adam), related hyper-parameters, and suggested values.

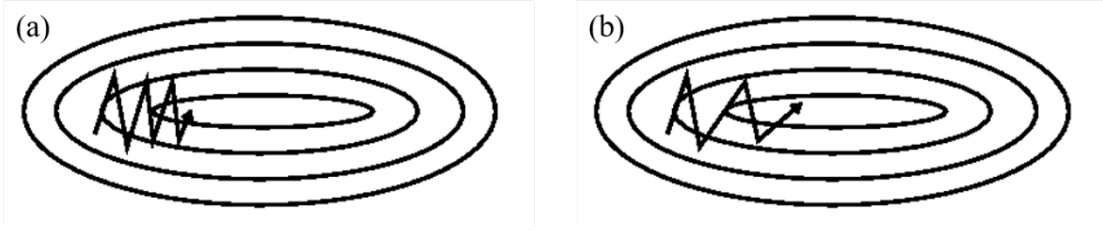


Figure 5: (a) SGD without momentum; (b) SGD with momentum (Source: <https://www.willamette.edu/gorr/classes/cs449/momrate.html>)

The aim of mini-batch gradient descent (Li et al., 2014) is to solve the following two problems. Compared with vanilla gradient descent (Li et al., 2015), mini-batch gradient descent (mini-batch GD) accelerates the training process, especially on a large dataset. Compared with SGD with a mini-batch size of 1, mini-batch GD reduces the noise and increases the probability of convergence (Ng, 2017). Mini-batch size is a hyper-parameter, and the value is highly related to the memory of the computation unit. Its value is suggested to be a power of 2 because of the access of CPU/GPU memory. The model runs faster if a power of 2 is used as the mini-batch size, and 32 could be a good default value (Bengio, 2012; Masters and Luschi, 2018). The maximum mini-batch size fitting in CPU/GPU memory. With a constant LR, researchers may find that their model oscillates within a small range but does not exactly converge in the last few steps (Figure 4). An opinion exists that for enhanced accuracy, mini-batch size and LR could be re-optimized after other hyper-parameters are fixed (Masters and Luschi, 2018). Vanilla mini-batch GD without momentum may take longer than some more recent optimizers, and the convergence relies on a robust initial value and LR. As the foundation of more advanced optimization algorithms, mini-batch GD is still used but not widely in recent publications (Ruder, 2016).

An improved method for solving the problem of oscillation and convergence speed is SGD with momentum (Loizou and Richtárik, 2017). The momentum method (Qian, 1999) accelerates the standard SGD by calculating the exponentially weighted averages for gradients (Figure 5). Furthermore, it helps the cost function go in the correct direction through adding a fraction β of the update vector:

$$v_{dw} = \beta v_{dw} + (1 - \beta) dw$$

$$w = w - lr * v_{dw}$$

where w indicates the weight. Here, momentum β is also a hyper-parameter. The momentum term is usually set as 0.9, or 0.99 or 0.999 if necessary. This reduces the oscillation by strengthening the update in the same direction and decreasing the change in different directions (Sutskever et al., 2013).

Root mean square prop (RMSprop) is one of the most widely used optimizers in the training of deep neural networks (Karpathy et al., 2016). RMSprop accelerates the gradient descent in a manner similar to Adagrad and Adadelata (Zeiler, 2012), but it exhibits superior performance when steps become smaller. It can be considered the development of Rprop (Igel and Hüsken, 2000) for mini-batch weight update. In RMSprop, the LR

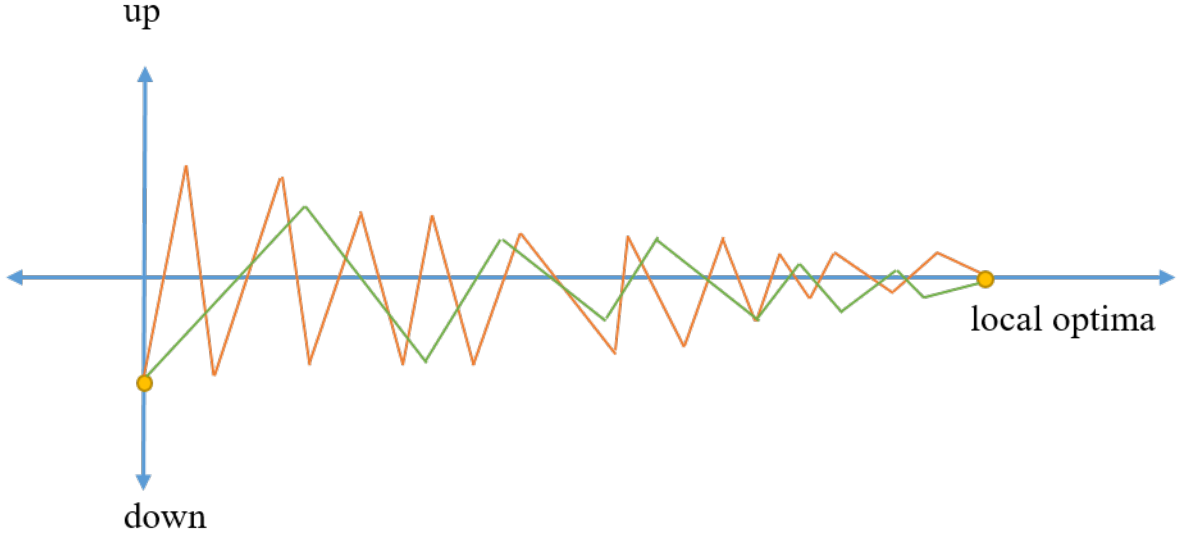


Figure 6: Comparison of SGD (orange) and RMSprop (green) for optimization

is adapted by dividing by the root of the squared gradient. Compared with the original GD, RMSprop slows the vertical oscillation and accelerates the horizontal movement from a much larger LR (Bushaev, 2018), and it is implemented as follows. RMSprop uses exponential weighted averages of the squares instead of directly using dw and db :

$$S_{dw} = \beta S_{dw} + (1 - \beta)dw^2$$

$$S_{db} = \beta S_{db} + (1 - \beta)db^2$$

$$w = w - lr * \frac{dw}{\text{sqrt}(S_{dw})}$$

where β is the moving average parameter, suggested as 0.9 (Hinton et al., 2012a). S_{dw} is relatively small and thus dw is divided by a relatively small number, whereas S_{db} is relatively large and thus db is divided by a larger number. In this way, RMSprop endures a slower movement in the bias direction (vertical) and a faster move in the weight direction (horizontal; Figure 6).

Adaptive momentum estimation (Adam) also achieves good results quickly on most neural network architectures. As a combination of GD with momentum and RMSprop, Adam adds bias-correction and momentum to RMSprop, enabling it to slightly outperform RMSprop in the late stage of optimization. It calculates the exponential weighted average as well as squares of past gradients. Parameters β_1 and β_2 control the decay rate as shown below. Moreover, Adam retains an exponential decaying average of past gradients. Its requirement for memory is in the middle of GD with momentum and RMSprop. In practice, Adam is suggested as the default optimization algorithm for deep learning training (Karpathy et al., 2016). It has more hyper-parameters compared with other optimizers, but works well with little tuning of hyper-parameters except for LR.

v_{dw} and v_{db} are updated as with GD with momentum:

$$\begin{aligned}v_{dw} &= \beta_1 v_{dw} + (1 - \beta_1) dw \\v_{db} &= \beta_1 v_{db} + (1 - \beta_1) db\end{aligned}$$

S_{dw} and S_{db} are updated as with RMSprop:

$$\begin{aligned}S_{dw} &= \beta_2 S_{dw} + (1 - \beta_2) dw^2 \\S_{db} &= \beta_2 S_{db} + (1 - \beta_2) db^2\end{aligned}$$

The biases are corrected as follows:

$$\begin{aligned}v_{dw}^{corrected} &= v_{dw} + (1 - \beta_1^t) \\v_{db}^{corrected} &= v_{db} + (1 - \beta_1^t) \\S_{dw}^{corrected} &= S_{dw} + (1 - \beta_w^t)\end{aligned}$$

weights w and biases b are updated as follows:

$$\begin{aligned}w &= w - lr * \frac{v_{dw}^{corrected}}{\text{sqrt}(S_{dw}^{corrected} + \epsilon)} \\b &= b - lr * \frac{v_{db}^{corrected}}{\text{sqrt}(S_{db}^{corrected} + \epsilon)}\end{aligned}$$

For the adjustment of LR, the previous section can be referred to, and it is reasonable to start from 0.001. β_1 is the parameter of momentum, with a suggested value of 0.9; β_2 is the parameter of RMSprop, with a suggested value of 0.999; ϵ here is used to avoid dividing by 0; and 10^{-8} is the default value. The default configurations are suitable for most problems, and therefore Adam is relatively easy to apply (Brownlee, 2017). The suggested values are adapted by most deep learning frameworks, including TensorFlow (Abadi et al., 2016), Keras (Chollet, 2018), Caffe (Jia et al., 2014), MxNet (Chen et al., 2015), and PyTorch (Paszke et al., 2019).

Generally, optimization algorithms must be adjusted along with the LR schedule. Most hyper-parameters for neural network training are highly related to optimizers and LR. RMSprop and Adam can be applied in similar situations. In most cases, Adam can act as the default optimizer (Ruder, 2016) because it is a more reliable improvement based on RMSprop. If Adam is used as the optimizer, LR must be adjusted accordingly, whereas default values are sufficient for most other hyper-parameters. Compared with RMSprop and Adam, SGD with momentum may require more time to find the optimum.

The abovementioned subsections discussed the most critical hyper-parameters related to model training. According to lecture notes of Ng (2017), the importance of hyper-parameters are listed as follows:

- LR, as in our discussion above.

- Momentum beta, for RMSprop etc.
- Mini-batch size, as in our discussion above, which is critical for all mini-batch GD.
- Number of hidden layers, which is a hyper-parameter related to model structure.
- LR decay, as in our discussion above.
- Regularization lambda, which is used to reduce variance and avoid overfitting.
- Activation functions, which is used to add nonlinear elements.
- Adam β_1 , β_2 and ϵ , as in our discussion above.

We can notice that hyper-parameters included in the list are not only related to training, but also determine the structure of model. , And therefore in the next section, we only talk about the number of hidden layers, regularization lambda, and activation function.

2.3 Model Design-Related Hyper-Parameters

The number of hidden layers(d) is a critical parameter for determining the overall structure of neural networks, which has a direct influence on the final output (Hinton et al., 2006). Deep learning networks with more layers are likely to obtain more complex features and relatively higher accuracy. Upscaling the neural network by adding more layers is a regular method of achieving better results. In this way, a baseline structure is repeated to increase the receptive field. For example, users can choose ResNet-18 to ResNet-200 according to their need for accuracy (He et al., 2016). This study (Tan and Le, 2019) created a series of neural networks with sets of depth and width hyper-parameters, obtaining high accuracy with low computational resources and a limited number of parameters.

The number of neurons in each layer(w) must also be carefully considered. Too few neurons in the hidden layers may cause underfitting because the model lacks complexity (Oppermann, 2019). By contrast, too many neurons may result in overfitting and increase the training time. The suggestions (Heaton) could be a good start for tuning the number of neurons:

$$\begin{aligned} w_{input} &< w < w_{output} \\ w &= \frac{2}{3}w_{input} + w_{output} \\ w &< 2w_{output} \end{aligned}$$

where w_{input} is the number of neurons for the input layer, and w_{output} is the number of neurons for the output layer.

In contrast to adding layers or neurons in each layer, regularization is applied to reduce the complexity of neural network models, especially those with insufficient training data. Overfitting usually occurs in complex neural networks with many layers and neurons, and the complexity can be reduced using regularization methods. A regularization term is added to avoid overfitting and address feature selections. The general expression of the

L1	L2
Penalizes the sum of the absolute value of weights	Penalizes the sum of square weights
Sparse solution	Nonsparse solution
Multiple solution	One solution
Built-in feature selection	No feature selection
Robust to outliers	Not robust to outliers
Unable to learn complex patterns	Able to learn complex data patterns

Table 1: Comparison between L1 and L2

regularization term is as follows (Ng, 2004):

$$cost = lossfunction + regularizationterm$$

The L1 term uses the sum of absolute values of weights. The regression model used for L1 regularization is also called *Lasso Regression* or L1 norm:

$$w^* = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij}W_j)^2 + \lambda \sum_{j=0}^M |W_j|$$

The L2 term uses the sum of least square values of weights. The regression model used for L2 regularization is also called *Ridge Regression* or L2 norm:

$$w^* = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij}W_j)^2 + \lambda \sum_{j=0}^M W_j^2$$

where λ is the regularization hyper-parameter. An overlarge λ will oversimplify the structure of the deep learning network because some weights are close to zero, whereas an undersized λ is not strong enough to reduce weights. In practice, L2 regularization is more widely used, mainly because of its computational efficiency; however, L1 regularization prevails over L2 on sparse properties (Yan, 2016). Models generated using L1 norm are simpler and interpretable, while L2 provides superior predictions. L1 and L2 have pros and cons, as shown in Table 1 adapted from (Khandelwal, 2018).

In addition to the abovementioned methods, data augmentation (Wang and Perez, 2017) and dropout (Srivastava et al., 2014) are commonly used regularization techniques. Data augmentation is the most straightforward, especially for image classification and object detection. The data augmentation method creates and adds fake data to the training dataset to avoid overfitting. Operations including image transformation, cropping, flipping, color adjustment, and image rotation can improve the generalization in different cases (DeVries and Taylor, 2017). For documentation recognition (LeCun et al., 1998), several transformations for enhanced model accuracy and robustness are mainly applied. For character recognition task (Bengio et al., 2011) image transformations and occlusions for their hand-written characters were combined.

Dropout is a technique used to select neurons randomly with a given probability that are not used during training, which makes the network less sensitive to the specific weights

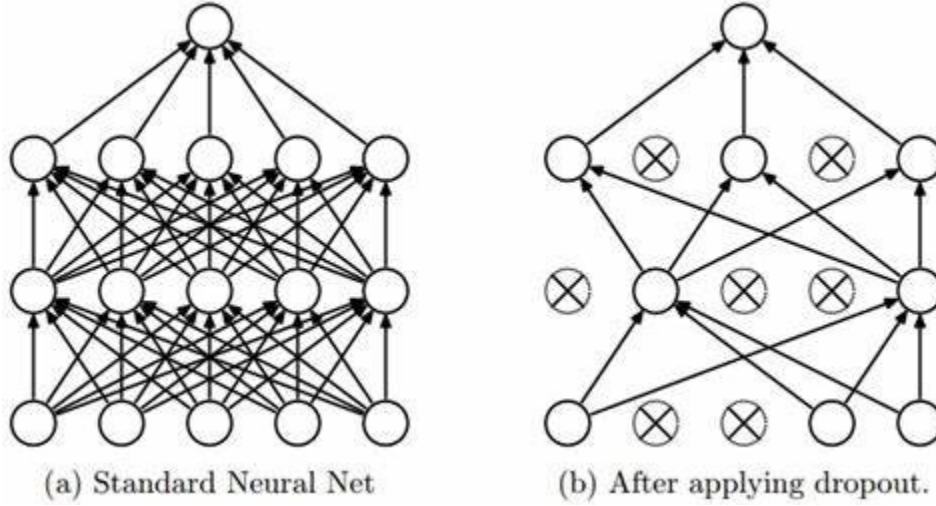


Figure 7: Comparison of a neural network before and after dropout is applied (Srivastava et al., 2014)

of neurons (Hinton et al., 2012b). As a result, the simplified version of a neuron network can reduce overfitting (Figure 7). The probability of neuron deactivation applied for each weight-updated step is suggested to be around 20%-50%. An overlarge dropout rate will over-simplify the model, whereas a small value will have little effect. In addition, a larger LR with decay and a larger momentum are suggested because fewer neurons updated with dropout requires more update for each batch (Brownlee, 2016).

Activation functions are crucial in deep learning for introducing nonlinear properties to the output of neurons. Without an activation function, a neural network will simply be a linear regression model that is unable to represent complicated features of data. Activation functions should be differentiable to compute the gradients of weight and perform back-propagation (Walia, 2017). The most popular and widely used activation functions include sigmoid, hyperbolic tangent (tanh), rectified linear units (ReLU) (Nair and Hinton, 2010), Maxout (Goodfellow et al., 2013), and Swish (Ramachandran et al., 2017a). Automatic search techniques have been applied in search of proper activation functions, including the structure of functions and related hyper-parameters (Ramachandran et al., 2017b).

Sigmoid functions are the most frequently used activation layer with a form of $f(x) = \frac{1}{1+e^{-x}}$. The output of a sigmoid function is always between 0 and 1 (Figure 8a) when negative infinite to positive infinite are encountered, and thus it will not blow up the activation. They provide nonlinearity and make clear distinctions, but in the past few years they have fallen out of popularity because of some major disadvantages. With sigmoid function, one finds that the output $f(x)$ responds very little to the input x , which gives rise to the vanishing gradient problem (Wang, 2019). This may not be a big problem for networks with a few layers, but it causes weights that are too small when more layers are required in a network. Another problem is it not being zero-centered, which means the average of data lies on the

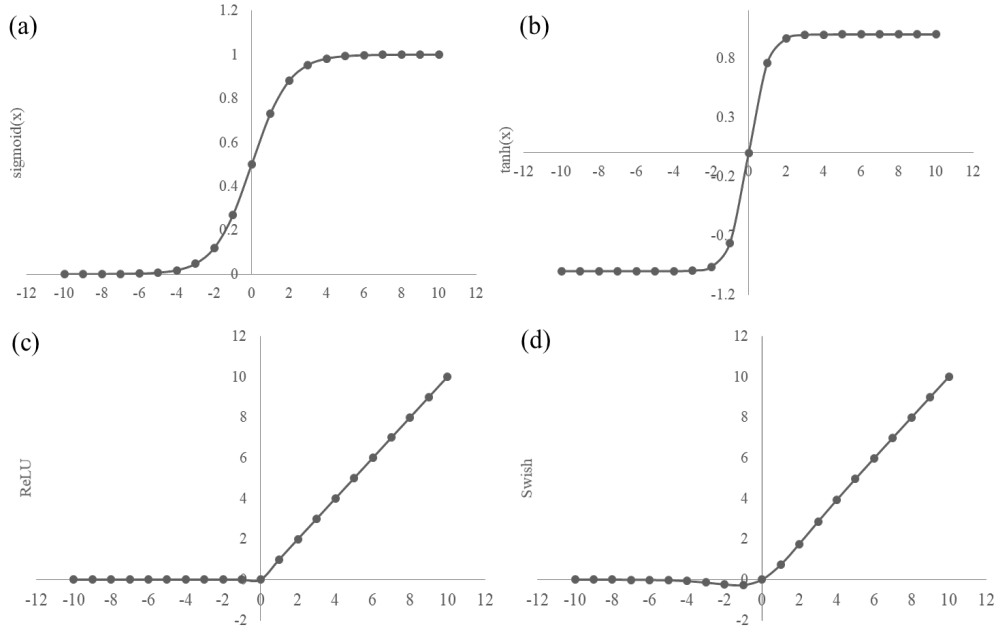


Figure 8: Comparison of activation functions: (a) sigmoid; (b) tanh; (c) ReLU; and (d) Swish

zero. During backpropagation, weights will either be all positive or all negative, making an undesirable zigzag on weights and making optimization more difficult. Softmax is a function similar to a sigmoid function with each input within $(0, 1)$. The major difference is that a sigmoid function is applied for binary classification, whereas a softmax function is for a multinomial classification. With j classes, the predicted probability of i^{th} class is as $f(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$. In addition, softmax function allows a probabilistic interpretation. With the softmax function, a high input value results in a higher probability than others, and the sum of probabilities will be 1. Whereas with the sigmoid function, a high value will lead to the high probability as well, but not the higher probability, and the sum of probabilities is not necessarily to be 1. While creating neural networks, sigmoid function is used as activation function, and softmax function is usually used in output layers.

Hyperbolic tangent (tanh) is in a form of $f(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$. It is zero-centered and the output ranges between minus one and one (Figure 8b). It is in a similar structure to a Sigmoid function but with a steeper derivative. Tanh is more efficient than a Sigmoid function and has a wider range for faster learning and grading (Kizrak, 2019), but it still has the problem of vanishing gradients. Sigmoid and tanh cannot be applied for networks with many layers.

In recent years, ReLU has become the most widely used activation function (Agarap, 2018). More advanced activation functions have arisen but ReLU and its variants (e.g., leaky ReLU, PReLU, ELU, and SeLU) are still the most typical and popular methods (Figure 8c).

$$f(x) = \begin{cases} 0, & x \leq 0 \\ x, & x > 0 \end{cases}$$

ReLU is mathematically simple and solves the problem of vanishing gradients. Compared with sigmoid and tanh, ReLU does not convert almost all neurons to be activated in a similar way, and it is great for multi-layer networks. However, ReLU has two major problems: the first is that its range is from zero to infinite, which means that it can blow up the activation, and the second is its sparsity. For a negative x , ReLU yields some weights to 0 and will never activate at any data point again (Lu et al., 2019). To solve the second problem, leaky ReLU was proposed, which uses a very small slope (e.g., $0.01x$) for a negative x to keep the update alive. Despite these problems, ReLU is suggested to be used by researchers who have little knowledge about their tasks or activation functions because it involves highly simple mathematical operations. For deep learning networks, ReLU could be the default choice in terms of ease and speed. If the problem of vanishing gradients occurs, leaky ReLU could be the alternative.

Maxout activation is a generalization of ReLU and leaky ReLU, and is defined as $f_i(x) = \max z_{ij}, z_{ij} = x^T W_{...ij} + b_{ij}, W \in \mathbf{R}^{d \times m \times k}$. It is actually a layer where the activation is the max of input. Maxout activation is a learnable activation function fixing all problems with ReLU, and it often works in combination with dropout. However, it doubles the total number of parameters because each neuron has an addition weight and bias.

Swish was proposed by Google Brain as a competitor of ReLU. It is defined as $f(x) = x \cdot \text{sigmoid}(x) = \frac{x}{1+e^{-x}}$ (Figure 8d), the multiplication of input x and sigmoid function of x . Swish has been claimed to outperform ReLU with similar simple calculations, but it is not as popular as ReLU. It has similar shape to ReLU, with bounded below and unbounded above. For a negative large x , Swish yields it into zero, whereas the values for a small negative x are captured. In addition, Swish is a smooth curve and thus it is more favorable if a smooth landscape is required.

3. Search Algorithms and Trial Schedulers on Hyper-Parameter Optimization

HPO is a problem with a long history, but it has recently become popular for the application of deep learning networks. The general philosophy remains the same: determine which hyper-parameters to tune and their search space, adjust them from coarse to fine, evaluate the performance of the model with different parameter sets, and determine the optimal combination (Strauss, 2007). This present study discusses the optimization methods for the second and third step: how to adjust hyper-parameters and how to evaluate the models performance.

For deep learning networks, the choice of hyper-parameters influences both the structure of the network (e.g., the number of hidden layers) and training accuracy (e.g., LR). The hyper-parameters could be integers, floating points, categorical data, or binary data, with a distribution of the search space. Mathematically, HPO is a process of finding a set of hyper-parameters to achieve minimum loss or maximum accuracy of an objective network.

The objective function is described as follows (Feurer and Hutter, 2019):

$$\lambda^* = \operatorname{argmin\ or\ argmax}_{\lambda} E_{(D_{train}, D_{valid}) \sim D} V(L, A_{\lambda}, D_{train}, D_{valid})$$

To maximize or minimize the V function using algorithm A_{λ} applied for hyper-parameters, the model is trained with dataset D_{train} and validated with dataset D_{valid} .

The state-of-the-art algorithms for HPO can be classified into two categories: search algorithms and trial schedulers. In general, search algorithms are applied for sampling whereas trial schedulers mainly deal with the early stopping methods for model evaluation. This section first discusses popular search algorithms followed by state-of-the-art early stopping strategies.

3.1 Search Algorithms

3.1.1 GRID SEARCH

Grid search is a basic method for HPO. It performs an exhaustive search on the hyper-parameter set specified by users. Users must have some preliminary knowledge on these hyper-parameters because it is they who generate all candidates. Grid search is applicable for several hyper-parameters with limited search space.

Grid search is the most straightforward search algorithm that leads to the most accurate predictions as long as sufficient resources are given, the user can always find the optimal combination (Joseph, 2018). It is easy to run grid search in parallel because every trial runs individually without the influence of time sequence. Results for one trial are independent of those from other trials. Computational resources can be allocated in a highly flexible manner (Figure 9). However, grid search suffers from the curse of dimensionality because the consumption of computational resources increases exponentially when more hyper-parameters are awaiting tuning. Of course, methods exist for dimensionality reduction (e.g., PCA), but any method comes with a cost on the robustness of the model (Yiu, 2019). In addition, a limited sampling range is acceptable for grid search because too many configurations are not desirable. In practice, grid search is almost only preferable when users have enough experience of these hyper-parameters to enable the definition of a narrow search space and no more than three hyper-parameters need to be tuned simultaneously.

Although other search algorithms may have more favorable features, grid search is still the most widely used method because of its mathematical simplicity (Bergstra and Bengio, 2012).

3.1.2 RANDOM SEARCH

Random search (Bergstra and Bengio, 2012) is a basic improvement on grid search. It indicates a randomized search over hyper-parameters from certain distributions over possible parameter values. The searching process continues till the predetermined budget is exhausted, or until the desired accuracy is reached. Random search is similar to grid search but has been proven to create better results because of the following two benefits (Maladkar, 2018): first, a budget can be assigned independently according to the distribution

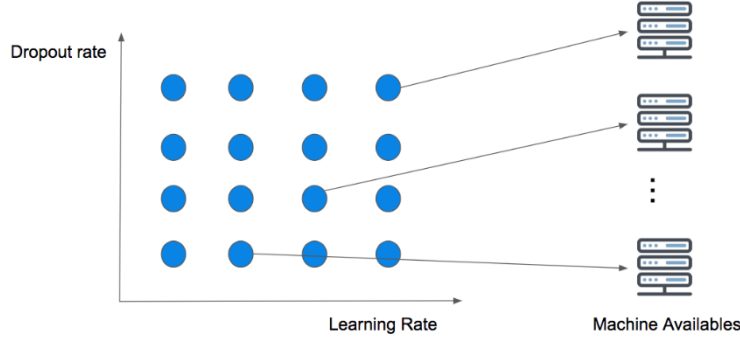


Figure 9: Grid search on dropout rate and learning rate in a parallel concurrent execution (Harrington, 2018)

of search space, whereas in grid search the budget for each hyper-parameter set is a fixed value $B^{1/N}$, Where B is the total budget and N is the number of hyper-parameters. Therefore, random search may perform better especially when some hyper-parameters are not uniformly distributed. In this search pattern, random search is comparatively more likely to find the optimal configuration than grid search (Figure 10). Second, although obtaining the optimum using random search is not promising, it is quite certain that greater time consumption will lead to a larger probability of finding the best hyper-parameter set. This logic is known as Monte Carlo techniques, which is popular when deal with large volume datasets in multi-dimensional deep learning scenarios (Harrison, 2010). By contrast, for grid search, a longer search time cannot guarantee better results. Easy parallelization and flexible resource allocation are also dominant advantages of random search (Krivulin et al., 2005).

In most cases, random search is more effective than grid search, but it is still a computationally intensive method. The use of random search is suggested in the early stage of HPO to rapidly narrow down the search space, before using a guided algorithm to obtain a finer result (from a coarse to fine sampling scheme) (Ng, 2017). Random search is often applied as the baseline of HPO to measure the efficiency of newly designed algorithms. Random search generally takes more time and computational resources than other guided search methods.

3.1.3 BAYESIAN OPTIMIZATION AND ITS VARIANTS

Bayesian optimization (BO) is a traditional algorithm with decades of history. It was raised by Mockus (Moćkus, 1975; Mockus et al., 1978), and later became popular when it was applied to the global optimization problem (Jones et al., 1998). BO is a typical method for almost all types of global optimization, and is aimed at becoming less wrong with more data (Koehrsen, 2018). It is a sequential model-based method aimed at finding the global optimum with the minimum number of trials. It balances exploration and exploitation (Silver, 2014) to avoid trapping into the local optimum. Exploitation is the process of making

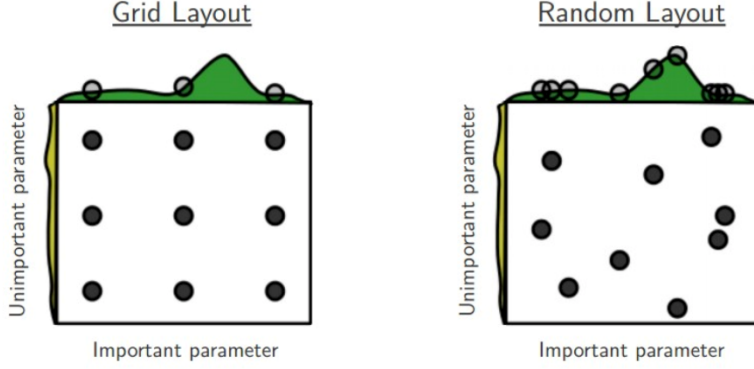


Figure 10: Layout comparison between grid search and random search (Bergstra and Bengio, 2012)

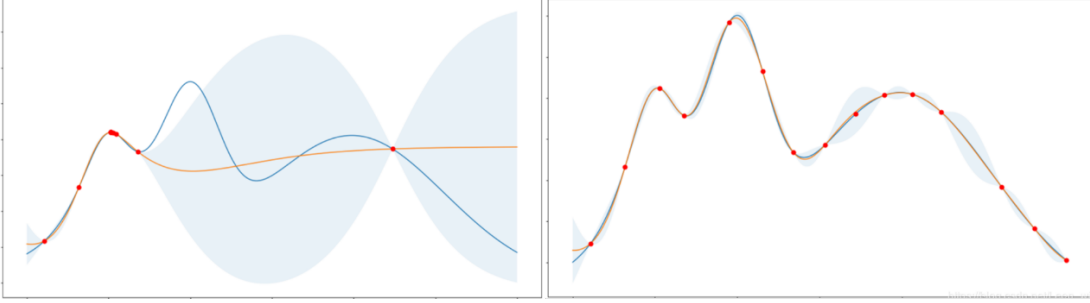


Figure 11: Exploration-oriented (left) and exploitation-oriented Bayesian optimization (right); the shade indicates uncertainty.

the best decision based on current information (Figure 11, right), whereas with exploration, the model will collect more information (Figure 11, left). BO outperforms random search and grid search in two aspects: the first is that users are not required to possess preliminary knowledge of the distribution of hyper-parameters, and the second is posterior probability, which is the core idea of BO. The process of BO is described as follows: a probability surrogate model of objectives is built, and then every attempt is made based on the assessment of previous trials, and furthermore, the probability model is updated for the next trial until the most promising hyper-parameter set is chosen. Compared with grid search and random search, BO is more computationally efficient with fewer attempts required to find the best hyper-parameter set, in particular, especially when very costly objective functions are encountered. BO has another remarkable advantage over grid search and random search it is applicable regardless of whether the objective function is stochastic or discrete, or convex or nonconvex.

This subsection presents a basic overview of the Bayesian algorithm, describes the variants based on BO, and discusses its recent applications in deep learning.

BO consists of two key ingredients: a Bayesian probability surrogate model to model the objective function, and an acquisition function to determine the next sampling point. The algorithms process can be described as follows: (1) build a prior distribution of the surrogate model; (2) obtain the hyper-parameter set that performs best on the surrogate model; (3) compute the acquisition function with the current surrogate model; (4) apply the hyper-parameter set to the objective function; and (5) update the surrogate model with new results. Steps 2-5 are repeated until the optimal configuration is found or the resource limit is reached (Brecque, 2018).

The first ingredient the surrogate model could be a specific analytic function or non-parametric model. The prior distribution describes the hypothesis on objective function, and the posterior distribution is fitted based on observational data thus far. The second ingredient the acquisition function plays a role in balancing exploration and exploitation. It ideally minimizes the loss function to select the optimal candidate points. Many probability models could be used in the BO algorithm (Eggensperger et al., 2013), but Gaussian process (GP) (Rasmussen, 2003) is overwhelmingly the most widely used. Various acquisition functions have been proposed in previous studies, including the probability of improvement (Kushner, 1964), GP upper confidence bound (GP-UCB) (Lai and Robbins, 1985; Srinivas et al., 2009), predictive entropy search (Hernández-Lobato et al., 2014), and even a portfolio containing multiple acquisition strategies (Hoffman et al., 2011). Among them, the expected improvement algorithm (Mockus et al., 1978) is definitely the most common choice. In this study, we only discussed the most popular choice of surrogate model and acquisition function. A lack of space limits further discussion on this topic; please refer to the review (Shahriari et al., 2015) for more information.

In BO, GP could be the default choice of surrogate model for objective functions (Figure 12). It is popular for two reasons: first, as a nonparametric Bayesian statistical method, GP is a method of obtaining flexible models. For a nonparametric model, the number of parameters is decided by the dataset size, without the requirement for it to be determined in advance. Second, GP takes multivariate Gaussian distribution as the prior distribution to infinite numbers of real-valued variables. In this stochastic process, any finite subset for random variables $x_1, \dots, x_n \in \chi$ follows multivariate Gaussian distribution (Do, 2007):

$$f(\lambda) \sim GP(\mu(\lambda), k(\lambda, \lambda'))$$

where $\mu(\cdot)$ is the mean vector,

$$\mu(\lambda) = E[x] = \begin{bmatrix} \mu(x_1) \\ \vdots \\ \mu(x_n) \end{bmatrix}$$

and $k(\cdot, \cdot')$ is the covariance function,

$$k(\lambda, \lambda') = E\left[\left(x - \mu(x)\right)\left(x' - \mu(x')\right)\right] = \begin{bmatrix} k(x_1, x_1) & \dots & k(x_1, x_m) \\ \vdots & \ddots & \vdots \\ k(x_m, x_1) & \dots & k(x_m, x_m) \end{bmatrix}$$

for any $x, x' \in X$. Similar to a Gaussian distribution defined by mean and covariance, a Gaussian process is a high-dimensional multivariate Gaussian defined with the mean vec-

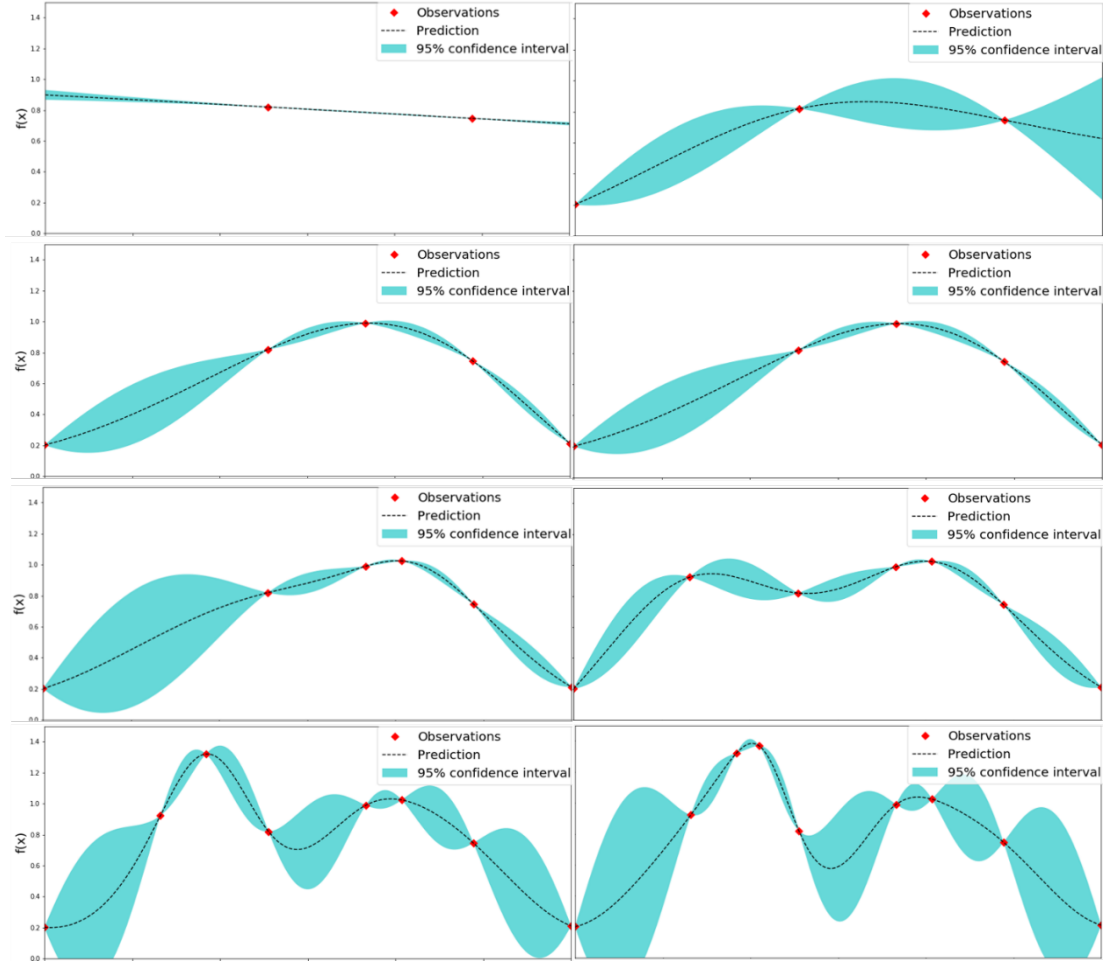


Figure 12: Gaussian process with 29 samplings

tor and covariance matrix. The mean function controls the smoothness and amplitude of samples while the choice of covariance function or kernel function $k(\lambda, \lambda')$ determines the quality of the surrogate model (Rasmussen, 2003). The most commonly used mean function is $\mu(\lambda) = 0$, and the most popular kernel is the squared exponential kernel or Gaussian kernel $k_S E(x, x') = \exp(-\frac{\|x\|^2}{2l^2})$. Matérn 5/2 kernel is widely used because of its flexibility (Duvenaud, 2014) with learnable length scale parameters are common default kernel functions for many applications (e.g., Spearmint and MOE). Kernel function can be combined in various ways, and thus two or more kernels can be used simultaneously for GP.

Prediction following a normal distribution $p(y|x, D) = N(y|\hat{\mu}, \hat{\sigma}^2)$ (Fasshauer and McCourt, 2015) with $\mu_0(x) = 0$ has a posterior distribution of

$$\begin{aligned}\mu_n(x) &= k(x)^T (K + \sigma^2 I)^{-1} y \\ \sigma_n^2(x) &= k(x, x) - k(x)^T (K + \sigma^2 I)^{-1} k(x)\end{aligned}$$

When the surrogate model is chosen, posterior distribution at any point could be determined accordingly by the mean function and kernel function. Mean indicates the expected results: a larger mean value implies a higher possibility of the optimum option. Kernel indicates the uncertainty: a larger covariance value implies that exploration is worthwhile. To avoid becoming trapped into the local optimum, whether choosing the location of the next sampling point with a larger mean or larger uncertainty is determined using acquisition functions.

In summary, GP is an attractive surrogate model for BO, which allows for the quantification of uncertainty in predictions. It is a nonparametric model and its number of parameters only depends on the input points. With a proper kernel function, GP is able to take advantage of the data structure. However, GP also has some drawbacks. For example, it is conceptually difficult to understand along with the theory of BO. Furthermore, its poor scalability with high dimensions or large number of data points is another key problem (Feurer and Hutter, 2019). Most kernels have hyper-parameters that determine the prior of a Gaussian process. Moreover, the choice of prior highly influences the performance for a given amount of data. GP itself is a computational resource-consuming process with the $O(N^3)$ computation cost. Computing the kernel matrix costs $O(DN^3)$ with $O(N^2)$ memory (Murray, 2016).

Acquisition function is a function of data point x , designed to choose the next sampling point. Acquisition functions must be carefully selected to trade off exploration over the search space and exploitation in current areas. The most common choice is the expected improvement (EI) because of its good performance and ease of use (Frazier, 2018). Its mathematical format is as follows:

$$E[I(\lambda)] = E[\max(f_{\min} - y, 0)]$$

The improvement function is defined as $I(x, v, \theta) = (v - \tau)I$. I is larger than zero when an improvement exists; v is a normally distributed random variable; and τ is the target. With the function of improvement I , the analytical expression of EI is as follows:

$$E[I(\lambda)] = E[I(x, v, \theta)] = \left(\mu_n(x) - \tau \right) \Phi\left(\frac{\mu_n(x) - \tau}{\theta_n(x)} \right) + \theta_n(x) \phi\left(\frac{\mu_n(x) - \tau}{\theta_n(x)} \right)$$

where $\sigma_n(x) > 0$, $\phi(\cdot)$ and $\Phi(\cdot)$ are the standard normal probability density function and cumulative standard normal distribution function; and $\mu_n(x)$ is the best observed value thus far (Jones et al., 1998). In addition to improvement-based policies, knowledge gradient function (Frazier et al., 2009), entropy search function (Hennig and Schuler, 2012), and predictive entropy search function (Hernández-Lobato et al., 2014) have recently been developed to enhance performance over different applications. A portfolio with acquisition functions was also proposed to cope with different strategies. It could be designed based on the hedge algorithm, using which the acquisition function was chosen with past performance (Hoffman et al., 2011). An entropy search portfolio (Shahriari et al., 2014) selects the acquisition function using the information gained toward optimization.

BO is efficient because it selects the next hyper-parameter set in a guided manner. With BO, users will make fewer calls to the objective function. For a machine learning model, BO is more efficient at finding the optimum hyper-parameter compared with grid or random search. However, there are some downsides of the vanilla Bayesian method. The most prominent problem is that BO is a sequential process in which a trial is proposed using experience from previous trials. Some solutions have been raised to solve the parallelism problem. A method was proposed based on the batch input of current results (Ginsbourger et al., 2011). As GP+EI is the most popular combination for Bayesian optimization, improvements based on EI (Ginsbourger et al., 2010; Snoek et al., 2012) and GP-UCB (Hutter et al., 2012; Desautels et al., 2014) Parallelization on other acquisition functions (e.g. information-based function) and the combination of BO with other algorithms (Falkner et al., 2018) are also feasible solutions.

Another drawback of BO is the greater consumption of computational resources compared with grid search and random search. This especially draw our attention when it is applied to deep learning networks. Memory consumption, training time, and power consumption are problems for training a neural network. Therefore, the search algorithm not only needs to be fast but also resource-efficient. A straightforward method for solving this problem is to set a boundary and determine whether a certain trial is still worthy of training. This boundary could be the loss function or average accuracy in the neural network. Domhan roposed a predictive termination method that can be combined with the Bayesian method. This idea can be combined with search algorithms other than BO (Domhan et al., 2015). Freeze-thaw Bayesian optimization (Swersky et al., 2014) utilizes a forecast curve to determine whether to freeze a current trial or thaw a previous one. Currently, BO has been applied for hyper-parameter tuning in deep learning networks, which has led to achievements in both image classification (Snoek et al., 2015) and natural language processing (Melis et al., 2017).

Bayesian methods were originally not applicable for integers, categorical values, or a conditional search space. However, in reality, hyper-parameters for deep learning could be in any data type. For vanilla Bayesian, kernel function and optimization process must be adapted to involve integers (e.g., batch size) and categorical data (Hutter, 2009; Garrido-Merchán and Hernández-Lobato, 2017). Conditional variables may only influence the results with some other variable taking certain values. This is common in deep learning networks, for example, hyper-parameters related to certain optimizers. Models with a tree-based structure have been designed to solve this problem. In addition to GP, random forests (Hutter

et al., 2011) are another popular type of surrogate model with a structure similar to a decision tree. Tree-structured Parzen Estimators (Bergstra et al., 2011, 2013) are discussed in depth in the following subsection.

3.1.4 TREE PARZEN ESTIMATORS

Tree Parzen Estimators (TPEs), models with a graphic structure that deal with the conditional search space, are an alternative choice for surrogate model. In the regular Bayes rule, the surrogate model is represented as $p(y|\lambda) = \frac{p(\lambda|y)p(y)}{p(\lambda)}$ of observation y given configuration. Furthermore, $p(\lambda|y)$ is the probability of hyper-parameters given the score on the objective function. TPE convert $p(\lambda|y)$ into a tree-structured expression:

$$p(\lambda|y) = \begin{cases} l(\lambda), & y < 0 \\ g(\lambda), & y \geq 0 \end{cases}$$

where α is the threshold dividing observations into good and bad. The percentage is usually set to 15%, and therefore the hyper-parameters have two different distributions, $l(\lambda)$ and $g(\lambda)$. Instead of directly drawing a value from $l(\lambda)$, TPE uses the ratio of $l(x)/g(x)$ to maximize the expected improvement acquisition function:

$$EI_{\alpha}(\lambda) = \frac{\gamma \alpha l(\lambda) - l(\lambda) \int_{-\infty}^{\alpha} p(y) dy}{\gamma l(\lambda) + (1 - \gamma) g(\gamma)} \propto \left(\gamma + \frac{g(\lambda)}{l(\lambda)} (1 - \gamma) \right)^{-1}$$

In summary, the workflow of TPE is as follows: draw a sample from $l(x)$, evaluate $l(x)/g(x)$ with EI, and obtain the optimum value of $l(x)/g(x)$ with the greatest EI. TPE outperforms the Bayesian method in dealing with conditional variables because of its tree structure (Strobl et al., 2008).

HyperOpt (Komer et al., 2014) realizes asynchronous parallelization and uses TPE to facilitate the HPO. BOHB combined TPE with HyperBand and successfully solved the problems of parallelization and conditional space. TPE is even more popular than the original Bayesian method when applied to deep learning networks because it is applicable for more data types. However, GPs still performance better when hyper-parameters have strong interaction because TPE do not model the interactions (Bissuel, 2018).

3.2 Optimization with an Early-stopping Policy

HPO is often a time-consuming process that is computationally costly. In a realistic scenario, it is necessary to design the HPO process with limited available resources. When experts tune hyper-parameters by hand, they are occasionally able to use their experience of parameters to narrow down the search space and evaluate the model during training, and determine whether to halt the training or continue it. This is especially common for neural networks because it takes a long time to train a model with each hyper-parameter set. An early stopping strategy is a series of methods that mimics the behavior of AI experts to maximize the computational-resource budget for promising hyper-parameter sets.

In addition to efficient search algorithms, a strategy to evaluate trials and determine whether to halt them early is another active area of research. The following early-stopping

algorithms can be applied in combination with a search algorithm or individually for both searching and halting. The early-stopping algorithms for HPO are similar to that for neural network training, but for neural network training, early stopping is applied to avoid over-fitting. This allows users to terminate a trial before finishing the whole training, thereby freeing computational resources for trials with a promising hyper-parameter set.

3.2.1 MEDIAN STOPPING

Median stopping is the most straightforward early termination policy, adapted by structured HPO frameworks such as Google Vizier (Golovin et al., 2017), Tune (Liaw et al., 2018), and NNI (Microsoft, 2018). It is model-free and applicable to a wide range of performance curves. Median stopping makes a decision based on the average of primary metrics (e.g., accuracy or loss) reported by previous runs. A trial X is stopped at step S if the best objective value by step S is strictly worse than the median value of the running average of all completed trials objective values reported at step S (Golovin et al., 2017). The median stopping strategy is not a model, and thus no hyper-parameters need to be determined by users.

3.2.2 CURVE FITTING

Curve fitting is an LPA (learning, predicting, assessing) algorithm (Kohavi and John, 1995; Provost et al., 1999). It is applicable in the combinations of search algorithms mentioned in Section 2, supported by Google Vizier and NNI. This early stopping rule makes a prediction of the final objective value (e.g., accuracy or loss) with a performance curve regressed from a set of completed or partially completed trials. A trial X will be halted at step S if the prediction of the final objective value is sufficiently worse than the tolerant value of the optimal in the trial history.

This algorithm has been commonly supported by Bayesian parametric regression (Swersky et al., 2014; Golovin et al., 2017). A method is proposed (Domhan et al., 2015) that used a weighted probability learning curve with a combination of 11 different increasing saturated curves (Figure 13). The combination model can be expressed as:

$$f_{comb}(t|\xi) = \sum_{k=1}^K w_k f_k(t|\theta_k)$$

with new combined parameter vectors

$$\xi = (w_1, \dots, w_K, \theta_1, \dots, \theta_K, \sigma^2)$$

where w_K is the weight for each model k , θ_K is the individual model parameter, and σ^2 is the noise variance.

In general, there are three steps to determine whether to stop:

- Learning: The parameters of the curve are learned from current completed trials: first, fit each curve with the least squares method and obtain parameter θ_K . Then, filter the curve and remove outliers. Finally, adjust the weight w_K for each curve using the Markov Chain Monte Carlo sampling method.

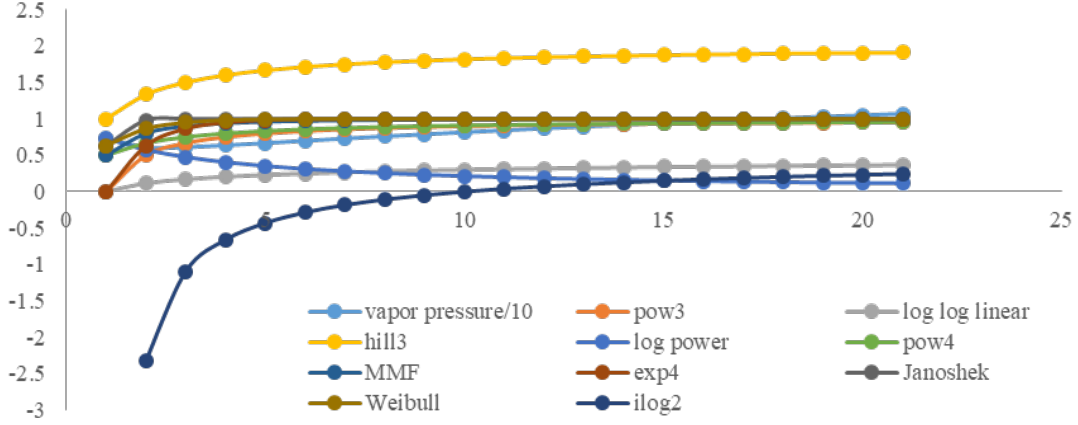


Figure 13: Functions used for curve-fitting, with related hyper-parameters $a = 1$, $b = 1$, $\alpha = 1$, $\beta = 0$, $\kappa = 1$, and $\delta = 1$ or 2 .

- Predicting: Calculate the final objective value at a certain step, with the curve obtained from the first step.
- Assessing: If the predictive result does not converge, the model will continue to train, obtain further information, and make a judgment again. If the predictive result is superior within a threshold of historical best values, the result will be retained and training stopped; otherwise, the result will be discarded and the training stopped.

Compared with median stopping, the curve fitting method is a model with parameters. Building the model is also a training process. When combined with search algorithms, the predictive termination accelerates the optimization process and then finds a state-of-the-art network. As mentioned previously, Freeze-Thaw BO can be viewed as a combination of BO and curve fitting.

3.2.3 SUCCESSIVE HALVING AND HYPERBAND

This and the next subsection discusses several bandit-based algorithms with strong performance in optimizing deep learning hyper-parameters. HPO in deep neural networks is more likely to be a tradeoff between accuracy and computational resources because training a DNN is a very costly process. Successive halving (SHA) and HyperBand outperform traditional searching algorithms without early stopping in saving resources for HPO, with random search as a sampling method and a bandit-based early stopping policy.

SHA (Jamieson and Talwalkar, 2016) is based on multi-bandit problems (Karnin et al.,

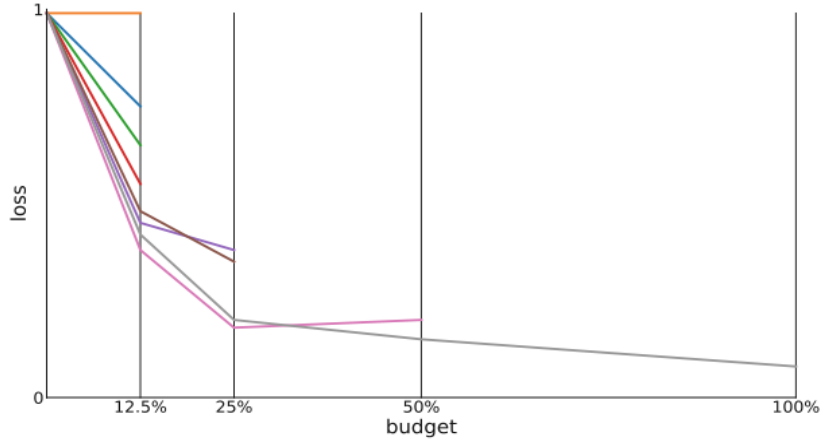


Figure 14: Budget allocation with successive halving (Bissuel, 2018)

2013). SHA converts HPO into a nonstochastic best-arm identification to allocate more computational resources to more promising hyper-parameter sets. Bandit processes are a special type of Markov Decision Process with several possible choices (Kaufmann and Garivier, 2017), and they have a long history of stochastic setting. K-armed bandit is an efficient tool for dealing with sequential decisions over time with uncertainty (Slivkins et al., 2019) and stopping rules based on the Gittins index theorem (Gittins, 1974). The SHA model was built upon the multi-armed bandit formulation (Agarwal et al., 2012; Sparks et al., 2015).

The SHA algorithm can be described as follows: users need to set an initial finite budget B and the number of trials n , uniformly query all hyper-parameter sets for a portion of initial budget, evaluate the performance of all trials, drop the worse performing half, double the budget for the remaining half, and repeat the whole pipeline until only one trial remains (Figure 14). For training a deep learning network, the budget could be the number of iterations or the total training time.

Compared with BO, SHA is theoretically easier to understand and more computationally efficient. Instead of evaluating models when they are fully trained to convergence, SHA evaluates the intermediate results to determine whether to terminate it. The major drawback of SHA is the allocation of resources. Given a certain budget, a tradeoff will occur between total budget B and the number of trials n (the n vs. B/n problem), which are both decided by users in advance. With an overlarge n , each trial will be given a small budget that may result in premature termination, while an insufficient n would not be able to provide enough optional choices. Furthermore, too large a budget may lead to a waste of choice, and too small a value may not promise an optimum.

HyperBand (HB) (Li et al., 2017) is an extension of SHA, mainly designed to solve the n vs. B/n problem by considering several possible n values with a fixed B . Similar to SHA, it also formulates the HPO into a pure-exploration, nonstochastic, infinite-armed bandit problem. HB has two improvements because it adds an out loop over the routine of SHA. The first modification is on the allocation of resources B . The out loop iterates over different

combinations of n and B , and thus different tradeoffs will occur between n and B/n . Users must set a maximum amount of resource R that can be allocated to a single trial to determine how many different combinations need to be considered. The other modification is the proportion of the trial to be discarded η . Instead of 50%, the worst fraction $\frac{\eta-1}{\eta}$ will be dropped and the budget for remaining trials multiplied by η . This is another hyper-parameter decided by users. The default value of η is 2 in SHA and 3 or 4 in HB. The suggested value and explanation of R and η were explicitly presented in the publication of HB (Li et al., 2017). R has a limitation based on computation or memory resources. Because the number of combinations S is a function of R ($S_{max} = \lfloor \log_n(R) \rfloor$), a smaller R usually indicates faster evaluation while a larger R gives a better guarantee of finding the optimal configuration. The value of η can be determined by users within a specific limit. A larger η implies a more aggressive rate for elimination and the user will receive the results faster.

HB is easy to deploy in parallel because all trials are randomly sampled and run independently. It accelerates random search through the early stopping inherited from SHA and a more reasonable adaptive allocation of resources. Compared with random search and BO, HB exhibits superior accuracy with less resources, especially in the case of stochastic gradient descent for deep neural networks. Currently, HB is involved in Tune and NNI as part of trial scheduler for the HPO framework.

3.2.4 ASYNCHRONOUS SUCCESSIVE HALVING AND BAYESIAN OPTIMIZATIONHYPERBAND

The paper on HB (Li et al., 2017) concludes by discussing potential improvements, including distributed implements, adjustment of the convergence rate, and non-random sampling. Methods discussed in this subsection can be viewed as extensions of HB. Asynchronous SHA (ASHA) (Li et al., 2018) proposes an enhanced distribution scheme for parallelizing HB and avoids straggler issues during elimination. As mentioned previously, Bayesian OptimizationHyperBand (BOHB) (Falkner et al., 2018) is a combination of BO and HB, and introduces a guided sampling method instead of simple random search (Table 2).

ASHA modifies the original HB by improving the efficiency of asynchronous parallelization. It has similar performance to HB in sequential setting or small-scale distribution. For large-scale parallelization, ASHA exceeds most state-of-the-art HPO methods. The computational resources are automatically allocated by ASHA, and thus there are no extra hyper-parameters for users compared with HB. The currently published ASHA is still an incomplete version, and it is incorporated into Tune as a trial scheduler.

HB is successful in allocating trials and resources but limited by random sampling, especially in the final few steps for HPO. Random search is effective in the early stage of sampling, but ineffective when approaching the optimum configuration. Several approaches combining BO and HB have been proposed in recent years (Bertrand et al., 2017; Falkner et al., 2018; Wang et al., 2018), to solve the native problem with HB. Among them, BOHB is the most widely applied as it is the only open-source one. The choice of budgets and schedule of trials are inherited from the original HB. The BO part closely resembles the

	SHA	HB	BOHB	ASHA
Discard ratio	half	adjusted by user (1/3)	adjusted by user	adjusted by user
B vs n	one choice	several combinations	several combinations	several combinations
Sampling	random	random	TPE	random
Parallelization	synchronous	synchronous	synchronous	asynchronous
Scalability	small scale	small scale	small scale	large scale
Time	2015	2018	2018	2018

Table 2: Comparison between bandit-based algorithms

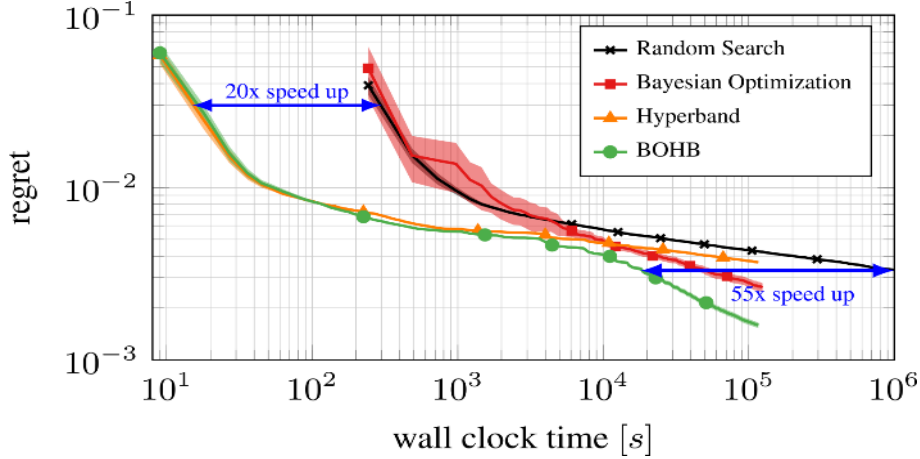


Figure 15: Comparison between BOHB and HyperBand with different budgets (Bissuel, 2018)

TPE method (Bergstra et al., 2011) with a single multidimensional Kernel Density Estimation (KDE). Compared with the hierarchy of one-dimensional KDEs used in the original TPE, the improved version can better handle interaction effects. BOHB achieves quick convergence to the best hyper-parameter set with Bayesian methods, and the feasibility of parallelization with HB.

Advantages inherited from HB include budget control, the ability to check performance at any time, computational efficiency in the early stage of tuning, and most crucially scalability. The involvement of BO ensures strong final performance, robustness, and computational efficiency in the later stage of tuning. The empirical performance of BOHB demonstrates state-of-the-art results over a wide range of HPO tasks, including Bayesian neural networks, reinforcement learning agents, and convolutional networks. From the results shown in Figure 15, BOHB can be noticed to mainly benefit from the strategy of HB if given a limited budget, and both exhibit 20-times acceleration over random search. However, with more sufficient budgets, BOHB outperforms HB because of the guided sampling strategy with the Bayesian method, exhibiting 55-times acceleration over random search. The original open-source version is HpBandSter on GitHub. In general, BOHB is a robust and computationally effective HPO method that is easy to apply in multiple units.

The major shortcoming of BOHB lies in the setting of budgets. Automatic adaption of budgets may solve the problem especially for users with less experience. The application of BOHB requires users to have some experience. In some extreme cases, either BO or the trail schedule borrowed from HB could be a drawback for efficient searching. Compared with random search, Bayesian methods may take longer to escape the local optimum (Biedenkapp, 2018). Another downside is inherited from bandit-based methods: users must define a proper budget that matches the required accuracy, but this is not an easy task because it needs sufficient experience of these algorithms. If a smaller budget is too noisy to make any decision on the model performance, adjustments to the budget plan are necessary, which is a time-consuming process. For this reason, BOHB could be several times slower than vanilla TPE, and the rate is influenced by the n vs. B/n problem. Setting a small budget is risky if one is not fully familiar with the training process or tuning algorithms. When the downside of bandit-based methods encounters the trap in the local optimum, BOHB may be less efficient compared with other optimization methods; however, this only happens in some extreme cases.

3.2.5 POPULATION-BASED TRAINING

Population-based methods are essentially series of random search methods based on genetic algorithms (Shiffman et al., 2012), such as evolutionary algorithms (Simon, 2013; Orive et al., 2014), particle swarm optimization (Eberhart and Shi, 1998; Lorenzo et al., 2017), and covariance matrix adaption evolutionary strategy (Hansen, 2016). The most important parts of genetic algorithms are initialization (random creation of a population), selection (evaluation of the current population and selection of parents), and reproduction (creation of the next generation). One of the most widely used population-based methods is population-based training (PBT) (Jaderberg et al., 2017; Li et al., 2019) proposed by DeepMind. PBT is a conceptually simple but computationally effective method. It is a unique method in two aspects: it allows for adaptive hyper-parameters during training and it combines parallel search and sequential optimization. These features make PBT especially suitable for HPO in deep learning networks.

The process of PBT can be simply described as being similar to generic algorithms. First, a population of trials with different hyper-parameter settings is initialized (Figure 16). The size of the population is determined by the user (2040 has been suggested in a publication) and all neural networks are trained in parallel. Then, for every m iterations, all models are evaluated with a certain metric, which could be loss or median accuracy. The parallelization is inherited from traditional genetic algorithms. Finally, each trial uses the information from the rest of the population to update the hyper-parameters through exploitation and exploration; their balance is similar to the concepts of hand tuning and BO. Exploitation of the best configuration is defined by replacing the current weights with weights that have the best performance. This process is similar to inheritance and crossover in genetic algorithms. Exploration is ensured by the random perturbation of hyper-parameters with a noise defined by the user. This process is similar to mutation in generic algorithms, and the noise can also be described as the mutation rate. Exploitation and exploration are performed periodically, and training will continue with the described steps.

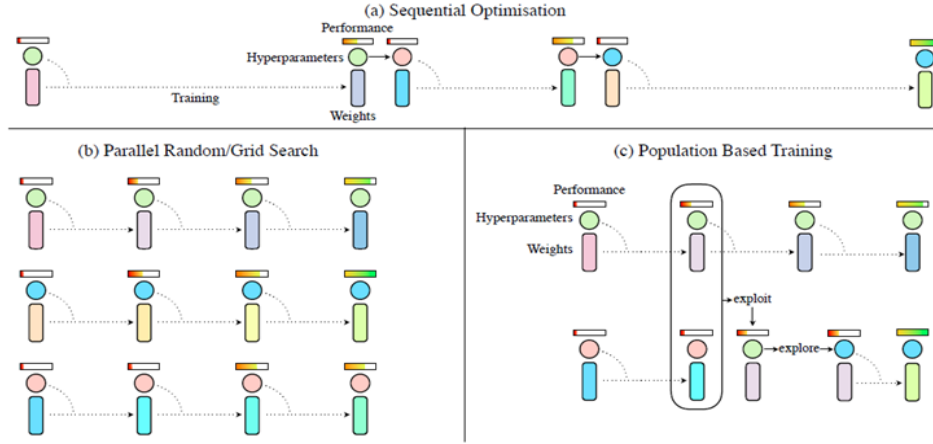


Figure 16: PBT as a combination of sequential and parallel optimization (Jaderberg et al., 2017)

PBT provides a way to involve HPO in regular training, using a warm start instead of waiting for convergence. This is meaningful for large models, such as the training for a generative adversarial network (GAN) (Goodfellow et al., 2014) and a transformer network (Vaswani et al., 2017). It takes a long time to train these large models to convergence. In this case, PBT will be far more effective than the abovementioned HPO methods, and it is the only algorithm using transformer as a show case to prove its effectiveness. For the optimization of adaptive hyper-parameters (e.g., LR), it is unnecessary for users to decide whether to use linear decay or exponential decay because PBT will change the values of all hyper-parameters periodically.

There are still some drawbacks to be explored in the future. For example, this method is not theoretically proven to obtain the optimal hyper-parameter set. It is convincing that PBT provides the best configuration with current computational resources, but whether it is the optimum configuration is still open to discussion. Another problem is the comparatively easy strategy for exploitation and exploration. PBT is still not extendable to advanced evolution and mutation decisions. In addition, the definition of hyper-parameters and the changes made to the computation graph are complicated.

Algorithms can be viewed as theories supporting HPO. For practical application, users always need to decide which hyper-parameters take into consideration, the way to apply the search algorithms and the process to train deep learning models. In this case, a tool is necessary to conduct these configurations.

4. Toolkits for Hyper-parameter Optimization

Hyper-parameter training is a time-consuming process, especially for deep learning networks. It may take decades of GPU days to finish the whole process because training a single neural network to convergence will take a whole day. A toolkit for HPO builds a bridge between network training and hyper-parameter tuning. The combination of tuning algorithm, scheduler for trials, and compatibility with major deep learning toolkits expedites the optimization process and lowers the threshold for research & development.

This section provides a general comparison of contemporary open source toolkits and services, and discusses the most integrated ones in detail. The frameworks are assessed by their ability to handle resources, support state-of-the-art optimization algorithms, and schedule for trials.

4.1 Overview

Generally, there are two types of toolkits for HPO: open source tools and services that rely on cloud computing resources, and each of which are described in this subsection.

Multiple open-source libraries have been created to cater to the demand for different tasks in automatic model design, including model construction, feature engineering, and HPO. Some of them were designed for the application of the proposed algorithm. For example, HyperOpt is specifically designed for asynchronous BO (TPE) based on Gaussian processes and regression trees. HpBandSter features in the implementation of BOHB, although it also provides random search and HB. Xcessive outperforms other libraries with an interactive graphical user interface (GUI) that is beginner-friendly. Users are able to manage the training, optimization, and evaluation of models through the GUI. However, Xcessive only supports automated hyper-parameter search through Bayesian methods, and it is also inconvenient for training deep learning networks. Scikit-Optimize is a comprehensive library cited by many later frameworks. It contains Bayesian methods, random search, random forest, and some other reliable optimization algorithms. All aforementioned methods have downsides in common: their implementation on search algorithms, their support for deep learning training frameworks, and their application schedule for trials. These inhibit efficient search for hyper-parameters for deep neural networks.

In addition, Google Cloud and Amazon Web Services (AWS) provide model selection and HPO. These services are supported by massive computational resources, applicable for thousands of parallel training processes and evaluations. However, they are a closed-source infrastructure and users must pay for the service.

The tools for HPO are designed to satisfy the following considerations: ease of use, state of the art, availability, scalability, and flexibility (Golovin et al., 2017). Table 3 displays a general comparison of the open source tools and services in regard to the abovementioned considerations. More details are discussed in the following subsections.

	Vizier & Sagemaker	NNI & Tune
Ease to use	Almost no configuration by users - Friendly to starters with simple workflow - Straightforward GUI for both task management and result visualization - Error correcting capability	- Need minimal configuration by users - Preliminary knowledge on model training - Visualization with Tensorboard
Scalability	- Very high with the support of cloud service	- Usually deployed on several units
State-of-the-art	- Support classical search algorithms - Some early stopping methods - Transfer learning	- Support almost all SOTA algorithms - Not support transfer or meta learning
Availability	- Need to pay for the convenience	- Open sourced and free
Flexibility	- Extendable search algorithm - Closed sourced in infrastructure	- Able to modify everything - High flexibility for experienced users

Table 3: Comparison between closed-source services and open-source toolkits

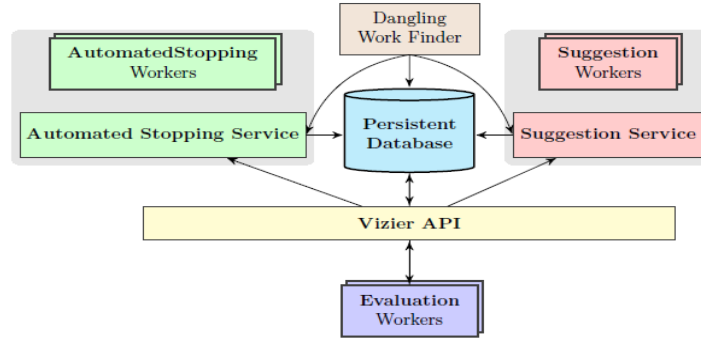


Figure 17: Architecture of Google Vizier (Golovin et al., 2017)

4.2 Google Vizier

Google Vizier (Golovin et al., 2017) is a scalable service instead of a library for black-box optimization based on Google’s Cloud Machine Learning subsystem. The most prominent advantage is its ease of use. With the service, users only need to choose an involved search algorithm and submit a configuration file, and then they will be provided with a suggested hyper-parameter set. All other setup tasks have been conducted by Google Vizier, such as the deployment of the system, management of computational resources and memory, and scheduling of trials. The simple client workflow and minimal configuration make it especially friendly for starters (Figure 17), while for experienced users it also allows them to customize their algorithm with the algorithm playground. Although Google Vizier is closed-source in infrastructure, it is easy to change or design new algorithms for HPO.

Another advantage over open-source libraries is the very high scalability. Google Vizier

is scalable to millions of experiments and billions of trials with different hyper-parameter sets. All jobs are partitioned across Google data centers and processed by Googles load-balancing infrastructure. The whole system with a strong error-correcting ability will pause a trial if it fails and prevent it from affecting the stability of the whole job.

A third benefit is the implementation of transfer learning, which helps users to accelerate their tuning processes but rarely implemented in an open-source library. This strategy, built with GP regression, is based on previous jobs submitted by millions of users. Instead of trained from scratch, the model submitted by user will be fine-tuned with provided dataset. Transfer learning is often more valuable for deep learning networks because it is expensive to train.

In addition, Google Vizier provides a benchmarking suite to measure the efficiency of algorithms with performance-over-time metrics.

Additional reason to choose Vizier is the GUI, which is applicable for both monitoring the status and changing the setting for an experiment. There is almost no technical threshold for common users because they are not even required command lines to create an experiment or request a new suggestion.

Vizier does not outperform in search algorithms and early stopping strategies. Similar to most open-source libraries, Google Vizier supports float, integer, discrete, and categorical parameters at a logarithmic scale. Grid and random search are involved, while batched Gaussian process bandit optimization (Desautels et al., 2014) is the default mode for most scenarios. Matérn kernel with an automatic relevance determination (Rasmussen, 2003) and expected improvement acquisition function (Mockus et al., 1978) are applied. The early stopping algorithms are Curve Stopping Rule (refer to 4.2.2) and Median Stopping Rule (refer to 4.2.1). In a recent study by DeepMind (Li et al., 2019), Vizier was implemented with a PBT framework. This work indicated the feasibility of implementing new optimization algorithms in Vizier, and the support of Viziers infrastructure was helpful in accelerating the tuning process. The suggested search algorithm and default values for hyper-parameters makes it friendlier for users with less experience, whereas with most open-source frameworks users must make decisions based upon their own experience.

In general, Vizier is a forerunner of a comprehensive HPO service. The key design goal is the significant reduction of setup efforts. As a beginner in hyper-parameter tuning, one can start and monitor an experiment with the dashboard, suggested with an efficient search algorithm and early stopping strategy, use the allocation of Google Cloud, and finally check the results on the UI. Very little effort is required if one is willing to use the default settings provided by Vizier. Advanced users are able to replace the algorithm with their own design, but they may still perceive a lack of flexibility. All in all, if deploying a framework and setting up an experiment generate no problems, one may not deem paying for these features to be worthwhile.

Advisor is an open-source version of Google Vizier (adv, 2018) that attempts to realize all of Viziers features without the support of Google Cloud. Grid search, random search, and BO are also supported in Advisor, as are early stop algorithms. Advisor even realizes an interactive GUI, allowing users to easily start and stop an experiment. However, the open-source version will never reproduce the extremely high scalability of Vizier because it is limited by the availability of computational resources.

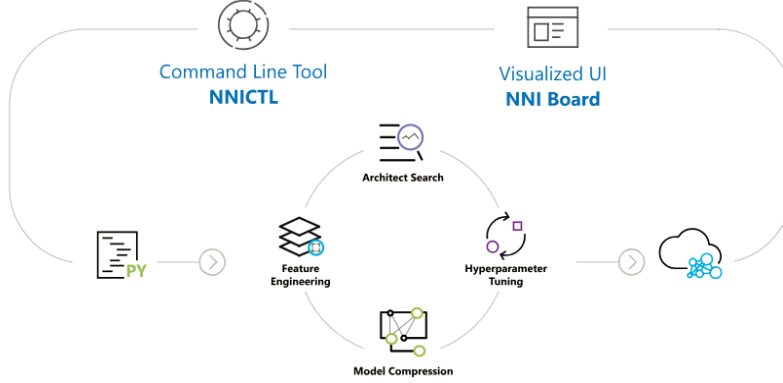


Figure 18: Structure of Neural Network Intelligence (Microsoft, 2018)

4.3 Automatic model tuning in Amazon SageMaker

Automatic model tuning is a module of Amazon SageMaker, a machine learning environment for simplifying the process of model building and deployment. As a service, it is similar to Google Vizier with the support of Amazon Web Services (AWS). The most attractive feature is also its ease-to-use: only a model and related training data are required for a HPO task. It supports optimization for complex models and datasets with parallelism on a large scale. The involvement of Jupyter simplifies the configuration for optimization and visualization of results. The HPO process and status of trials can be viewed with the Amazon Sagemaker console, which supports early stop and warm start. Search algorithms are limited to random search and BO, but users are provided with enough freedom to add their own algorithms. Amazon SageMakers automatic model tuning shares most of the advantages and limits of Google Vizier because both of them are cloud-based services. The major difference is that Amazon SageMaker, as a fully managed service, integrates every step of training a neural network; HPO is just a module of this service. It is not necessary to switch to different tools to build, train, and tune a model.

4.4 Neural Network Intelligence

Neural Network Intelligence (NNI) is an open-source toolkit for both automated machine learning (AutoML) and HPO released by Microsoft. Compared with the abovementioned closed-source services, NNI provides not only a framework to train a model and tune hyper-parameters but also more freedom for customization (Figure 18). This toolkit can be deployed in different environments such as local machines, remote servers, and Docker. The support from Microsoft provides promising maintenance and update potential. In general, it is more suitable for researchers with some knowledge of machine learning if they are willing to (a) perform the deployment job by themselves; (b) test more algorithms or networks in a different environment; and (c) involve HPO or AutoML in the NNI platform.

This section discusses the main advantages and drawbacks of NNI. NNI implements more search algorithms than Google Vizier and SageMaker, and provides an extensible in-

terface to write a new algorithm. In NNI, all algorithms related to HPO are generally called Tuners or Advisors, and Assessors contain two early stopping algorithms. Actually, there are two groups of algorithms. Tuner is similar to search algorithms, as mentioned in Section 4.1, including grid search, random search, TPE, and their variants. The other group refers to the schedulers mentioned in Section 4.2, including HB and BOHB. Almost all state-of-the-art algorithms (except PBT) are built into NNI. If a user wishes to explore the efficiency of different algorithms, NNI is a convenient choice. Tuners can be applied in combination with Assessors. Advisors already have their own early stopping strategy, and thus Assessors are not applicable. All configurations for Tuners, Assessors, and Advisors are set in the *config.yml* file.

In addition, NNI is designed with high extensibility. As well as the built-in Tuners and Assessors, researchers can test new self-designed algorithms. They only need to inherit the base Tuner/Assessor, implement related functions, and configure the new algorithm into the *config.yml* file.

NNI is able to be deployed in all kinds of infrastructure, including local machines, remote servers, and Kubernetes-based Dockers. It is extensible to implement more machine learning frameworks, both by users and developers of NNI. In addition, NNI is compatible with most popular deep learning frameworks and libraries, including PyTorch, Keras, TensorFlow, MXNet, Scikit-learn, and XGBoost. There are also extensible points for new platforms in NNI.

When an HPO experiment is initialized, its process and intermediate results can be monitored through a visualization interface named Web UI (Figure 19). Web UI displays almost all information in need of AutoML and HPO, including the status of the experiment and each trial, search space and configuration information, log, intermediate results, plots between metrics and hyper-parameters, plots showing relations between hyper-parameters, and comparisons between different trials. TensorBoard and TensorBoardX are also compatible with NNI. Compared with Web UI, plots are created by command lines in TensorBoard, which is more flexible but also requires more knowledge.

Documents and examples are helpful for starters. Examples for all search algorithms, frameworks, and hardware platforms are prepared to lower the technical threshold. NNIs developers are still working on the implementation of more algorithms and platforms, which guarantees that it will be updated for SOTA development in AutoML.

However, NNI still has some downsides. Most AutoML/HPO tools are designed for a specific algorithm (e.g., TPE) or certain tasks (e.g., image classification). The advantage is that users only need to provide labeled data, and then the model training will be performed by tools. The complexity of the model is wrapped up using HPO tools, and therefore knowledge for model training is not a must. Compared with most existing tools and services, NNI is universal for all training and optimization tasks, with any machine learning framework on any platform. This is definitely an advantage, but could be a drawback for beginners because model training experience is a prerequisite.

Furthermore, the function of Web UI could be extended. Compared with Google Vizier, NNI only displays the status and results of experiments with no interactive function. Neither the hyper-parameter setting, choice of algorithm, or configuration of resources can be adjusted from Web UI.

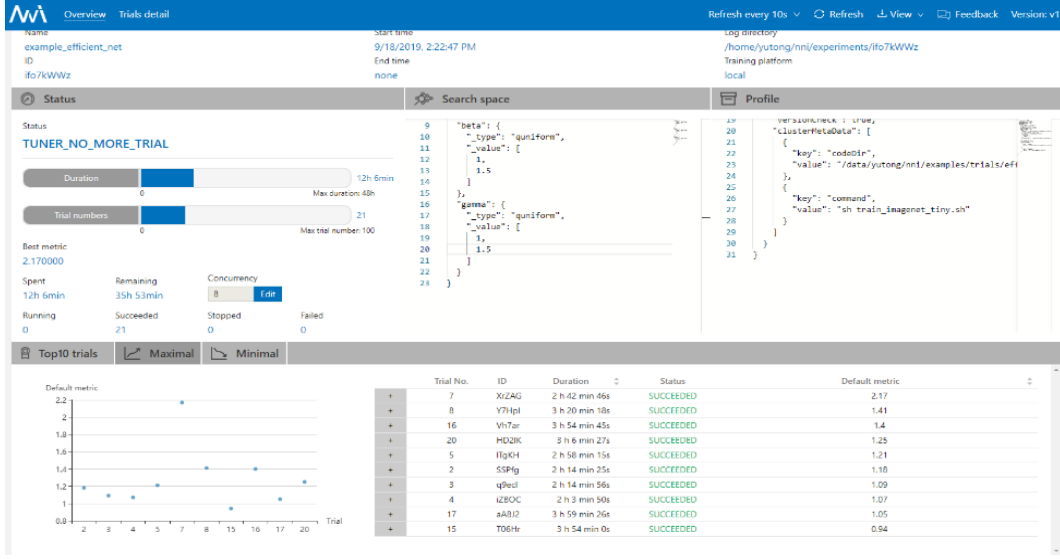


Figure 19: Example of Web UI for NNI

A third problem lies in debug information. Specifically, the debugging process could be more straightforward. Users may need more details in the log file, especially when the NNI manager or dispatcher fails.

NNI supports almost all cutting-edge search algorithms except for population-based algorithms. PBT is quite a unique algorithm in its combination of the process of hyper-parameter tuning and model training. It is especially efficient for models on a very large scale.

4.5 Ray.Tune: An open-source toolkit for hyper-parameter search

Ray.Tune is a library of Ray developed by Berkeley RISELab. As a distributed framework for model training, Ray guarantees the efficient allocation of computational resources. Most searching methods implemented in Tune can be realized in parallel. As an open-source framework, Tune shares similarities with NNI in many applications, but there are also some major differences (Table 4), which are listed as follows:

- NNI is a toolkit for both AutoML and HPO, whereas Tune is a library as a part of Ray only supporting hyper-parameter tuning.
- Currently, Tune is not applicable for Kubernetes-based Docker.
- Tune support PBT, which is a great advantage over NNI.
- Tune implements most search algorithms by importing existing libraries (e.g., BayesOpt and HyperOpt), whereas the developers of NNI rewrite most of them. Therefore, users may need to install the relevant package when using a certain algorithm.

	NNI	Tune
Application	Toolkit for AutoML , HPO and model compression	Only for HPO
Deployment	Local, remote and Dockers	Local and remote
Algorithm	Support almost all SOTA algorithms except PBT	Support almost all SOTA algorithms including PBT
Implementation	By rewriting the algorithm in format of NNI	By importing existing libraries with the interface if Tune
Start an Experiment	Import NNI+ <i>config. file</i> + search space file	Define a trainable class + <i>tune.run</i> function
Modification on original code	Less. By adding individual files	More. By build a trainable class
Visualization	Web UI and TensorBoard (TensorBoardX)	TensorBoard (TensorBoardX)

Table 4: Comparison between closed-source services and open-source toolkits

- To start an experiment, users must define a trainable class and all configurations are set in a `tune.run` function. Thus, users need to make more modifications to their original version compared with NNI, but it is not necessary to prepare individual files for configuration or search space.
- Results can be visualized using TensorBoard for both NNI and Tune. Nevertheless, NNI outperforms Tune in the design of Web UI, which is easier to use.

Tune and NNI also have advantages in common, which are as follows:

- Ease of deployment: Compared to closed-source services, users need to deploy the framework by their own, but both Tune and NNI can be deployed with only one line of command.
- Scalability: Both of them perform parallel tuning on multi computation units or servers. Users just need to add a few lines in their original code or in a certain file.
- Support: Support for more state-of-the-art hyper-parameter searching and early stopping algorithms compared with Google Vizier and SageMaker.
- Tune implements most search algorithms by importing existing libraries (e.g., BayesOpt and HyperOpt), whereas the developers of NNI rewrite most of them. Therefore, users may need to install the relevant package when using a certain algorithm.
- Open source implies flexibility: Both provide extensible interfaces for algorithms, allowing users to implement new ones. Furthermore, open source implies the lack of the need to pay for all of these features.

However, Tune and NNI also share some drawbacks. They are indeed easy to use as a comprehensive HPO framework, but not as convenient as closed-source toolkits supported

by web services. Users cannot avoid deployment and configuration work, and they cannot expect to run thousands of trials simultaneously because of the limitation of resources. Moreover, neither has an interactive GUI; users must use command lines to start adjusting an HPO experiment.

Nevertheless, Tune and NNI are still the most convenient toolkits for hyper-parameter training. In general, they are more suitable for developers with some experience of model training.

5. Discussion and Extensions

This section concludes the paper with a discussion on the comparison between algorithms (Section 5.1) and evaluations of HPO (Section 5.2).

The concept of HPO has a history of over 20 years (Kohavi and John, 1995), and it is a widely used mature technique. In the last few years, with the prosperous development of deep learning, automatic HPO has also become a potential tool to adjust a model for certain applications. Given the scale of deep learning models and their set of hyper-parameters, the key challenges concerning automatic HPO are as follows:

- Many classical search algorithms (e.g., grid search, random search, and Bayesian methods) have become infeasible because of the large number of hyper-parameters and complex configuration space.
- Evaluation of models with a certain structure or certain choice of hyper-parameters could be extremely expensive.
- Parallelism will dramatically reduce the total training time. It is a must for HPO tasks, especially for deep models.
- All in all, computational efficiency is extremely crucial.

5.1 Comparability between different algorithms

This section makes a brief comparison between major algorithms, including their advantages for saving computational resources, disadvantages in accuracy and efficiency, and their applicability (Table 5).

- *Efforts made by algorithms to save computational resources*

Random search: For deep neural networks, the most important feature of random search is computational parallelism. In addition, it is easy to combine with early stopping strategies, and such combination will largely improve the efficiency for narrowing down the search space.

BO: This is the classical method for optimization addressing the experience of prior search. It is a guided search method with surrogate model and acquisition functions, balancing the exploration and exploitation processes. Bayesian optimization and its variants are the most reliable and convincing methods for optimization. Many recent studies have

	Advantage	Disadvantage	Applicability for DNN
Grid Search	- Simple - Parallelism	- curse of dimensionality	- Applicable if only a few HPs to tune
Random search	- Parallelism - Easy to combine with early stopping methods	- Low efficiency - Cannot promise an optimum	- Convenient for early stage
Bayesian optimization	- Reliable and promising - Foundation of many other algorithms	- Difficult for parallelism - Conceptually complex	- Default algorithm for tools - Variants of BO could be more applicable (TPE)
Multi-bandit methods	- Conceptually simple - Computationally efficient	- Balance between budget and number of trials	- Could be a default choice - Implemented by open-sourced libraries.
PBT methods	- Combine HPO and model training - Parallelism	- Constant changes to computation graph - Not extendable to advanced evolution	- For computationally expensive models

Table 5: Comparison between major HPO algorithms

been conducted to fit this method into the HPO task for deep neural networks.

Multi-armed bandit algorithms: These are actually a series of algorithms based on the non-stochastic infinite-armed bandit problem, including SHA, HB, BOHB, and ASHA. They are different from the abovementioned search algorithms because they introduce an early stopping method to balance the performance and training time. The core schedule for trials is the same for these algorithms: first assign all trials a given initial budget, remove the worse half or a certain percentage, adjust the budget for the remaining trials according to the drop percentage, and repeat until only one trial is left. Trials with poor performance will be terminated halfway instead of trained to convergence to save computation time. These algorithms are more computationally efficient for dealing with increasingly complex models and increasing data size for deep learning networks. Furthermore, they are easy to deploy in parallel.

PBT: This is unique in that it combines hyper-parameter tuning and model trainings. It is a hybrid of random search (parallel optimization) and hand tuning (sequential optimization). PBT starts with several networks trained in parallel, drops trials with the worst performance, and adapts the model by refining hyper-parameters with a certain ratio and inheriting them from the model with good performance. With PBT, a model is trained to have nonstationary hyper-parameters at different stages, with warm start instead of waiting to convergence. Among all the algorithms and schedules, PBT is the most efficient because the parameters and hyper-parameters are trained simultaneously.

- In the meantime, every method has its shortcomings when applied in deep neural networks.

Grid search: The number of function evaluations grows exponentially with dimensionality and search space. Grid search will quickly become infeasible if the hyper-parameters cannot be limited in a very narrow range.

Random search: Low efficiency occurs, especially when the performance is close to the optimum. Random search only guarantees better results with more resources; it cannot promise to obtain the optimal results, which is consistent with the Monte Carlo method.

BO: The most fatal drawback of the Bayesian method is its deployment in a distributed system. As mentioned previously, parallelism is especially necessary for deep learning, while the feature of borrowing ideas from precious results prevent Bayesian method from direct parallelization. Many recent developments have solved this problem, but it is still not as natural as random search. The conceptual complexity makes the application or improvement of this algorithm challenging.

Multi-armed bandit algorithms: These have their native shortcomings in sampling, but such problems are gradually overcome with the evolution of algorithms. SHA must deal with the tradeoff between budget and number of trials, and drops half every time; HB fixes the first problem by testing several combinations of budgets and trial numbers, and users can determine the scale to drop; and BOHB replaces random search in HB with BO, thereby ensuring both strong anytime performance with early stopping and strong final performance with guided search algorithms. ASHA improves the distributed implementations, and trials can be deployed asynchronously as soon as machines free up. Another unsolved problem is the design of budgets, which is a tricky task in need of user experience of network training and model tuning.

The limitation of PBT lies in the constant changes made to the computation graph. If the model is trained with a pre-defined graph, this process could be complicated. Users can never obtain the best hyper-parameter set or a constant value for any hyper-parameter, because neither of them will be stationary during training. The evolutionary algorithm is comparatively simple for the original version of PBT; however, it is not quite extendable to advanced evolutionary decisions.

- Applicable for deep learning networks?

Grid search: Theoretically, grid search is not a good method unless only a few hyper-parameters within a narrow spectrum are searched. However, it is still quite popular in practical terms because users tend to narrow down the search space with their knowledge of some hyper-parameters, such as LR. In this case, only several potential values must be tested. In a recent study, the structural parameters determining the width and depth of EfficientNet were achieved using the grid search method (Tan and Le, 2019), which indicated the applicability of this straightforward method.

Random search: Random search is convenient to use for early stages of HPO, especially when combined with an early stopping strategy. When users have no experience in hyper-parameters, they need to narrow down the search space at the first step. The combination of random search and early stopping is efficient in this case because it will explore the whole search space with similar cost to more complicated algorithms. When the search space is narrowed down, random search is not the best choice for a nonguided method.

BO: Many improved versions have been made based on the vanilla BO method for deep neural networks, mainly in the parallelism and configuration spaces. One of the most pop-

ular variants is TPE, which is suitable for conditional hyper-parameters and natural for parallelization. It is applied in hyper-parameter tuning for convolutional neural networks and neural language modeling. BO and its variants are implemented in almost every auto-tuning framework, and it is the default search algorithm for Google Vizier and Sagemaker. This indicates that BO is still an active research area for HPO.

Multi-armed bandit algorithms: These could be a default choice for deep neural networks because of their efficiency in computational resources. For a massive parallel system, ASHA outperforms most SOTA algorithms for both language processing and model classification models. In sequential systems and distributed systems with decades of computing units, BOHB could be the first choice for tuning most neural networks and reinforcement learning algorithms because of its efficiency and accuracy. Multi-armed bandit algorithms are implemented in both NNI and Tune.

PBT: This is suitable for computationally expensive models (e.g., generative adversarial networks and transformers) and nonstationary hyper-parameters (e.g., LR). With traditional search algorithms, a model is trained to convergence with different hyper-parameter sets, and then the set with the best performance is chosen. This process could be extremely expensive for large models or large datasets, and PBT is efficient in this case because the hyper-parameters are tuned during training. For nonstationary hyper-parameters, PBT makes a schedule rather than a certain value for them, which is more practical than hand tuning or a man-made schedule.

5.2 Evaluation of results

The previous section mainly discussed the search algorithms for hyper-parameter sets, as well as their feasibility and efficiency. For a large model, the evaluation method could be more critical for saving computation time for unnecessary training because training is usually much more time-consuming compared with the sampling process. An ideal evaluator is supposed to be fast, accurate, and simple. However, the choice of evaluation is actually to seek a balance between accuracy and speed (Figure 20). In case of deep neural networks, the less time spent on training, the lower accuracy and higher variance there will be.

In the following paragraphs, we review several widely used methods and relevant algorithms for coping with the acceleration of evaluation. The most straightforward and widely used method is a direct evaluation on the target dataset. The direct method is used to evaluate hyper-parameter sets by training a corresponding model to convergence and comparing the loss or accuracy between models. This method is inefficient as it usually takes a long time between two samplings, but it is without doubt the most accurate method. Most users with limited resources cannot afford its expensive time costs, and thus faster methods have been proposed. The most straightforward idea is to decrease the fidelity to trade evaluation time (March and Willcox, 2012; Fernández-Godino et al., 2016).

Furthermore, subsampling has been adapted to deal with time consumption caused by a large training dataset. With this method, the model is trained with a subset of original training data. However, different datasets tend to have different hyper-parameter choices for optimum performance (Kohavi and John, 1995); therefore, subsampling is risky in terms of the potential to introduce more noise and uncertainty. A smaller dataset will acceler-

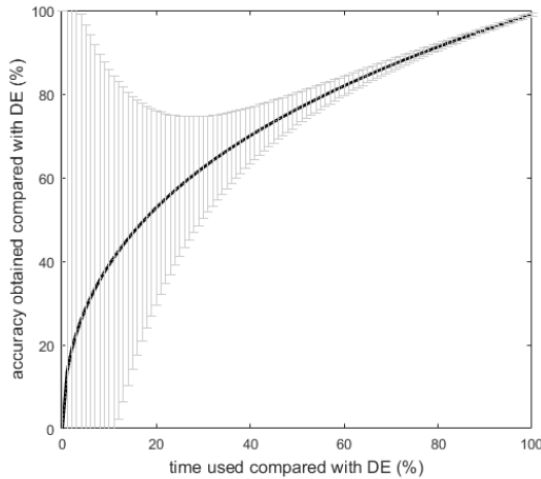


Figure 20: Example of Web UI for NNI

ate the whole process, and the size of the subset must be carefully determined to ensure accuracy. The direct application of a small dataset would definitely generate bias and low fidelity. To solve the problem posed by a small data subset, a transfer series expansion method was proposed on hyper-parameter tuning tasks to accelerate the evaluation with a combination of a set of predictors (Hu et al., 2019).

In addition, an early stopping strategy saves training time, as was discussed in Section 6. With an early stopping strategy, the whole training process is divided into sections. At each checkpoint, the model with a performance worse than a certain standard will be terminated. As previously mentioned, median stopping and curve-fitting are typical early stopping methods that can be applied in combination with random search and Bayesian methods among other. Multi-bandit-based algorithms and population-based algorithms are essentially early stopping with different methods for determining checkpoints and judgement criteria. They save training time by terminating poor models halfway, while it is still possible that a models performance is worse during the early stage but improves more than remaining ones with more training do. In this case, the model is killed too early to be good. Early stopping is the most common method used to save resources, and it is implemented in most AutoML frameworks.

Transfer learning uses knowledge from previous learning tasks to improve the learning or save resources on a target task (Pan and Yang, 2009). The target model is supposed to have a similar structure to the source model (Vanschoren, 2018; Yosinski et al., 2014); thus, transfer learning is especially feasible for hyper-parameter tuning. Parameter reuse is the most straightforward application of transfer learning in hyper-parameter tuning. The model is initialized with parameters inherited from similar configurations, and is then fine-tuned for further improvement in performance (Yosinski et al., 2014). Parameters inherited from similar evaluated configurations could be a feasible initial value to save training time (Bengio et al., 2007). The noise introduced by transfer learning is not as predictable as that introduced by subsampling and early stopping. The model could be trapped in different

local optima from different initializations, whereas a well-designed initialization performs better with a similar training process (Sutskever et al., 2013). Transfer learning has been approved to be workable for tasks with large image datasets, such as ImageNet (Sharif Razavian et al., 2014). Furthermore, transfer learning is supported by Google Vizier to accelerate the study of warm starting the model training with useful information from relevant studies (Yogatama and Mann, 2014). For a service based on Google Cloud, it is possible to borrow results from other users experiments; however, most open-source AutoML frameworks have not implemented a transfer learning module.

Meta learning is also applicable for directly predicting performance of a given model with certain hyper-parameter configurations, by building a surrogate model with previous configurations (Eggensperger et al., 2015; Baker et al., 2017). Surrogate models are approximations for making a functional relationship between available data and different types of fidelities (Fernández-Godino et al., 2016). In early works, linear regression was the most common surrogate model (Bensusan and Kalousis, 2001). With the development of computational resources, more expensive models have also been developed. The curve-fitting method (Domhan et al., 2015) discussed in subsection 5.2.2 predicted the accuracy with a weighted linear combination. In addition to the most straightforward linear regression, other machine learning algorithms have also been applied as a surrogate model. Guerra made the prediction using support vector machines (Guerra et al., 2008); Klein used Bayesian neural networks to build the surrogate model (Klein et al., 2016); and Davis the MultiLayer Perceptron to predict the performance of the model with a specific configuration (Davis and Giraud-Carrier, 2018). This method is especially suitable for HPO among all AutoML tasks because it is easy to use for quantification (Yao et al., 2018). The choice of surrogate model may introduce bias because it determines the accuracy for prediction. The accuracy of a surrogate model is influenced by sampling points available for model construction and the accuracy of sampling data (Simpson et al., 2001).

6. Conclusion

This study is motivated by the increasing application of deep neural networks. In this paper, we provided a systematic review on HPO approaches, especially on neural networks. The research began with the introduction of important hyper-parameters for training and structure, and then extended to state-of-the-art search algorithms and schedulers for automated HPO. In addition to outlining related algorithms, we covered services and open-source frameworks, including their pros and cons and applications. Comparisons of algorithms and evaluation methods were further discussed in terms of their feasibility for large models and their consumption of computational resources. This study is dedicated to summarizing information on HPO as a reference for researchers and industrial users.

References

Hyperparameter: optimization methods and real world model management. *URL*
<https://missinglink.ai/guides/neural-network-concepts/hyperparameters-optimization->

methods-and-real-world-model-management/.

Advisor. <https://github.com/tobegit3hub/advisor>, 2018.

Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, et al. Tensorflow: Large-scale machine learning on heterogeneous distributed systems. *arXiv preprint arXiv:1603.04467*, 2016.

Pieter Abbeel. Deep reinforcement learning for robotics. <https://vmayoral.github.io/robots/ai/deep/learning/rl/reinforcement/learning/2016/07/06/rl-intro/>, 2016.

Abien Fred Agarap. Deep learning using rectified linear units (relu). *arXiv preprint arXiv:1803.08375*, 2018.

Alekh Agarwal, Peter L Bartlett, and John C Duchi. Oracle inequalities for computationally adaptive model selection. *arXiv preprint arXiv:1208.0129*, 2012.

Samarth Agrawal. Hyperparameters in deep learning. <https://vmayoral.github.io/robots/ai/deep/learning/rl/reinforcement/learning/2016/07/06/rl-intro/>, 2018.

Amazon. Advisor. <https://github.com/tobegit3hub/advisor>, 2018.

Bowen Baker, Otkrist Gupta, Ramesh Raskar, and Nikhil Naik. Accelerating neural architecture search using performance prediction. *arXiv preprint arXiv:1705.10823*, 2017.

Irwan Bello, Barret Zoph, Vijay Vasudevan, and Quoc V Le. Neural optimizer search with reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning-Volume 70*, pages 459–468. JMLR. org, 2017.

Yoshua Bengio. Practical recommendations for gradient-based training of deep architectures. In *Neural networks: Tricks of the trade*, pages 437–478. Springer, 2012.

Yoshua Bengio, Pascal Lamblin, Dan Popovici, and Hugo Larochelle. Greedy layer-wise training of deep networks. In *Advances in neural information processing systems*, pages 153–160, 2007.

Yoshua Bengio, Frédéric Bastien, Arnaud Bergeron, Nicolas Boulanger-Lewandowski, Thomas Breuel, Youssouf Chherawala, Moustapha Cisse, Myriam Côté, Dumitru Erhan, Jeremy Eustache, et al. Deep learners benefit more from out-of-distribution examples. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, pages 164–172, 2011.

Hilan Bensusan and Alexandros Kalousis. Estimating the predictive accuracy of a classifier. In *European Conference on Machine Learning*, pages 25–36. Springer, 2001.

James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of machine learning research*, 13(Feb):281–305, 2012.

- James Bergstra, Daniel Yamins, and David Daniel Cox. Making a science of model search: Hyperparameter optimization in hundreds of dimensions for vision architectures. 2013.
- James S Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyperparameter optimization. In *Advances in neural information processing systems*, pages 2546–2554, 2011.
- Hadrien Bertrand, Roberto Ardon, Matthieu Perrot, and Isabelle Bloch. Hyperparameter optimization of deep neural networks: Combining hyperband with bayesian model selection. In *Conférence sur l'Apprentissage Automatique*, 2017.
- André Biedenkapp. Bohb: robust and efficient hyperparameter optimization at scale. <https://medium.com/criteo-labs/hyper-parameter-optimization-algorithms-2fe447525903>, 2018.
- Aloïs Bissuel. Hyper-parameter optimization algorithms: a short review. https://www.automl.org/blog_bohb/, 2018.
- Charles Brecque. The intuition behind bayesian optimization with gaussian processes. <https://towardsdatascience.com/the-intuitions-behind-bayesian-optimization-with-gaussian-processes-7e00fcc898a0>, 2018.
- L Breierova and M Choudhari. An introduction to sensitivity analysis. massachusetts institute of technology. Technical report, D-4526-2, 1996.
- Jason Brownlee. Dropout regularization in deep learning models with keras. <https://machinelearningmastery.com/dropout-regularization-deep-learning-models-keras/>, 2016.
- Jason Brownlee. Gentle introduction to the adam optimization algorithm for deep learning. <https://machinelearningmastery.com/adam-optimization-algorithm-for-deep-learning/>, 2017.
- Jason Brownlee. How to configure the learning rate when training deep learning neural networks. <https://machinelearningmastery.com/learning-rate-for-deep-learning-neural-networks/>, 2019.
- Vitaly Bushaev. Understanding rmsprop faster neural network learning. <https://towardsdatascience.com/understanding-rmsprop-faster-neural-network-learning-62e116fcf29a>, 2018.
- Tianqi Chen, Mu Li, Yutian Li, Min Lin, Naiyan Wang, Minjie Wang, Tianjun Xiao, Bing Xu, Chiyuan Zhang, and Zheng Zhang. Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems. *arXiv preprint arXiv:1512.01274*, 2015.
- Francois Chollet. *Deep Learning mit Python und Keras: Das Praxis-Handbuch vom Entwickler der Keras-Bibliothek*. MITP-Verlags GmbH & Co. KG, 2018.

- C Davis and C Giraud-Carrier. Annotative experts for hyperparameter selection. In *AutoML Workshop at ICML*, 2018.
- Thomas Desautels, Andreas Krause, and Joel W Burdick. Parallelizing exploration-exploitation tradeoffs in gaussian process bandit optimization. *Journal of Machine Learning Research*, 15:3873–3923, 2014.
- Terrance DeVries and Graham W Taylor. Improved regularization of convolutional neural networks with cutout. *arXiv preprint arXiv:1708.04552*, 2017.
- Chuong B Do. Gaussian processes. *Stanford University, Stanford, CA, accessed Dec*, 5: 2017, 2007.
- Tobias Domhan, Jost Tobias Springenberg, and Frank Hutter. Speeding up automatic hyperparameter optimization of deep neural networks by extrapolation of learning curves. In *Twenty-Fourth International Joint Conference on Artificial Intelligence*, 2015.
- John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of machine learning research*, 12(Jul): 2121–2159, 2011.
- David Duvenaud. *Automatic model construction with Gaussian processes*. PhD thesis, University of Cambridge, 2014.
- Russell C Eberhart and Yuhui Shi. Comparison between genetic algorithms and particle swarm optimization. In *International conference on evolutionary programming*, pages 611–616. Springer, 1998.
- Katharina Eggensperger, Matthias Feurer, Frank Hutter, James Bergstra, Jasper Snoek, Holger Hoos, and Kevin Leyton-Brown. Towards an empirical foundation for assessing bayesian optimization of hyperparameters. In *NIPS workshop on Bayesian Optimization in Theory and Practice*, volume 10, page 3, 2013.
- Katharina Eggensperger, Frank Hutter, Holger Hoos, and Kevin Leyton-Brown. Efficient benchmarking of hyperparameter optimizers via surrogates. In *Twenty-Ninth AAAI Conference on Artificial Intelligence*, 2015.
- Stefan Falkner, Aaron Klein, and Frank Hutter. Bohb: Robust and efficient hyperparameter optimization at scale. *arXiv preprint arXiv:1807.01774*, 2018.
- Gregory Fasshauer and Michael McCourt. *Kernel-based approximation methods using Matlab*, volume 19. World Scientific Publishing Company, 2015.
- M Giselle Fernández-Godino, Chanyoung Park, Nam-Ho Kim, and Raphael T Haftka. Review of multi-fidelity models. *arXiv preprint arXiv:1609.07196*, 2016.
- Matthias Feurer and Frank Hutter. Hyperparameter optimization. In *Automated Machine Learning*, pages 3–33. Springer, 2019.

- Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Springenberg, Manuel Blum, and Frank Hutter. Efficient and robust automated machine learning. In *Advances in neural information processing systems*, pages 2962–2970, 2015.
- Peter Frazier, Warren Powell, and Savas Dayanik. The knowledge-gradient policy for correlated normal beliefs. *INFORMS journal on Computing*, 21(4):599–613, 2009.
- Peter I Frazier. A tutorial on bayesian optimization. *arXiv preprint arXiv:1807.02811*, 2018.
- Eduardo C Garrido-Merchán and Daniel Hernández-Lobato. Dealing with integer-valued variables in bayesian optimization with gaussian processes. *arXiv preprint arXiv:1706.03673*, 2017.
- David Ginsbourger, Rodolphe Le Riche, and Laurent Carraro. Kriging is well-suited to parallelize optimization. In *Computational intelligence in expensive optimization problems*, pages 131–162. Springer, 2010.
- David Ginsbourger, Janis Janusevskis, and Rodolphe Le Riche. Dealing with asynchronicity in parallel gaussian process based global optimization. 2011.
- Ross Girshick. Fast r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 1440–1448, 2015.
- John Gittins. A dynamic allocation index for the sequential design of experiments. *Progress in statistics*, pages 241–266, 1974.
- Daniel Golovin, Benjamin Solnik, Subhodeep Moitra, Greg Kochanski, John Karro, and D Sculley. Google vizier: A service for black-box optimization. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, pages 1487–1495, 2017.
- Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in neural information processing systems*, pages 2672–2680, 2014.
- Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.
- Ian J Goodfellow, David Warde-Farley, Mehdi Mirza, Aaron Courville, and Yoshua Bengio. Maxout networks. *arXiv preprint arXiv:1302.4389*, 2013.
- Silvio B Guerra, Ricardo BC Prudêncio, and Teresa B Ludermir. Predicting the performance of learning algorithms using support vector machines as meta-regressors. In *International Conference on Artificial Neural Networks*, pages 523–532. Springer, 2008.
- DM Hamby. A review of techniques for parameter sensitivity analysis of environmental models. *Environmental monitoring and assessment*, 32(2):135–154, 1994.
- Mark Hammond. Deep reinforcement learning in the enterprise: Bridging the gap from games to industry, 2017.

- Nikolaus Hansen. The cma evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772*, 2016.
- Charlie Harrington. Practical guide to hyperparameters optimization for deep learning models. 2018.
- Robert L Harrison. Introduction to monte carlo simulation. In *AIP conference proceedings*, volume 1204, pages 17–21. American Institute of Physics, 2010.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- Jeff Heaton. The number of hidden layers, 2017. URL <https://www.heatonresearch.com/2017/06/01/hidden-layers.html>.
- Philipp Hennig and Christian J Schuler. Entropy search for information-efficient global optimization. *Journal of Machine Learning Research*, 13(Jun):1809–1837, 2012.
- José Miguel Hernández-Lobato, Matthew W Hoffman, and Zoubin Ghahramani. Predictive entropy search for efficient global optimization of black-box functions. In *Advances in neural information processing systems*, pages 918–926, 2014.
- Geoffrey Hinton, Nitish Srivastava, and Kevin Swersky. Neural networks for machine learning. *Coursera, video lectures*, 264:1, 2012a.
- Geoffrey E Hinton, Simon Osindero, and Yee-Whye Teh. A fast learning algorithm for deep belief nets. *Neural computation*, 18(7):1527–1554, 2006.
- Geoffrey E Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan R Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv preprint arXiv:1207.0580*, 2012b.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- Matthew D Hoffman, Eric Brochu, and Nando de Freitas. Portfolio allocation for bayesian optimization. In *UAI*, pages 327–336. Citeseer, 2011.
- Yi-Qi Hu, Yang Yu, Wei-Wei Tu, Qiang Yang, Yuqiang Chen, and Wenyuan Dai. Multifidelity automatic hyper-parameter tuning via transfer series expansion. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 3846–3853, 2019.
- Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyounJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems*, pages 103–112, 2019.
- Frank Hutter. *Automated configuration of algorithms for solving hard computational problems*. PhD thesis, University of British Columbia, 2009.

- Frank Hutter, Holger H Hoos, and Kevin Leyton-Brown. Sequential model-based optimization for general algorithm configuration. In *International conference on learning and intelligent optimization*, pages 507–523. Springer, 2011.
- Frank Hutter, Holger H Hoos, and Kevin Leyton-Brown. Parallel algorithm configuration. In *International Conference on Learning and Intelligent Optimization*, pages 55–70. Springer, 2012.
- Christian Igel. No free lunch theorems: Limitations and perspectives of metaheuristics. In *Theory and principled methods for the design of metaheuristics*, pages 1–23. Springer, 2014.
- Christian Igel and Michael Hüsken. Improving the rprop learning algorithm. In *Proceedings of the second international ICSC symposium on neural computation (NC 2000)*, volume 2000, pages 115–121. Citeseer, 2000.
- Max Jaderberg, Valentin Dalibard, Simon Osindero, Wojciech M Czarnecki, Jeff Donahue, Ali Razavi, Oriol Vinyals, Tim Green, Iain Dunning, Karen Simonyan, et al. Population based training of neural networks. *arXiv preprint arXiv:1711.09846*, 2017.
- Kevin Jamieson and Ameet Talwalkar. Non-stochastic best arm identification and hyperparameter optimization. In *Artificial Intelligence and Statistics*, pages 240–248, 2016.
- Yangqing Jia, Evan Shelhamer, Jeff Donahue, Sergey Karayev, Jonathan Long, Ross Girshick, Sergio Guadarrama, and Trevor Darrell. Caffe: Convolutional architecture for fast feature embedding. In *Proceedings of the 22nd ACM international conference on Multimedia*, pages 675–678, 2014.
- Haifeng Jin, Qingquan Song, and Xia Hu. Efficient neural architecture search with network morphism. *arXiv preprint arXiv:1806.10282*, 2018.
- Donald R Jones, Matthias Schonlau, and William J Welch. Efficient global optimization of expensive black-box functions. *Journal of Global optimization*, 13(4):455–492, 1998.
- Rohan Joseph. Grid search for model tuning. <https://towardsdatascience.com/grid-search-for-model-tuning-3319b259367e>, 2018.
- Zohar Karnin, Tomer Koren, and Oren Somekh. Almost optimal exploration in multi-armed bandits. In *International Conference on Machine Learning*, pages 1238–1246, 2013.
- Andrej Karpathy et al. Cs231n convolutional neural networks for visual recognition. *Neural networks*, 1:1, 2016.
- Gilad Katz, Eui Chul Richard Shin, and Dawn Song. Explorekit: Automatic feature generation and selection. In *2016 IEEE 16th International Conference on Data Mining (ICDM)*, pages 979–984. IEEE, 2016.
- Emilie Kaufmann and Aurélien Garivier. Learning the distribution with largest mean: two bandit frameworks. *ESAIM: Proceedings and surveys*, 60:114–131, 2017.

- R Khandelwal. L1 and l2 regularization. *Data Driven Investor*, [Online]. Available: <https://medium.com/datadriveninvestor/l1-l2-regularization-7f1b4fe948f2>, 2018.
- Ross D. King, Cao Feng, and Alistair Sutherland. Statlog: comparison of classification algorithms on large real-world problems. *Applied Artificial Intelligence an International Journal*, 9(3):289–333, 1995.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Ayyüce Kizrak. Comparison of activation functions for deep neural networks. <https://towardsdatascience.com/comparison-of-activation-functions-for-deep-neural-networks-706ac4284c8a?>, 2019.
- Aaron Klein, Stefan Falkner, Jost Tobias Springenberg, and Frank Hutter. Learning curve prediction with bayesian neural networks. 2016.
- Will Koehrsen. Comparison of activation functions for deep neural networks. <https://towardsdatascience.com/bayes-rule-applied-75965e4482ff>, 2018.
- Ron Kohavi and George H John. Automatic parameter selection by minimizing estimated error. In *Machine Learning Proceedings 1995*, pages 304–312. Elsevier, 1995.
- Brent Komer, James Bergstra, and Chris Eliasmith. Hyperopt-sklearn: automatic hyperparameter configuration for scikit-learn. In *ICML workshop on AutoML*, volume 9. Citeseer, 2014.
- Nikolai K Krivulin, Dennis Guster, and Charles Hall. Parallel implementation of a random search procedure: an experimental study. In *5th WSEAS International Conference on Simulation, Modeling and Optimization (SMO'05), Corfu Island, Greece, August 17-19, 2005*. WORLD SCIENTIFIC PUBL CO PTE LTD, 2005.
- Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, pages 1097–1105, 2012.
- Harold J Kushner. A new method of locating the maximum point of an arbitrary multipeak curve in the presence of noise. 1964.
- Tze Leung Lai and Herbert Robbins. Asymptotically efficient adaptive allocation rules. *Advances in applied mathematics*, 6(1):4–22, 1985.
- Suki Lau. Learning rate schedules and adaptive learning rate methods for deep learning. *Towards Data Science*, 2017.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

- Ang Li, Ola Spyra, Sagi Perel, Valentin Dalibard, Max Jaderberg, Chenjie Gu, David Budden, Tim Harley, and Pramod Gupta. A generalized framework for population based training. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1791–1799, 2019.
- Fei-Fei Li, Andrej Karpathy, and Justin Johnson. Convolutional neural networks for visual recognition, 2015.
- Liam Li, Kevin Jamieson, Afshin Rostamizadeh, Ekaterina Gonina, Moritz Hardt, Benjamin Recht, and Ameet Talwalkar. Massively parallel hyperparameter tuning. *arXiv preprint arXiv:1810.05934*, 2018.
- Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *The Journal of Machine Learning Research*, 18(1):6765–6816, 2017.
- Mu Li, Tong Zhang, Yuqiang Chen, and Alexander J Smola. Efficient mini-batch training for stochastic optimization. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 661–670, 2014.
- Richard Liaw, Eric Liang, Robert Nishihara, Philipp Moritz, Joseph E Gonzalez, and Ion Stoica. Tune: A research platform for distributed model selection and training. *arXiv preprint arXiv:1807.05118*, 2018.
- Nicolas Loizou and Peter Richtárik. Momentum and stochastic momentum for stochastic gradient, newton, proximal point and subspace descent methods. *arXiv preprint arXiv:1712.09677*, 2017.
- Pablo Ribalta Lorenzo, Jakub Nalepa, Michal Kawulok, Luciano Sanchez Ramos, and José Ranilla Pastor. Particle swarm optimization for hyper-parameter selection in deep neural networks. In *Proceedings of the genetic and evolutionary computation conference*, pages 481–488, 2017.
- Lu Lu, Yeonjong Shin, Yanhui Su, and George Em Karniadakis. Dying relu and initialization: Theory and numerical examples. *arXiv preprint arXiv:1903.06733*, 2019.
- Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. Shufflenet v2: Practical guidelines for efficient cnn architecture design. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 116–131, 2018.
- Kishan Maladkar. Why is random search better than grid search more machine learning. <https://towardsdatascience.com/bayes-rule-applied-75965e4482ff>, 2018.
- Andrew March and Karen Willcox. Provably convergent multifidelity optimization algorithm not requiring high-fidelity derivatives. *AIAA journal*, 50(5):1079–1089, 2012.
- Dominic Masters and Carlo Luschi. Revisiting small batch training for deep neural networks. *arXiv preprint arXiv:1804.07612*, 2018.

- Gábor Melis, Chris Dyer, and Phil Blunsom. On the state of the art of evaluation in neural language models. *arXiv preprint arXiv:1707.05589*, 2017.
- Microsoft. Neural network intelligence. <https://github.com/microsoft/nni#nni-released-reminder>, 2018.
- Jonas Moćkus. On bayesian methods for seeking the extremum. In *Optimization techniques IFIP technical conference*, pages 400–404. Springer, 1975.
- Jonas Mockus, Vytautas Tiesis, and Antanas Zilinskas. The application of bayesian methods for seeking the extremum. *Towards global optimization*, 2(117-129):2, 1978.
- I Murray. Gaussian processes and kernels, 2016.
- nachiket tanksale. Finding good learning rate and the one cycle policy. <https://towardsdatascience.com/finding-good-learning-rate-and-the-one-cycle-policy-7159fe1db5d6>, 2018.
- Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th international conference on machine learning (ICML-10)*, pages 807–814, 2010.
- Andrew Ng. Improving deep neural networks: Hyperparameter tuning, regularization and optimization. *Deeplearning. ai on Coursera*, 2017.
- Andrew Y Ng. Feature selection, l_1 vs. l_2 regularization, and rotational invariance. In *Proceedings of the twenty-first international conference on Machine learning*, page 78, 2004.
- Artem Oppermann. Overfitting and underfitting in deep learning. <https://www.deeplearning-academy.com/p/ai-wiki-overfitting-underfitting>, 2019.
- David Orive, G Sorrosal, Cruz Enrique Borges, C Martín, and A Alonso-Vicario. Evolutionary algorithms for hyperparameter tuning on neural networks models. In *Proceedings of the 26th european modeling & simulation symposium. Burdeos, France*, pages 402–409, 2014.
- Sinno Jialin Pan and Qiang Yang. A survey on transfer learning. *IEEE Transactions on knowledge and data engineering*, 22(10):1345–1359, 2009.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, pages 8024–8035, 2019.
- Foster Provost, David Jensen, and Tim Oates. Efficient progressive sampling. In *Proceedings of the fifth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 23–32, 1999.
- Ning Qian. On the momentum term in gradient descent learning algorithms. *Neural networks*, 12(1):145–151, 1999.

- Prajit Ramachandran, Barret Zoph, and Quoc V Le. Swish: a self-gated activation function. *arXiv preprint arXiv:1710.05941*, 7, 2017a.
- Prajit Ramachandran, Barret Zoph, and Quoc V Le. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017b.
- Carl Edward Rasmussen. Gaussian processes in machine learning. In *Summer School on Machine Learning*, pages 63–71. Springer, 2003.
- Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016.
- Brian D Ripley. Statistical aspects of neural networks. *Networks and chaos: statistical and probabilistic aspects*, 50:40–123, 1993.
- Herbert Robbins and Sutton Monro. A stochastic approximation method. *The annals of mathematical statistics*, pages 400–407, 1951.
- Jesus Rodriguez. Understanding hyperparameters optimization in deep learning models: Concepts and tools, 2018.
- Sebastian Ruder. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*, 2016.
- Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4510–4520, 2018.
- Bobak Shahriari, Ziyu Wang, Matthew W Hoffman, Alexandre Bouchard-Côté, and Nando de Freitas. An entropy search portfolio for bayesian optimization. *arXiv preprint arXiv:1406.4625*, 2014.
- Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P Adams, and Nando De Freitas. Taking the human out of the loop: A review of bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2015.
- Ali Sharif Razavian, Hossein Azizpour, Josephine Sullivan, and Stefan Carlsson. Cnn features off-the-shelf: an astounding baseline for recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*, pages 806–813, 2014.
- Daniel Shiffman, Shannon Fry, and Zannah Marsh. *The nature of code*. D. Shiffman, 2012.
- David Silver. Lecture 9: Exploration and exploitation. *Computer Science Department, University of London*, 2014.
- Dan Simon. *Evolutionary optimization algorithms*. John Wiley & Sons, 2013.
- Timothy W Simpson, JD Poplinski, Patrick N Koch, and Janet K Allen. Metamodels for computer-based engineering design: survey and recommendations. *Engineering with computers*, 17(2):129–150, 2001.

- Aleksandrs Slivkins et al. Introduction to multi-armed bandits. *Foundations and Trends® in Machine Learning*, 12(1-2):1–286, 2019.
- Leslie N Smith. Cyclical learning rates for training neural networks. In *2017 IEEE Winter Conference on Applications of Computer Vision (WACV)*, pages 464–472. IEEE, 2017.
- Jasper Snoek, Hugo Larochelle, and Ryan P Adams. Practical bayesian optimization of machine learning algorithms. In *Advances in neural information processing systems*, pages 2951–2959, 2012.
- Jasper Snoek, Oren Rippel, Kevin Swersky, Ryan Kiros, Nadathur Satish, Narayanan Sundaram, Mostofa Patwary, Mr Prabhat, and Ryan Adams. Scalable bayesian optimization using deep neural networks. In *International conference on machine learning*, pages 2171–2180, 2015.
- Evan R Sparks, Ameet Talwalkar, Michael J Franklin, Michael I Jordan, and Tim Kraska. Tupaq: An efficient planner for large-scale predictive analytic queries. *arXiv preprint arXiv:1502.00068*, 2015.
- Niranjan Srinivas, Andreas Krause, Sham M Kakade, and Matthias Seeger. Gaussian process optimization in the bandit setting: No regret and experimental design. *arXiv preprint arXiv:0912.3995*, 2009.
- Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.
- Martin J. Strauss. Typical steps for solving optimization problems. *Electrical Engineering and Computer Science, University of Michigan*, 2007.
- Carolin Strobl, Anne-Laure Boulesteix, Thomas Kneib, Thomas Augustin, and Achim Zeileis. Conditional variable importance for random forests. *BMC bioinformatics*, 9(1):307, 2008.
- Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. On the importance of initialization and momentum in deep learning. In *International conference on machine learning*, pages 1139–1147, 2013.
- Kevin Swersky, Jasper Snoek, and Ryan Prescott Adams. Freeze-thaw bayesian optimization. *arXiv preprint arXiv:1406.3896*, 2014.
- Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2818–2826, 2016.
- Mingxing Tan and Quoc V Le. Efficientnet: Rethinking model scaling for convolutional neural networks. *arXiv preprint arXiv:1905.11946*, 2019.

- Mingxing Tan, Bo Chen, Ruoming Pang, Vijay Vasudevan, Mark Sandler, Andrew Howard, and Quoc V Le. Mnasnet: Platform-aware neural architecture search for mobile. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2820–2828, 2019.
- Joaquin Vanschoren. Meta-learning: A survey. *arXiv preprint arXiv:1810.03548*, 2018.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017.
- Anish Singh Walia. Activation functions and its types-which is better? *Towards Data Science*, 29, 2017.
- CF Wang. The vanishing gradient problem. *Towards Data Science*, 2019.
- Jason Wang and Luis Perez. The effectiveness of data augmentation in image classification using deep learning. *Convolutional Neural Networks Vis. Recognit*, page 11, 2017.
- Jiazhao Wang, Jason Xu, and Xuejun Wang. Combination of hyperband and bayesian optimization for hyperparameter optimization in deep learning. *arXiv preprint arXiv:1801.01596*, 2018.
- David H Wolpert and William G Macready. No free lunch theorems for optimization. *IEEE transactions on evolutionary computation*, 1(1):67–82, 1997.
- Yan. L1 norm regularization and sparsity explained for dummies. <https://medium.com/mlreview/l1-norm-regularization-and-sparsity-explained-for-dummies-5b0e4be3938a>, 2016.
- Quanming Yao, Mengshuo Wang, Yuqiang Chen, Wenyan Dai, Hu Yi-Qi, Li Yu-Feng, Tu Wei-Wei, Yang Qiang, and Yu Yang. Taking human out of learning applications: A survey on automated machine learning. *arXiv preprint arXiv:1810.13306*, 2018.
- Tony Yiu. The curse of dimensionality. <https://towardsdatascience.com/the-curse-of-dimensionality-50dc6e49aa1e>, 2019.
- Dani Yogatama and Gideon Mann. Efficient transfer learning method for automatic hyperparameter tuning. In *Artificial intelligence and statistics*, pages 1077–1085, 2014.
- Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in neural information processing systems*, pages 3320–3328, 2014.
- Yang You, Igor Gitman, and Boris Ginsburg. Large batch training of convolutional networks. *arXiv preprint arXiv:1708.03888*, 2017.
- Sergey Zagoruyko and Nikos Komodakis. Wide residual networks. *arXiv preprint arXiv:1605.07146*, 2016.

Matthew D Zeiler. Adadelata: an adaptive learning rate method. *arXiv preprint arXiv:1212.5701*, 2012.

Barret Zoph and Quoc V Le. Neural architecture search with reinforcement learning. *arXiv preprint arXiv:1611.01578*, 2016.

DEEP LEARNING FOR FINANCE

MODULE 1 | LESSON 2

Multilayer Perceptron: Market Timing in AAPL Stock

Reading Time	60 minutes
Prior Knowledge	MLP, Market timing, Python
Keywords	MLP, Classification, Neural Network, Dropout

*In this lesson we will start increasing the complexity of our Neural Networks by considering more dense MLPs that incorporate features such as **Dropout** in order to enhance its performance. Also, different from the regression example we used in the Machine Learning course on timing the Momentum factor, we will deal here with a **classification** problem. Specifically, we will try to design a strategy that times the return of Apple stock ('AAPL'). For that, we will employ an MLP network that aims to predict whether the future return of AAPL is positive or negative. Based on the predictions from the model, we will later on check how our strategy performs.*

1. Data and sources

As with any other Deep Learning endeavor, the single most important and necessary ingredient is data. Different from what we did in the Momentum timing example, in this case we will retrieve Apple stock price data from Yahoo! Finance using the famous **yfinance** library in Python, which you are already familiar with:

Darell Moodley inserted this code

```
In [1]: # !pip install keras-tuner
import tensorflow as tf
from tensorflow import keras
import keras_tuner as kt
import numpy as np
```

2025-09-24 07:55:33.743440: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

2025-09-24 07:55:33.796784: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

2025-09-24 07:55:35.377415: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

```
In [2]: n_data = 5000

# Create a simple dataset for demonstration
# We'll use a regression problem, where  $y = 2x + 1$  with some noise.
X = np.random.rand(n_data, 1) * 10
y = 2 * X + 1 + np.random.randn(n_data, 1) * 2

# Split data into training and validation sets
from sklearn.model_selection import train_test_split
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_st
```

```
In [3]: def build_model(hp):
        model = keras.Sequential()

        # Input Layer
        model.add(keras.layers.Input(shape=(1,)))

        # Hyperparameter search for the number of hidden layers

        for i in range(hp.Int('num_layers', min_value=5, max_value=60, step=5)):
            model.add(tf.keras.layers.Dense(units=15, activation='relu'))

        # Output Layer
        model.add(keras.layers.Dense(units=1))

        # Compile the model
        model.compile(
            optimizer='adam',
            loss='mean_squared_error',
            metrics=['mean_absolute_error'])

        return model
```

```
In [ ]: ### correct answer
        for i in range(hp.Int('num_layers', min_value=5, max_value=60, step=5)):
            model.add(tf.keras.layers.Dense(units=15, activation='relu'))
```

```
In [4]: tuner = kt.RandomSearch(
        hypermodel=build_model,
        objective='val_mean_absolute_error',
        max_trials=5, # The number of models to test
        executions_per_trial=2, # Number of times to run each model to reduce variance
        directory='my_dir',
        project_name='tuning_hidden_layers')
```

```
# # Run the search
tuner.search(X_train, y_train, epochs=20, validation_data=(X_val, y_val))

# # Get the optimal hyperparameters
best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]
print(f"The optimal number of hidden layers is: {best_hps.get('num_layers')}")

# # Get the best model
best_model = tuner.get_best_models(num_models=1)[0]
best_model.summary()
```

Reloading Tuner from my_dir/tuning_hidden_layers/tuner0.json

The optimal number of hidden layers is: 45

/usr/local/lib/python3.11/site-packages/keras/src/saving/saving_lib.py:757: UserWarning: Skipping variable loading for optimizer 'adam', because it has 2 variables whereas the saved optimizer has 186 variables.

saveable.load_own_variables(weights_store.get(inner_path))

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 15)	30
dense_1 (Dense)	(None, 15)	240
dense_2 (Dense)	(None, 15)	240
dense_3 (Dense)	(None, 15)	240
dense_4 (Dense)	(None, 15)	240
dense_5 (Dense)	(None, 15)	240
dense_6 (Dense)	(None, 15)	240
dense_7 (Dense)	(None, 15)	240
dense_8 (Dense)	(None, 15)	240
dense_9 (Dense)	(None, 15)	240
dense_10 (Dense)	(None, 15)	240
dense_11 (Dense)	(None, 15)	240
dense_12 (Dense)	(None, 15)	240
dense_13 (Dense)	(None, 15)	240
dense_14 (Dense)	(None, 15)	240
dense_15 (Dense)	(None, 15)	240
dense_16 (Dense)	(None, 15)	240
dense_17 (Dense)	(None, 15)	240
dense_18 (Dense)	(None, 15)	240
dense_19 (Dense)	(None, 15)	240
dense_20 (Dense)	(None, 15)	240
dense_21 (Dense)	(None, 15)	240
dense_22 (Dense)	(None, 15)	240
dense_23 (Dense)	(None, 15)	240
dense_24 (Dense)	(None, 15)	240
dense_25 (Dense)	(None, 15)	240
dense_26 (Dense)	(None, 15)	240
dense_27 (Dense)	(None, 15)	240
dense_28 (Dense)	(None, 15)	240

dense_29 (Dense)	(None, 15)	240
dense_30 (Dense)	(None, 15)	240
dense_31 (Dense)	(None, 15)	240
dense_32 (Dense)	(None, 15)	240
dense_33 (Dense)	(None, 15)	240
dense_34 (Dense)	(None, 15)	240
dense_35 (Dense)	(None, 15)	240
dense_36 (Dense)	(None, 15)	240
dense_37 (Dense)	(None, 15)	240
dense_38 (Dense)	(None, 15)	240
dense_39 (Dense)	(None, 15)	240
dense_40 (Dense)	(None, 15)	240
dense_41 (Dense)	(None, 15)	240
dense_42 (Dense)	(None, 15)	240
dense_43 (Dense)	(None, 15)	240
dense_44 (Dense)	(None, 15)	240
dense_45 (Dense)	(None, 1)	16

Total params: 10,606 (41.43 KB)

Trainable params: 10,606 (41.43 KB)

Non-trainable params: 0 (0.00 B)

In []:

In []:

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
In [2]: # Read the CSV file
file_path = 'AAPL.csv'
df = pd.read_csv(file_path, delimiter=';', decimal=',', parse_dates=['Date'], da
df = df.sort_values('Date')
df = df.set_index('Date')

df.head()
```

Out[2]:

	Price	CVol	Change	% Change	% Return	Total Return (Gross)	Cumulative Return %	Open	High
Date									
1980-12-12	0.13	46903360.0	0.13	NaN	NaN	0.13	NaN	NaN	0.13
1980-12-15	0.12	17588480.0	-0.01	-5.22	-5.22	0.12	-5,22	NaN	0.12
1980-12-16	0.11	105728.0	-0.01	-7.32	-7.32	0.11	-12,16	NaN	0.11
1980-12-17	0.12	8644160.0	0.00	2.48	2.48	0.12	-9,98	NaN	0.12
1980-12-18	0.12	7344960.0	0.00	2.86	2.86	0.12	-7,40	NaN	0.12

In [3]:

```
# Rename '% Return' column to 'Ret'
df = df.rename(columns={'% Return': 'Ret'})

# Define the name variable
name = 'Ret'

# Set 'Date' as index and keep only 'Ret' column
df = df.dropna()

df.head()
```

Out[3]:

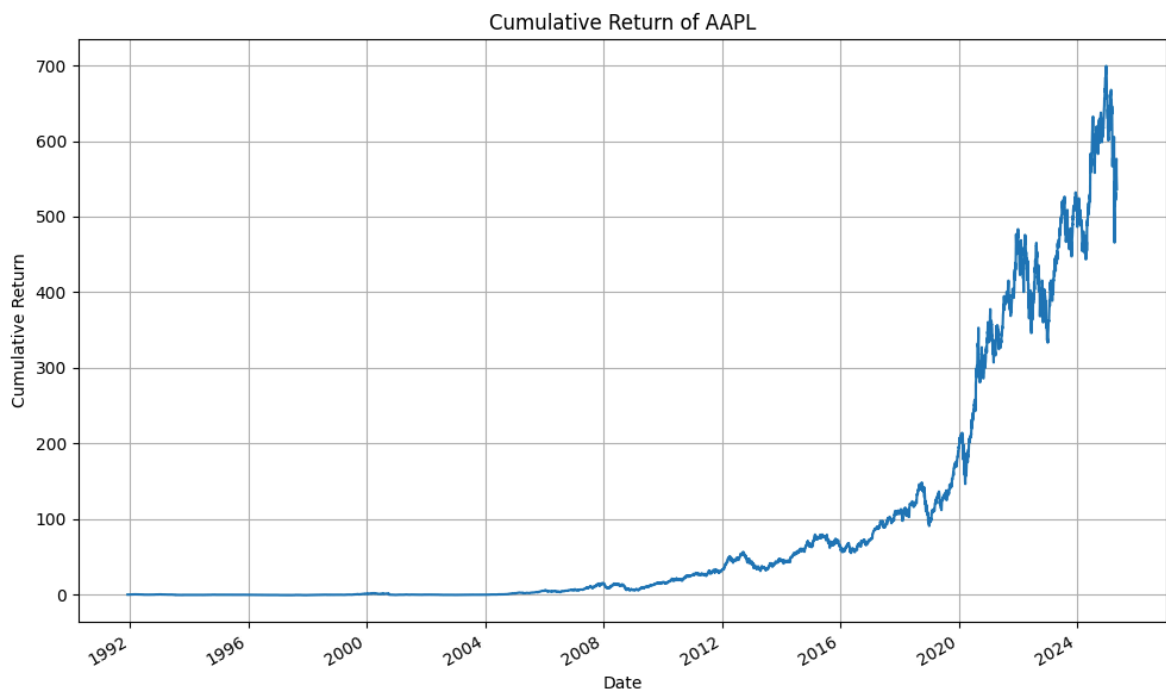
	Price	CVol	Change	% Change	Ret	Total Return (Gross)	Cumulative Return %	Open	High	L
Date										
1991-12-02	0.46	118899320.0	0.01	1.97	1.97	0.48	276,80	0.45	0.46	C
1991-12-03	0.45	102860910.0	-0.01	-2.42	-2.42	0.47	267,70	0.46	0.46	C
1991-12-05	0.45	99198490.0	0.00	-0.99	-0.99	0.47	264,06	0.45	0.46	C
1991-12-06	0.44	196985780.0	-0.01	-2.50	-2.50	0.46	254,96	0.44	0.44	C
1991-12-09	0.44	97832080.0	0.00	0.77	0.77	0.46	257,69	0.44	0.45	C

We will retrieve daily stock price data from AAPL (Apple Inc.) from January 1st, 1980, but we keep only data that do not have any missing observations, reducing the sample size substantially.

Once we have daily prices, we can compute daily returns, which will be used to construct the inputs (and output) of our MLP:

```
In [4]: # Calculate cumulative return
df['Cumulative Return'] = (1 + df['Ret'] / 100).cumprod() - 1

# Plot the cumulative return
plt.figure(figsize=(10, 6))
df['Cumulative Return'].plot()
plt.title('Cumulative Return of AAPL')
plt.xlabel('Date')
plt.ylabel('Cumulative Return')
plt.grid(True)
plt.tight_layout()
plt.show()
```



2. Timing Apple stock with Multilayer Perceptron (MLPs)

As we have already mentioned, the purpose of our MLP is to predict the future return (we will shortly see for which time horizon and how) of AAPL using past returns. Hence, one of the first things we must do is decide and define the inputs and outputs of the model.

2.1 Inputs and outputs

As you already know, there are multiple choices to make when selecting the inputs of a MLP model. Some of them are based on sound theory, but also some simply stem from a trial and error process. In this case, we will apply a very similar approach to what we did when building our models for factor momentum timing and consider as inputs in the network the stock returns from the past 25, 60, 90, 120 and 240 days. Once again, after going over and understanding the entire notebook here, please feel free to play around

with these time frames or even add more/less inputs into the MLP to see how its performance changes.

```
In [5]: df = df[['Ret']]

df["Ret25_i"] = df[name].rolling(25).apply(lambda x: 100 * (np.prod(1 + x / 100)
df["Ret60_i"] = df[name].rolling(60).apply(lambda x: 100 * (np.prod(1 + x / 100)
df["Ret90_i"] = df[name].rolling(90).apply(lambda x: 100 * (np.prod(1 + x / 100)
df["Ret120_i"] = df[name].rolling(120).apply(lambda x: 100 * (np.prod(1 + x / 100)
df["Ret240_i"] = df[name].rolling(240).apply(lambda x: 100 * (np.prod(1 + x / 100)

df = df.dropna()
df.tail(10)
```

```
Out[5]:
```

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i
Date						
2025-04-23	2.43	-3.795072	-10.881083	-17.324816	-11.375229	13.166308
2025-04-24	1.84	-3.186662	-12.437332	-15.363482	-10.514112	14.812281
2025-04-25	0.44	-2.242569	-12.454764	-15.498093	-10.228100	15.098768
2025-04-28	0.41	-3.719238	-11.440489	-15.210988	-8.459465	14.426408
2025-04-29	0.51	-4.309509	-10.388438	-15.764123	-6.287033	15.809066
2025-04-30	0.61	-5.026928	-6.678198	-16.064459	-4.444495	14.500296
2025-05-01	0.39	-3.703195	-8.241178	-13.894452	-3.686575	14.238568
2025-05-02	-3.74	-8.267882	-11.549127	-17.690963	-7.887429	8.640630
2025-05-05	-3.15	-8.729653	-14.608582	-21.754709	-10.493604	5.155357
2025-05-06	-0.19	-10.636714	-12.675026	-22.144726	-12.535408	4.934575

Defining the output: Classification

Finally, we need to define our output label/s. We have decided to focus on the \$+120\$ (trading) days return for Apple stock. Remember that in this case we will perform a classification task with our MLP model, so that we simply aim to predict whether, on a given time \$t\$, the return of AAPL from \$t\$ to \$t+120\$ days will be positive or negative (note how a zero return, although unlikely, will also be classified as negative).

Therefore, we first investigate, at a given time \$t\$, what would be the 120-day return on AAPL. Then, we calculate our output variable, keeping in mind that we will be running a classification task and, hence, we need to convert our output variable to a \$0, 1\$ variable (\$0\$ for negative 120 days return, \$1\$ for positive):

```
In [6]: df["Ret120"] = df["Ret120_i"].shift(-120)
df["Output"] = df["Ret120"] > 0
df["Output"] = df["Output"].astype(int)
del df["Ret120"]
df = df.dropna()
df.tail(10)
```

Out[6]:

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i	Output
Date							
2025-04-23	2.43	-3.795072	-10.881083	-17.324816	-11.375229	13.166308	0
2025-04-24	1.84	-3.186662	-12.437332	-15.363482	-10.514112	14.812281	0
2025-04-25	0.44	-2.242569	-12.454764	-15.498093	-10.228100	15.098768	0
2025-04-28	0.41	-3.719238	-11.440489	-15.210988	-8.459465	14.426408	0
2025-04-29	0.51	-4.309509	-10.388438	-15.764123	-6.287033	15.809066	0
2025-04-30	0.61	-5.026928	-6.678198	-16.064459	-4.444495	14.500296	0
2025-05-01	0.39	-3.703195	-8.241178	-13.894452	-3.686575	14.238568	0
2025-05-02	-3.74	-8.267882	-11.549127	-17.690963	-7.887429	8.640630	0
2025-05-05	-3.15	-8.729653	-14.608582	-21.754709	-10.493604	5.155357	0
2025-05-06	-0.19	-10.636714	-12.675026	-22.144726	-12.535408	4.934575	0

It is always useful to see some summary statistics of the different variables in our model. We can very easily observe them with the `'describe()'` feature in Python:

In [7]: `df.describe()`

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i	
count	8035.000000	8035.000000	8035.000000	8035.000000	8035.000000	8035.000000	8
mean	0.113769	2.878347	7.260452	11.143045	15.113084	32.569285	
std	2.699816	13.394873	21.601984	27.569107	33.106180	56.024248	
min	-51.870000	-69.598132	-77.074930	-72.483802	-74.729250	-75.007329	
25%	-1.160000	-4.860311	-5.001998	-5.042512	-4.139412	-6.002929	
50%	0.100000	2.831957	7.050250	10.625607	14.641836	29.047810	
75%	1.375000	10.547327	19.246392	26.021418	30.664592	59.045564	
max	33.230000	121.286374	116.442841	135.727260	167.513049	300.683525	



2.2 Train-Test samples and Scaling

Next important step in building our model is in defining our train and test samples, together with the scaling of variables. As we have usually done before, we will take 20% of observations and devote them to testing, while 80% will be used for training the model. Please note, once again, the importance of doing this in chronological order!

```
In [8]: # Train-test split
ts = int(0.2 * len(df)) # Number of observations in test sample (20%)
split_time = len(df) - ts # Start of test sample
test_time = df.index[split_time:] # Extract Date index for test period
Ret_vector = df['Ret'][split_time:].values # Extract Ret values for test period
df.tail()
```

```
Out[8]:
```

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i	Output
Date							
2025-04-30	0.61	-5.026928	-6.678198	-16.064459	-4.444495	14.500296	0
2025-05-01	0.39	-3.703195	-8.241178	-13.894452	-3.686575	14.238568	0
2025-05-02	-3.74	-8.267882	-11.549127	-17.690963	-7.887429	8.640630	0
2025-05-05	-3.15	-8.729653	-14.608582	-21.754709	-10.493604	5.155357	0
2025-05-06	-0.19	-10.636714	-12.675026	-22.144726	-12.535408	4.934575	0

Now we are ready to use **sklearn** to formally define the input and output matrices for training (**X_train** and **y_train**) and test (**X_test** and **y_test**). One more time, make sure you keep the 'shuffle' option set to 'False'!

```
In [9]: from sklearn.model_selection import train_test_split

Xdf, ydf = df.iloc[:, 1:-1], df.iloc[:, -1]
X = Xdf.astype("float32")
y = ydf.astype("float32")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=ts, shuffle=False
) # It is important to keep "shuffle=False"
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
(6428, 5) (1607, 5) (6428,) (1607,)
```

The last 'print' in the previous chunk of code gives as an output very useful information specially for a later time where we will move on to more complex models. In this case, we know that we have \$6,428\$ observations in training for \$5\$ different input series. Just as a concept check, could you say how many observations are in the test sample?

- **What about scaling?**

As you see, we have not implemented scaling here. Scaling of variables is actually a very good practice that we recommend you always follow. In this case, since we are working with returns from the same stock, the impact that scaling of inputs will have is presumably very limited. (To be clear, we use 'scaled' data by taking prices that adjust for any dividends and stock splits). Nonetheless, we leave this task to you. Once you have completed the whole notebook, come back here and scale input variables to see whether model performance changes by a lot. Feel free to go back to the last module of Machine Learning, where we introduced scaling.

2.3 Model and Training

Finally, it is time to set up our model. In this case, we will use 3 hidden layers with 25, 15 and 10 units respectively, and a final single-unit output layer. For all the hidden layers, we will use a **ReLU** activation function, whereas we will opt for a **sigmoid** activation in the output layer.

- **Dropout**

Novel to MLP models we have used before (in the Machine Learning course), we are here introducing a **dropout layer** after each of the hidden layers. As you already know, dropout randomly sets some units of a hidden layer to zero. The question of how many units (i.e., frequency rate) is up to the user to define. In this case, we set **n_dropout = 0.2** to shut down 20% of the units in the layer. You can find more info on how the dropout layers work in the official Keras documentation:

https://keras.io/api/layers/regularization_layers/dropout/

- **Loss function?**

So far, we have dealt with regression problems where we used loss functions based on Mean Squared Error (MSE) or Mean Absolute Error (MAE). But now we are in a classification problem with 2 labels (0, 1), so we select a loss function, **binary cross-entropy** that is essentially a log-likelihood:

$$\begin{equation*} H(y_i) = - \frac{1}{N} \sum_{i=1}^N y_i \log(p(y_i)) + (1-y_i) \log(1-p(y_i)) \end{equation*}$$

For more information on how this loss function works, check the **official documentation** in Keras:

https://www.tensorflow.org/api_docs/python/tf/keras/losses/BinaryCrossentropy

A simple explanation of binary cross-entropy may be found here:

<https://towardsdatascience.com/understanding-binary-cross-entropy-log-loss-a-visual-explanation-a3ac6025181a>

- **Metric for loss function?**

Still, one question remains about which metric would we use for our loss function based on binary cross-entropy. A **metric** is simply a function that is used to judge the performance of your model. See all metrics available in Keras here:

<https://keras.io/api/metrics/>

Here, we choose to judge our model performance using '**accuracy**', which in this case corresponds to binary accuracy:

https://keras.io/api/metrics/accuracy_metrics/#binaryaccuracy-class

```
In [10]: import tensorflow as tf

tf.keras.backend.clear_session() # We clear the backend to reset the random seed
tf.random.set_seed(
    1234
) # We will set a random seed so that results obtained are somewhat replicable

act_fun = "relu"
hp_units = 25
hp_units_2 = 15
hp_units_3 = 10
n_dropout = 0.2

model = tf.keras.models.Sequential()
model.add(tf.keras.layers.Dense(units=hp_units, activation=act_fun))
model.add(tf.keras.layers.Dropout(n_dropout))
model.add(tf.keras.layers.Dense(units=hp_units_2, activation=act_fun))
model.add(tf.keras.layers.Dropout(n_dropout))
model.add(tf.keras.layers.Dense(units=hp_units_3, activation=act_fun))
model.add(tf.keras.layers.Dropout(n_dropout))
model.add(tf.keras.layers.Dense(units=1, activation="sigmoid"))

hp_lr = 1e-5 # Learning rate

adam = tf.keras.optimizers.Adam(learning_rate=hp_lr) # Adam optimizer

model.compile(optimizer=adam, loss="binary_crossentropy", metrics=["accuracy"])
```

2025-05-07 07:31:19.938792: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

2025-05-07 07:31:19.983019: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

2025-05-07 07:31:21.038369: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

2.3.1 Validation and callbacks (Earlystopping)

We have already built our model, but there are still a few questions that need an answer when training it: how many epochs do we train for? when does the model stop?

To solve these questions, we will use **EarlyStopping**, a method you are already familiar with (feel free to revisit the documentation for Module 7 of Machine Learning). In this case the variable that we will monitor will be model accuracy in the validation set. Naturally, we will aim to obtain the maximum accuracy in validation, and set the patience to 20, so that model training will stop after 20 epochs in which there is no improvement in the accuracy that the model weights yield in the validation set.

```
In [11]: es = tf.keras.callbacks.EarlyStopping(  
    monitor="val_accuracy",  
    mode="max",  
    verbose=1,  
    patience=20,  
    restore_best_weights=True,  
)
```

2.3.2 Classification on imbalance data: class_weight

When we work on classification tasks, there is always the possibility that one of the labels we are trying to predict is underrepresented in the training sample. Ideally you would want the model to give a heavier weight to underrepresented labels so that you do not overlook this in future prediction. For example, suppose that you are trying to predict corporate default, but your data has very small percentage of observation when a firm actually defaults. You may want the model to pay more attention to these observations where the actual action occurs. For that, we can use **class_weights**.

Class-weights essentially consist on passing Keras a weight for each class in the sample, so that we can make the model focus on a particular class more than it will based on its representation in the sample.

Here you have a complete tutorial on how to perform these kind of tasks that use imbalance data in Keras. Please note that his tutorial covers a wide range of topics that we will eventually cover as well. In order not to get lost in details that are not important for now, we suggest you focus on the 'Class weights' section:

https://www.tensorflow.org/tutorials/structured_data/imbalanced_data?hl=en

In this case, we would like the model to pay more attention to labels of 0, which correspond to days in which the 120-day-ahead return is negative. There are many ways to assign and modify class weights, we follow here a simple one, completely discretionary:

```
In [12]: class_weight = {0: (np.mean(y_train) / 0.5) * 1.2, 1: 1.0}  
print(class_weight)  
  
{0: 1.6312382698059082, 1: 1.0}
```

Finally, we are ready to train our model with all these ingredients!

```
In [13]: history = model.fit(  
    X_train,  
    y_train,
```

```
validation_split=0.2,  
epochs=500,  
batch_size=32,  
verbose=2,  
callbacks=[es],  
class_weight=class_weight,  
)
```

Epoch 1/500
161/161 - 1s - 9ms/step - accuracy: 0.5998 - loss: 5.6416 - val_accuracy: 0.7706
- val_loss: 1.2708
Epoch 2/500
161/161 - 0s - 3ms/step - accuracy: 0.5862 - loss: 5.3652 - val_accuracy: 0.7714
- val_loss: 1.1826
Epoch 3/500
161/161 - 1s - 4ms/step - accuracy: 0.5994 - loss: 5.3257 - val_accuracy: 0.7706
- val_loss: 1.0847
Epoch 4/500
161/161 - 0s - 2ms/step - accuracy: 0.5838 - loss: 5.1315 - val_accuracy: 0.7714
- val_loss: 0.9956
Epoch 5/500
161/161 - 0s - 2ms/step - accuracy: 0.5957 - loss: 4.7177 - val_accuracy: 0.7722
- val_loss: 0.9188
Epoch 6/500
161/161 - 0s - 2ms/step - accuracy: 0.5867 - loss: 4.6330 - val_accuracy: 0.7714
- val_loss: 0.8548
Epoch 7/500
161/161 - 0s - 2ms/step - accuracy: 0.5932 - loss: 4.4603 - val_accuracy: 0.7706
- val_loss: 0.7948
Epoch 8/500
161/161 - 0s - 2ms/step - accuracy: 0.5685 - loss: 4.3878 - val_accuracy: 0.7698
- val_loss: 0.7364
Epoch 9/500
161/161 - 0s - 2ms/step - accuracy: 0.5762 - loss: 4.1500 - val_accuracy: 0.7621
- val_loss: 0.6938
Epoch 10/500
161/161 - 0s - 2ms/step - accuracy: 0.5694 - loss: 4.1567 - val_accuracy: 0.7605
- val_loss: 0.6574
Epoch 11/500
161/161 - 0s - 2ms/step - accuracy: 0.5729 - loss: 3.8353 - val_accuracy: 0.7543
- val_loss: 0.6245
Epoch 12/500
161/161 - 0s - 2ms/step - accuracy: 0.5681 - loss: 4.0955 - val_accuracy: 0.7465
- val_loss: 0.6002
Epoch 13/500
161/161 - 1s - 4ms/step - accuracy: 0.5722 - loss: 3.6306 - val_accuracy: 0.7403
- val_loss: 0.5881
Epoch 14/500
161/161 - 0s - 2ms/step - accuracy: 0.5712 - loss: 3.5619 - val_accuracy: 0.7278
- val_loss: 0.5813
Epoch 15/500
161/161 - 0s - 2ms/step - accuracy: 0.5710 - loss: 3.3990 - val_accuracy: 0.7068
- val_loss: 0.5795
Epoch 16/500
161/161 - 1s - 4ms/step - accuracy: 0.5490 - loss: 3.3264 - val_accuracy: 0.6921
- val_loss: 0.5795
Epoch 17/500
161/161 - 0s - 2ms/step - accuracy: 0.5545 - loss: 3.4711 - val_accuracy: 0.6820
- val_loss: 0.5798
Epoch 18/500
161/161 - 0s - 2ms/step - accuracy: 0.5537 - loss: 3.2934 - val_accuracy: 0.6750
- val_loss: 0.5800
Epoch 19/500
161/161 - 0s - 2ms/step - accuracy: 0.5580 - loss: 3.2438 - val_accuracy: 0.6625
- val_loss: 0.5826
Epoch 20/500
161/161 - 0s - 2ms/step - accuracy: 0.5537 - loss: 3.0105 - val_accuracy: 0.6594
- val_loss: 0.5858

```

Epoch 21/500
161/161 - 0s - 2ms/step - accuracy: 0.5371 - loss: 3.0470 - val_accuracy: 0.6493
- val_loss: 0.5900
Epoch 22/500
161/161 - 0s - 2ms/step - accuracy: 0.5424 - loss: 2.9000 - val_accuracy: 0.6376
- val_loss: 0.5926
Epoch 23/500
161/161 - 0s - 2ms/step - accuracy: 0.5449 - loss: 3.0201 - val_accuracy: 0.6159
- val_loss: 0.5980
Epoch 24/500
161/161 - 0s - 2ms/step - accuracy: 0.5447 - loss: 2.8814 - val_accuracy: 0.6003
- val_loss: 0.6010
Epoch 25/500
161/161 - 0s - 2ms/step - accuracy: 0.5288 - loss: 2.8589 - val_accuracy: 0.5902
- val_loss: 0.6030
Epoch 25: early stopping
Restoring model weights from the end of the best epoch: 5.

```

As usual, let's also inspect the summary of the model layers, which in the presence of dropout makes much more sense to have clear in mind:

```
In [14]: model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 25)	150
dropout (Dropout)	(None, 25)	0
dense_1 (Dense)	(None, 15)	390
dropout_1 (Dropout)	(None, 15)	0
dense_2 (Dense)	(None, 10)	160
dropout_2 (Dropout)	(None, 10)	0
dense_3 (Dense)	(None, 1)	11



Total params: 2,135 (8.34 KB)

Trainable params: 711 (2.78 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 1,424 (5.57 KB)

Note that the presence of the dropout layer does not interfere with the number of parameters in the model. This is because dropout neither adds nor reduces the number of model parameters (or units) as whole, as units are randomly shut down during training epochs, but of course the unit affected is not always the same!

This section concludes all relative to the construction and training of the MLP

classification model. Now, as we have usually done, let's explore if we could use this model, which achieves a decent accuracy in the validation set, to build a trading strategy:

3. Financial performance of the model

As we have done before, let's now evaluate the use of this model for a financial strategy. It is very important that we clearly understand that evaluating the **financial performance** is a completely separate thing from evaluating the **predictive performance** of the model. Obviously, the two tasks are interrelated, but they may offer completely different conclusions after a thorough analysis. We have already trained our model and assess its fit. Now, we will check if the predictions delivered in the test sample are valid for developing a trading strategy!

3.1 Model performance in test sample

We will start by obtaining the predictions of the model in the test sample and its accuracy. Remember, we are evaluating how well our model predicts, using past AAPL returns for 5 different windows, whether the next 120 days returns is positive or negative. The way we will operate is defining model prediction as a 1 (i.e., positive return) if the probability assigned by the model is higher than 0.5; and 0 (i.e., negative return) else.

```
In [15]: import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import metrics

y_prob = model.predict(X_test)
y_pred = np.where(y_prob > 0.50, 1, 0)

acc = model.evaluate(X_test, y_test)
print("Model accuracy in test: ", acc)
```

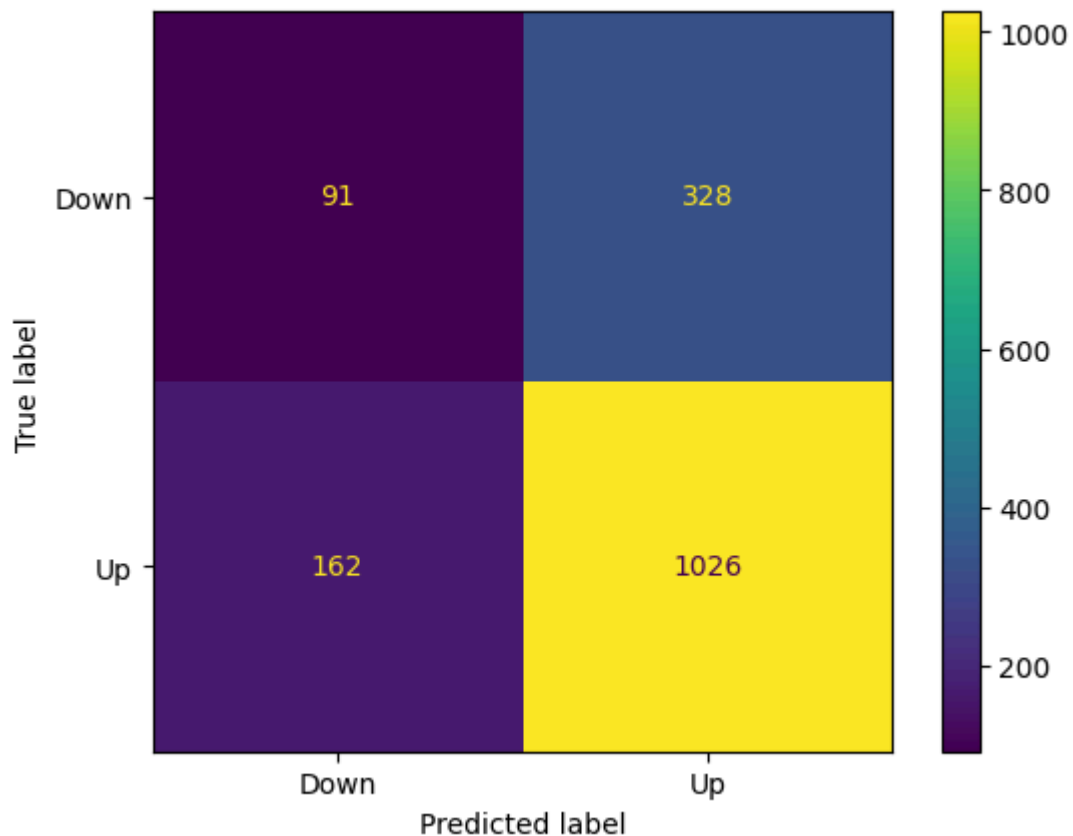
```
51/51 ————— 0s 1ms/step
51/51 ————— 0s 1ms/step - accuracy: 0.7859 - loss: 0.6321
Model accuracy in test: [0.9570235013961792, 0.6950840353965759]
```

As you can see, we obtain a very decent accuracy of the model in the test sample. But we would also like to see when does the model 'miss', and when it is mostly 'right' about a prediction. For that, we come to our old friend, the confusion matrix:

```
In [16]: ### DMoodley correction
from sklearn.metrics import ConfusionMatrixDisplay

cm = metrics.confusion_matrix(y_test.values, y_pred[:,0])
disp = ConfusionMatrixDisplay(cm, display_labels=["Down", "Up"])
disp.plot()
```

```
Out[16]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x77b37cfa1d0>
```



3.2 Trading strategy based on model predictions

Next, let's evaluate the extent to which our model predictions can act as the foundation of a trading strategy. To that end, we will perform a **backtest** in the same spirit of Module 7 in Machine Learning:

```
In [17]: df_predictions = pd.DataFrame(
    {
        'Date': test_time, # No flatten needed for DatetimeIndex
        'Pred': y_pred.flatten() if y_pred.ndim > 1 else y_pred, # Flatten if 2
        'Ret': Ret_vector.flatten() if Ret_vector.ndim > 1 else Ret_vector, # F
    }
)
df_predictions.tail()
```

```
Out[17]:
```

	Date	Pred	Ret
1602	2025-04-30	0	0.61
1603	2025-05-01	0	0.39
1604	2025-05-02	0	-3.74
1605	2025-05-05	0	-3.15
1606	2025-05-06	0	-0.19

We will define the positions that our trading strategy will take as long (\$+1\$) if the prediction of the model is higher than \$0.5\$, and short (\$-1\$) if less. (Note, nonetheless, that in practice our prediction can only take values 0 or 1)

As in previous cases, we will backtest 3 trading strategies:

- A long/short strategy that will take a long or short position when model prediction indicate so.
- A long-only strategy that will go to cash (return = 0) when model predicts a negative 120-day return.
- A Buy-and-hold strategy that will buy the stock at the beginning of the test period and hold it until the end of the period.

```
In [18]: df = df_predictions

df['Ret'] = df['Ret'] / 100

df["Positions"] = np.where(df["Pred"] > 0.5, 1, -1)
df["Strat_ret"] = df["Positions"].shift(1) * df["Ret"]
df["Positions_L"] = df["Positions"].shift(1)
df["Positions_L"][df["Positions_L"] == -1] = 0
df["Strat_ret_L"] = df["Positions_L"] * df["Ret"]
df["CumRet"] = df["Strat_ret"].expanding().apply(lambda x: np.prod(1 + x) - 1)
df["CumRet_L"] = df["Strat_ret_L"].expanding().apply(lambda x: np.prod(1 + x) - 1)
df["bhRet"] = df["Ret"].expanding().apply(lambda x: np.prod(1 + x) - 1)

Final_Return_L = np.prod(1 + df["Strat_ret_L"]) - 1
Final_Return = np.prod(1 + df["Strat_ret"]) - 1
Buy_Return = np.prod(1 + df["Ret"]) - 1

print("Strat Return Long Only =", Final_Return_L * 100, "%")
print("Strat Return =", Final_Return * 100, "%")
print("Buy and Hold Return =", Buy_Return * 100, "%")
```

/tmp/ipykernel_5841/2685361883.py:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df["Positions_L"][df["Positions_L"] == -1] = 0
```

Strat Return Long Only = 313.1173394983836 %

Strat Return = 194.30818715943187 %

Buy and Hold Return = 376.11163150377774 %

```
In [19]: import matplotlib.pyplot as plt

fig = plt.figure(figsize=(12, 6))
ax = plt.gca()
df.plot(x="Date", y="bhRet", label="Buy&Hold", ax=ax)
df.plot(x="Date", y="CumRet_L", label="Strat Only Long", ax=ax)
df.plot(x="Date", y="CumRet", label="Strat Long/Short", ax=ax)
plt.xlabel("date")
plt.ylabel("Cumulative Returns")
plt.grid()
plt.show()

df.describe()
```



Out[19]:

	Date	Pred	Ret	Positions	Strat_ret	Positior
count	1607	1607.000000	1607.000000	1607.000000	1606.000000	1606.000
mean	2022-02-18 12:06:43.235843328	0.842564	0.001174	0.685128	0.000877	0.843
min	2018-12-07 00:00:00	0.000000	-0.128600	-1.000000	-0.153300	0.000
25%	2020-07-15 12:00:00	1.000000	-0.008150	1.000000	-0.008300	1.000
50%	2022-02-16 00:00:00	1.000000	0.001500	1.000000	0.001150	1.000
75%	2023-09-26 12:00:00	1.000000	0.011900	1.000000	0.011600	1.000
max	2025-05-06 00:00:00	1.000000	0.153300	1.000000	0.119800	1.000
std	NaN	0.364325	0.020169	0.728650	0.020171	0.363



4. Conclusion

In this lesson, we have worked with TensorFlow Keras on a MLP model for classification purposes. Specifically, we designed a MLP model that aims to predict the forward 120 day return of Apple stock. In doing so, we have introduced some novel features of Deep Learning models, such as dropout and class weights, which will also come in handy later on for improving the performance of our models. Still, there is much to be done in terms of improving model performance by, for example, optimally selecting some of the hyperparameters used in the model such as the learning rate. That is what we will do in the remainder of the module introducing hyperparameter tuning. For that, see you in the next lesson!

On Hyperparameter Optimization of Machine Learning Algorithms: Theory and Practice

Li Yang and Abdallah Shami

*Department of Electrical and Computer Engineering, University of Western Ontario,
1151 Richmond St, London, Ontario, Canada N6A 3K7*

Abstract

Machine learning algorithms have been used widely in various applications and areas. To fit a machine learning model into different problems, its hyper-parameters must be tuned. Selecting the best hyper-parameter configuration for machine learning models has a direct impact on the model's performance. It often requires deep knowledge of machine learning algorithms and appropriate hyper-parameter optimization techniques. Although several automatic optimization techniques exist, they have different strengths and drawbacks when applied to different types of problems. In this paper, optimizing the hyper-parameters of common machine learning models is studied. We introduce several state-of-the-art optimization techniques and discuss how to apply them to machine learning algorithms. Many available libraries and frameworks developed for hyper-parameter optimization problems are provided, and some open challenges of hyper-parameter optimization research are also discussed in this paper. Moreover, experiments are conducted on benchmark datasets to compare the performance of different optimization methods and provide practical examples of hyper-parameter optimization. This survey paper will help industrial users, data analysts, and researchers to better develop machine learning models by identifying the proper hyper-parameter configurations effectively.¹

Keywords: Hyper-parameter optimization, machine learning, Bayesian

Email address: {lyang339, abdallah.shami}@uwo.ca (Li Yang and Abdallah Shami)

¹General Hyperparameter Optimization Code and Tutorials: <https://github.com/LiYangHart/Hyperparameter-Optimization-of-Machine-Learning-Algorithms>

optimization, particle swarm optimization, genetic algorithm, grid search.

1. Introduction

Machine learning (ML) algorithms have been widely used in many applications domains, including advertising, recommendation systems, computer vision, natural language processing, and user behavior analytics [1]. This is because they are generic and demonstrate high performance in data analytics problems. Different ML algorithms are suitable for different types of problems or datasets [2]. In general, building an effective machine learning model is a complex and time-consuming process that involves determining the appropriate algorithm and obtaining an optimal model architecture by tuning its hyper-parameters (HPs) [3].

Two types of parameters exist in machine learning models: one that can be initialized and updated through the data learning process (*e.g.*, the weights of neurons in neural networks), named model parameters; while the other, named hyper-parameters, cannot be directly estimated from data learning and must be set before training a ML model because they define the architecture of a ML model [4]. Hyper-parameters are the parameters that are used to either configure a ML model (*e.g.*, the penalty parameter C in a support vector machine, and the learning rate to train a neural network) or to specify the algorithm used to minimize the loss function (*e.g.*, the activation function and optimizer types in a neural network, and the kernel type in a support vector machine) [5].

To build an optimal ML model, a range of possibilities must be explored. The process of designing the ideal model architecture with an optimal hyper-parameter configuration is named hyper-parameter tuning. Tuning hyper-parameters is considered a key component of building an effective ML model, especially for tree-based ML models and deep neural networks, which have many hyper-parameters [6]. Hyper-parameter tuning process is different among different ML algorithms due to their different types of hyper-parameters, including categorical, discrete, and continuous hyper-parameters [7]. Manual testing is a traditional way to tune hyper-parameters and is still prevalent in graduate student research, although it requires a deep understanding of the used ML algorithms and their hyper-parameter value settings [8]. However, manual tuning is ineffective for many problems due to certain factors, including a large number of hyper-parameters, complex models, time-consuming model evaluations, and non-linear hyper-parameter interactions.

These factors have inspired increased research in techniques for automatic optimization of hyper-parameters; so-called hyper-parameter optimization (HPO) [9]. The main aim of HPO is to automate hyper-parameter tuning process and make it possible for users to apply machine learning models to practical problems effectively [3]. The optimal model architecture of a ML model is expected to be obtained after a HPO process. Some important reasons for applying HPO techniques to ML models are as follows [6]:

1. It reduces the human effort required, since many ML developers spend considerable time tuning the hyper-parameters, especially for large datasets or complex ML algorithms with a large number of hyper-parameters.
2. It improves the performance of ML models. Many ML hyper-parameters have different optimums to achieve best performance in different datasets or problems.
3. It makes the models and research more reproducible. Only when the same level of hyper-parameter tuning process is implemented can different ML algorithms be compared fairly; hence, using a same HPO method on different ML algorithms also helps to determine the most suitable ML model for a specific problem.

It is crucial to select an appropriate optimization technique to detect optimal hyper-parameters. Traditional optimization techniques may be unsuitable for HPO problems, since many HPO problems are non-convex or non-differentiable optimization problems, and may result in a local instead of a global optimum [10]. Gradient descent-based methods are a common type of traditional optimization algorithm that can be used to tune continuous hyper-parameters by calculating their gradients [11]. For example, the learning rate in a neural network can be optimized by a gradient-based method.

Compared with traditional optimization methods like gradient descent, many other optimization techniques are more suitable for HPO problems, including decision-theoretic approaches, Bayesian optimization models, multi-fidelity optimization techniques, and metaheuristics algorithms [7]. Apart from detecting continuous hyper-parameters, many of these algorithms also have the capacity to effectively identify discrete, categorical, and conditional hyper-parameters.

Decision-theoretic methods are based on the concept of defining a hyper-parameter search space and then detecting the hyper-parameter combina-

tions in the search space, ultimately selecting the best-performing hyper-parameter combination. Grid search (GS) [12] is a decision-theoretic approach that involves exhaustively searching for a fixed domain of hyper-parameter values. Random search (RS) [13] is another decision-theoretic method that randomly selects hyper-parameter combinations in the search space, given limited execution time and resources. In GS and RS, each hyper-parameter configuration is treated independently.

Unlike GS and RS, Bayesian optimization (BO) [14] models determine the next hyper-parameter value based on the previous results of tested hyper-parameter values, which avoids many unnecessary evaluations; thus, BO can detect the optimal hyper-parameter combination within fewer iterations than GS and RS. To be applied to different problems, BO can model the distribution of the objective function using different models as the surrogate function, including Gaussian process (GP), random forest (RF), and tree-structured Parzen estimators (TPE) models [15]. BO-RF and BO-TPE can retain the conditionality of variables [15]. Thus, they can be used to optimize conditional hyper-parameters, like the kernel type and the penalty parameter C in a support vector machine (SVM). However, since BO models work sequentially to balance the exploration of unexplored areas and the exploitation of currently-tested regions, it is difficult to parallelize them.

Training a ML model often takes considerable time and space. Multi-fidelity optimization algorithms are developed to tackle problems with limited resources, and the most common ones being bandit-based algorithms. Hyperband [16] is a popular bandit-based optimization technique that can be considered an improved version of RS. It generates small versions of datasets and allocates a same budget to each hyper-parameter combination. In each iteration of Hyperband, poorly-performing hyper-parameter configurations are eliminated to save time and resources.

Metaheuristic algorithms are a set of techniques used to solve complex, large search space and non-convex optimization problems to which HPO problems belong [17]. Among all metaheuristic methods, genetic algorithm (GA) [18] and particle swarm optimization (PSO) [19] are the two most prevalent metaheuristic algorithms used for HPO problems. Genetic algorithms detect well-performing hyper-parameter combinations in each generation, and pass them to the next generation until the best-performing combination is identified. In PSO algorithms, each particle communicates with other particles to detect and update the current global optimum in each iteration until the final optimum is detected. Metaheuristics can efficiently

explore the search space to detect optimal or near-optimal solutions. Hence, they are particularly suitable for the HPO problems with large configuration space due to their high efficiency. For instance, they can be used in deep neural networks (DNNs) which have a large configuration space with multiple hyper-parameters, including the activation and optimizer types, the learning rate, drop-out rate, etc.

Although using HPO algorithms to tune the hyper-parameters of ML models greatly improves the model performance, certain other aspects, like their computational complexity, still have much room for improvement. On the other hand, since different HPO models have their own advantages and suitable problems, overviewing them is necessary for proper optimization algorithm selection in terms of different types of ML models and problems.

This paper makes the following contributions:

1. It reviews common ML algorithms and their important hyper-parameters.
2. It analyzes common HPO techniques, including their benefits and drawbacks, to help apply them to different ML models by appropriate algorithm selection in practical problems.
3. It surveys common HPO libraries and frameworks for practical use.
4. It discusses the open challenges and research directions of the HPO research domain.

In this survey paper, we begin with a comprehensive introduction of the common optimization techniques used in ML hyper-parameter tuning problems. Section 2 introduces the main concepts of mathematical optimization and hyper-parameter optimization, as well as the general HPO process. In Section 3, we discuss the key hyper-parameters of common ML models that need to be tuned. Section 4 covers the various state-of-the-art optimization approaches that have been proposed for tackling HPO problems. In Section 5, we analyze different HPO methods and discuss how they can be applied to ML algorithms. In Section 6, we provide an introduction to various public libraries and frameworks that are developed to implement HPO. Section 7 presents and discusses the experimental results of using HPO on benchmark datasets for HPO method comparison and practical use case demonstration. In Section 8, we discuss several research directions and open challenges that should be considered to improve current HPO models or develop new HPO approaches. We conclude the paper in Section 9.

2. Mathematical Optimization and Hyper-parameter Optimization Problems

The key process of machine learning is to solve optimization problems. To build a ML model, its weight parameters are initialized and optimized by an optimization method until the objective function approaches a minimum value or the accuracy approaches a maximum value [20]. Similarly, hyper-parameter optimization methods aim to optimize the architecture of a ML model by detecting the optimal hyper-parameter configurations. In this section, the main concepts of mathematical optimization and hyper-parameter optimization for machine learning models are discussed.

2.1. Mathematical Optimization

Mathematical optimization is the process of finding the best solution from a set of available candidates to maximize or minimize the objective function [20]. Generally, optimization problems can be classified as constrained or unconstrained optimization problems based on whether they have constraints for the decision variables or the solution variables.

In unconstrained optimization problems, a decision variable, x , can take any values from the one-dimensional space of all real numbers, $\mathbb{R}$. An unconstrained optimization problem can be denoted by [21]:

$$\min_{x \in \mathbb{R}} f(x), \quad (1)$$

where $f(x)$ is the objective function.

On the other hand, most real-life optimization problems are constrained optimization problems. The decision variable x for constrained optimization problems should be subject to certain constraints which could be mathematical equalities or inequalities. Therefore, constrained optimization problems or general optimization problems can be expressed as [21]:

$$\begin{aligned} & \min_x f(x) \\ & \text{subject to} \\ & g_i(x) \leq 0, i = 1, 2, \dots, m, \\ & h_j(x) = 0, j = 1, 2, \dots, p, \\ & x \in X, \end{aligned} \quad (2)$$

where $g_i(x), i = 1, 2, \dots, m$, are the inequality constraint functions; $h_j(x), j = 1, 2, \dots, p$, are the equality constraint function; and X is the domain of x .

The role of constraints is to limit the possible values of the optimal solution to certain areas of the search space, named the feasible region [21]. Thus, the feasible region D of x can be represented by:

$$D = \{x \in X | g_i(x) \leq 0, h_j(x) = 0\}. \quad (3)$$

To conclude, an optimization problem consists of three major components: a set of decision variables x , an objective function $f(x)$ to be either minimized or maximized, and a set of constraints that allow the variables to take on values in certain ranges (if it is a constrained optimization problem). Therefore, the goal of optimization tasks is to obtain the set of variable values that minimize or maximize the objective function while satisfying any applicable constraints.

Regarding ML models, many HPO problems have certain constraints, like the feasible domain of the number of clusters in k-means, as well as time and space constraints. Therefore, constrained optimization techniques are widely-used in HPO problems [3].

For optimization problems, in many cases, only a local instead of a global optimum can be obtained. For example, to obtain the minimum of a problem, assuming D is the feasible region of a decision variable x , a global minimum is the point $x^* \in D$ satisfying $f(x^*) \leq f(x) \forall x \in D$, while a local minimum is a point $x^* \in D$ in a neighborhood N satisfying $f(x^*) \leq f(x) \forall x \in N \cap D$ [21]. Thus, the local optimum may only be an optimum in a small range instead of being the optimal solution in the entire feasible region.

A local optimum is only guaranteed to be the global optimum in convex functions [22]. Convex functions are the functions that only have one optimum. Therefore, continuing to search along the direction in which the objective function decreases can detect the global minimum value. A function $f(x)$ is a convex function if for $\forall x_1, x_2 \in X, \forall t \in [0, 1]$,

$$f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2), \quad (4)$$

where X is the domain of decision variables, and t is a coefficient in the range of $[0, 1]$.

An optimization problem is a convex optimization problem only when the objective function $f(x)$ is a convex function and the feasible region C is a convex set, denoted by [22]:

$$\begin{aligned} & \min_x f(x) \\ & \text{subject to } x \in C. \end{aligned} \quad (5)$$

On the other hand, nonconvex functions have multiple local optimums, but only one of these optimums is the global optimum. Most ML and HPO problems are nonconvex optimization problems. Thus, utilizing inappropriate optimization methods may only result in a local instead of a global optimum.

There are many traditional methods that can be used to solve optimization problems, including gradient descent, Newton’s method, conjugate gradient, and heuristic optimization methods [20]. Gradient descent is a commonly-used optimization method that uses the negative gradient direction as the search direction to move towards the optimum. However, gradient descent cannot guarantee to detect the global optimum unless the objective function is a convex function. Newton’s method uses the inverse matrix of the Hessian matrix to obtain the optimum. Newton’s method has faster convergence speed than gradient descent, but often requires more time and larger space than gradient descent to store and calculate the Hessian matrix. Conjugate gradient searches along the conjugated directions constructed by the gradient of known data points to detect the optimum. Conjugate gradient has faster convergence speed than gradient descent, but its calculation of conjugate gradient is more complex. Unlike other traditional methods, heuristic methods use empirical rules to solve the optimization problems instead of following systematical steps to obtain the solution. Heuristic methods can often detect the approximate global optimum within a few iterations, but cannot guarantee to detect the global optimum [20].

2.2. Hyper-parameter Optimization Problem Statement

During the design process of ML models, effectively searching the hyper-parameters’ space using optimization techniques can identify the optimal hyper-parameters for the models. The hyper-parameter optimization process consists of four main components: an estimator (a regressor or a classifier) with its objective function, a search space (configuration space), a search or optimization method used to find hyper-parameter combinations, and an evaluation function to compare the performance of different hyper-parameter configurations.

The domain of a hyper-parameter can be continuous (*e.g.*, learning rate), discrete (*e.g.*, number of clusters), binary (*e.g.*, whether to use early stopping or not), or categorical (*e.g.*, type of optimizer). Therefore, hyper-parameters are classified as continuous, discrete, and categorical hyper-parameters. For continuous and discrete hyper-parameters, their domains are usually bounded

in practical applications [12] [23]. On the other hand, the hyper-parameter configuration space sometimes contains conditionality. A hyper-parameter may need to be used or tuned depending on the value of another hyper-parameter, called a conditional hyper-parameter [10]. For instance, in SVM, the degree of the polynomial kernel function only needs to be tuned when the kernel type is chosen to be polynomial.

In simple cases, all hyper-parameters can take unrestricted real values, and the feasible set X of hyper-parameters can be a real-valued n -dimensional vector space. However, in most cases, the hyper-parameters of a ML model often take on values from different domains and have different constraints, so their optimization problems are often complex constrained optimization problems [24]. For instance, the number of considered features in a decision tree should be in the range of 0 to the number of features, and the number of clusters in k-means should not be larger than the size of data points. Additionally, categorical features can often only take several certain values, like the limited choices of the activation function and the optimizer of a neural network. Therefore, the feasible domain of X often has a complex structure, which increases the problems' complexity [24].

In general, for a hyper-parameter optimization problem, the aim is to obtain [19]:

$$x^* = \arg \min_{x \in X} f(x), \quad (6)$$

where $f(x)$ is the objective function to be minimized, such as the error rate or the root mean squared error (RMSE); x^* is the hyper-parameter configuration that produces the optimum value of $f(x)$; and a hyper-parameter x can take any value in the search space X .

The aim of HPO is to achieve optimal or near-optimal model performance by tuning hyper-parameters within the given budgets [3]. The mathematical expression of the function f varies, depending on the objective function of the chosen ML algorithm and the performance metric function. Model performance can be evaluated by various metrics, like accuracy, RMSE, F1-score, and false alarm rate. On the other hand, in practice, time budgets are an essential constraint for optimizing HPO models and must be considered. It often requires a massive amount of time to optimize the objective function of a ML model with a reasonable number of hyper-parameter configurations. Every time a hyper-parameter value is tested, the entire ML model needs to be retrained, and the validation set needs to be processed to generate a score that reflects the model performance.

The main process of HPO is as follows [10]:

1. Select the objective function and the performance metrics;
2. Select the hyper-parameters that require tuning, summarize their types, and determine the appropriate optimization technique;
3. Train the ML model using the default hyper-parameter configuration or common values as the baseline model;
4. Start the optimization process with a large search space as the hyper-parameter feasible domain determined by manual testing and/or domain knowledge;
5. Narrow the search space based on the regions of currently-tested well-performing hyper-parameter values, or explore new search spaces if necessary.
6. Return the best-performing hyper-parameter configuration as the final solution.

However, most traditional optimization techniques [25] are unsuitable for HPO, since HPO problems are different from traditional optimization problems in the following aspects [10]:

1. The optimization target, the objective function of ML models, is usually a non-convex and non-differentiable function. Therefore, many traditional optimization methods designed to solve convex or differentiable optimization problems are often unsuitable for HPO problems, since these methods may return a local optimum instead of a global optimum. Additionally, an optimization target lacking smoothness makes certain traditional derivative-free optimization models perform poorly for HPO problems [26].
2. The hyper-parameters of ML models include continuous, discrete, categorical, and conditional hyper-parameters. Thus, many traditional numerical optimization methods [27] that only aim to tackle numerical or continuous variables are unsuitable for HPO problems.
3. It is often computationally expensive to train a ML model on a large-scale dataset. HPO techniques sometimes use data sampling to obtain approximate values of the objective function. Thus, effective optimization techniques for HPO problems should be able to use these approximate values. However, function evaluation time is often ignored in many black-box optimization (BBO) models, so they often require exact instead of approximate objective function values. Consequently,

many BBO algorithms are often unsuitable for HPO problems with limited time and resource budgets.

Therefore, appropriate optimization algorithms should be applied to HPO problems to identify optimal hyper-parameter configurations for ML models.

3. Hyper-parameters in Machine Learning Models

To boost ML models by HPO, firstly, we need to find out what the key hyper-parameters are that people need to tune to fit the ML models into specific problems or datasets.

In general, ML models can be classified as supervised and unsupervised learning algorithms, based on whether they are built to model labeled or unlabeled datasets [28]. Supervised learning algorithms are a set of machine learning algorithms that map input features to a target by training on labeled data, and mainly include linear models, k-nearest neighbors (KNN), support vector machines (SVM), naïve Bayes (NB), decision-tree-based models, and deep learning (DL) algorithms [29]. Unsupervised learning algorithms are used to find patterns from unlabeled data and can be divided into clustering and dimensionality reduction algorithms based on their aims. Clustering methods mainly include k-means, density-based spatial clustering of applications with noise (DBSCAN), hierarchical clustering, and expectation-maximization (EM); while two common dimensionality reduction algorithms are principal component analysis (PCA) and linear discriminant analysis (LDA) [30]. Moreover, there are several ensemble learning methods that combine different singular models to further improve model performance, like voting, bagging, and AdaBoost. In this paper, the important hyper-parameters of common ML models are studied based on their names in Python libraries, including scikit-learn (sklearn) [31], XGBoost [32], and Keras[33].

3.1. Supervised Learning Algorithms

In supervised learning, both the input x and the output y are available, and the goal is to obtain an optimal predictive model function f^* to minimize the cost function $\mathcal{L}(f(x), y)$ that models the error between the estimated output and ground-truth labels. The predictive model function f varies based on its model structure. With limited model architectures determined by different hyper-parameter configurations, the domain of the ML model

function f is restricted to a set of functions F . Thus, the optimal predictive model f^* can be obtained by [34]:

$$f^* = \arg \min_{f \in F} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x_i), y_i) \quad (7)$$

where n is the number of training data points, x_i is the feature vector of the i -th instance, y_i is the corresponding actual output, and L is the cost function value of each sample.

Many different loss functions exist in supervised learning algorithms, including the square of Euclidean distance, cross-entropy, information gain, etc. [34]. On the other hand, different ML algorithms generate different predictive model architectures based on different hyper-parameter configurations, which will be discussed in detail in this subsection.

3.1.1. Linear Models

In general, supervised learning models can be classified as regression and classification techniques when used to predict continuous or discrete target variables, respectively. Linear regression [35] is a typical regression model that predicts a target y by the following equation:

$$\hat{y}(\mathbf{w}, \mathbf{x}) = w_0 + w_1 x_1 + \dots + w_p x_p, \quad (8)$$

where the target variable y is expected to be a linear combination of p input features $\mathbf{x} = (x_1, \dots, x_p)$, and $\hat{y}$ is the predicted value. The weight vector $\mathbf{w} = (w_1, \dots, w_p)$ is designated as an attribute 'coef_', and w_0 is defined as another attribute 'intercept_' in the linear model of sklearn. Usually, no hyper-parameter needs to be tuned in linear regression. A linear model's performance mainly depends on how well the problem or data follows a linear distribution.

To improve the original linear regression models, ridge regression was proposed in [36]. Ridge regression imposes a penalty on the coefficients, and aims to minimize the objective function [37]:

$$\alpha \|\mathbf{w}\|_2^2 + \sum_{i=1}^p (y_i - w_i \cdot x_i)^2, \quad (9)$$

where $\|\mathbf{w}\|_2$ is the L_2 -norm of the coefficient vector, and α is the regularization strength. A larger value of α indicates a larger amount of shrinkage; thus, the coefficients are also more robust to collinearity.

Lasso regression [38] is another linear model used to estimate sparse coefficients, consisting of a linear model with an L_1 priori added regularization term. It aims to minimize the objective function [37]:

$$\alpha \|w\|_1 + \sum_{i=1}^p (y_i - w_i \cdot x_i)^2, \quad (10)$$

where α is the regularization strength and $\|w\|_1$ is the L_1 -norm of the coefficient vector. Therefore, the regularization strength α is an crucial hyper-parameter of both ridge and lasso regression models.

Logistic regression (LR) [39] is a linear model used for classification problems. In LR, its cost function may be different, depending on the regularization method chosen for the penalization. There are three main types of regularization methods in LR: L_1 -norm, L_2 -norm, and elastic-net regularization [40].

Therefore, the first hyper-parameter that needs to be tuned in LR is to the regularization method used in the penalization, 'l1', 'l2', 'elasticnet' or 'none', which is called 'penalty' in sklearn. The coefficient, 'C', is another essential hyper-parameter that determines the regularization strength of the model. In addition, the 'solver' type, representing the optimization algorithm type, can be set to 'newton-cg', 'lbfgs', 'liblinear', 'sag', or 'saga' in LR. The 'solver' type has correlations with 'penalty' and 'C', so they are conditional hyper-parameters.

3.1.2. KNN

K-nearest neighbor (KNN) is a simple ML algorithm that is used to classify data points by calculating the distances between different data points. In KNN, the predicted class of each test sample is set to the class to which most of its k-nearest neighbors in the training set belong.

Assuming the training set $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, x_i is the feature vector of an instance, and $y_i \in \{c_1, c_2, \dots, c_m\}$ is the class of the instance, $i = (1, 2, \dots, n)$, for a test instance x , its class y can be denoted by [41]:

$$y = \arg \max_{c_j} \sum_{x_i \in N_k(x)} I(y_i = c_j), i = 1, 2, \dots, n; j = 1, 2, \dots, m, \quad (11)$$

where $I(x)$ is an indicator function, $I = 1$ when $y_i = c_j$, otherwise $I = 0$; $N_k(x)$ is the field involving the k-nearest neighbors of x .

In KNN, the number of considered nearest neighbors, k , is the most crucial hyper-parameter [42]. If k is too small, the model will be under-fitting; if k is too large, the model will be over-fitting and require high computational time. In addition, the weighted function used in the prediction can also be chosen from 'uniform' (points are weighted equally) or 'distance' (points are weighted by the inverse of their distance), depending on specific problems. The distance metric and the power parameter of the Minkowski metric can also be tuned as it can result in minor improvement. Lastly, the 'algorithm' used to compute the nearest neighbors can also be chosen from a ball tree, a k -dimensional (KD) tree, or a brute force search. Typically, the model can determine the most appropriate algorithm itself by setting the 'algorithm' to 'auto' in sklearn [31].

3.1.3. SVM

A support vector machines (SVM) [43] is a supervised learning algorithm that can be used for both classification and regression problems. SVM algorithms are based on the concept of mapping data points from low-dimensional into high-dimensional space to make them linearly separable; a hyperplane is then generated as the classification boundary to partition data points [44]. Assuming there are n data points, the objective function of SVM is [45]:

$$\arg \min_{\mathbf{w}} \left\{ \frac{1}{n} \sum_{i=1}^n \max \{0, 1 - y_i f(x_i)\} + C \mathbf{w}^T \mathbf{w} \right\}, \quad (12)$$

where $\mathbf{w}$ is a normalization vector; C is the penalty parameter of the error term, which is an important hyper-parameter of all SVM models.

The kernel function $f(x)$, which is used to measure the similarity between two data points x_i and x_j , can be chosen from multiple types of kernels in SVM models. Therefore, the kernel type would be a vital hyper-parameter to be tuned. Common kernel types in SVM include linear kernels, radial basis function (RBF), polynomial kernels, and sigmoid kernels.

The different kernel functions can be denoted as follows [46]:

1. Linear kernel:

$$f(x) = x_i^T x_j; \quad (13)$$

2. Polynomial kernel:

$$f(x) = (\gamma x_i^T x_j + r)^d; \quad (14)$$

3. RBF kernel:

$$f(x) = \exp \left(-\gamma \|x - x'\|^2 \right); \quad (15)$$

4. Sigmoid kernel:

$$f(x) = \left(\tanh \left(\gamma x_i^T x_j + r \right) \right); \quad (16)$$

As shown in the kernel function equations, a few other different hyper-parameters need to be tuned after a kernel type is chosen. The coefficient γ , denoted by 'gamma' in sklearn, is the conditional hyper-parameter of the 'kernel type' hyper-parameter when it is set to polynomial, RBF, or sigmoid; r , specified by 'coef0' in sklearn, is the conditional hyper-parameter of polynomial and sigmoid kernels. Moreover, the polynomial kernel has an additional conditional hyper-parameter d representing the 'degree' of the polynomial kernel function. In support vector regression (SVR) models, there is another hyper-parameter, 'epsilon', indicating the distance error to of its loss function [31].

3.1.4. Naïve Bayes

Naïve Bayes (NB) [47] algorithms are supervised learning algorithms based on Bayes' theorem. Assuming there are n dependent features $x_1, \dots, x_n$ and a target variable y , the objective function of naïve Bayes can be denoted by:

$$\hat{y} = \arg \max_y P(y) \prod_{i=1}^n P(x_i|y), \quad (17)$$

where $P(y)$ is the probability of a value y , and $P(x_i|y)$ is the posterior probabilities of x_i given the values of y . Regarding the different assumptions of the distribution of $P(x_i|y)$, there are different types of naïve Bayes classifiers. The four main types of NB models are: Bernoulli NB, Gaussian NB, multinomial NB, and complement NB [48].

For Gaussian NB [49], the likelihood of features is assumed to follow a Gaussian distribution:

$$P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp \left(-\frac{(x_i - \mu_y)^2}{2\sigma_y^2} \right). \quad (18)$$

The maximum likelihood method is used to calculate the mean value, μ_y , and the variance, σ_y^2 . Normally, there is not any hyper-parameter that needs to be tuned for Gaussian NB. The performance of a Gaussian NB model mainly depends on how well the dataset follows Gaussian distributions.

Multinomial NB [50] is designed for multinomially-distributed data based on the naïve Bayes algorithm. Assuming there are n features, and θ_{yi} is the distribution of each value of the target variable y , which equals the conditional probability $P(x_i|y)$ when a feature value i is involved in a data point belonging to the class y . Based on the concept of relative frequency counting, θ_y can be estimated by a smoothed version of θ_{yi} [31]:

$$\hat{\theta}_{yi} = \frac{N_{yi} + \alpha}{N_y + \alpha n}, \quad (19)$$

where N_{yi} is the number of times when feature i is in a data point belonging to class y , and N_y is the sum of all N_{yi} ($i = 0, 1, 2, \dots, n$). The smoothing priors $\alpha \geq 0$ are used for features that are not in the learning samples. When $\alpha = 1$, it is called Laplace smoothing; when $\alpha < 1$, it is called Lidstone smoothing.

Complement NB [51] is an improved version of the standard multinomial NB algorithm and is suitable for processing imbalanced data, while Bernoulli NB [52] requires samples to have binary-valued feature vectors so that the data can follow multivariate Bernoulli distributions. They both have the additive (Laplace/Lidstone) smoothing parameter, α , as the main hyper-parameter that needs tuning. To conclude, for naïve Bayes algorithms, developers often do not need to tune hyper-parameters or only need to tune the smoothing parameter α , which is a continuous hyper-parameter.

3.1.5. Tree-based Models

Decision tree (DT) [53] is a common classification method that uses a tree-structure to model decisions and possible consequences by summarizing a set of classification rules from the data. A DT has three main components: a root node representing the entire data; multiple decision nodes indicating decision tests and sub-node splits over each feature; and several leaf nodes representing the result classes [54]. DT algorithms recursively split the training set with better feature values to achieve good decisions on each subset. Pruning, which means removing some of the sub-nodes of decision nodes, is used in DT to avoid over-fitting. Since a deeper tree has more sub-trees to make more accurate decisions, the maximum tree depth, 'max_depth', is an essential hyper-parameter that controls the complexity of DT algorithms [55].

There are many other important HPs to be tuned to build effective DT models [56]. Firstly, the quality of splits can be measured by setting a mea-

measuring function, denoted by 'criterion' in sklearn. Gini impurity and information gain are the two main types of measuring functions. The split selection method, 'splitter', can also be set to 'best' to choose the best split, or 'random' to select a random split. The number of considered features to generate the best split, 'max_features', can also be tuned as a feature selection process. Moreover, there are several discrete hyper-parameters related to the splitting process: the minimum number of data points to split a decision node or to obtain a leaf node, denoted by 'min_samples_split' and 'min_samples_leaf', respectively; the 'max_leaf_nodes', indicating the maximum number of leaf nodes, and the 'min_weight_fraction_leaf' that means the minimum weighted fraction of the total weights, can also be tuned to improve model performance [31] [56].

Based on the concept of DT models, many decision-tree-based ensemble algorithms have been proposed to improve model performance by combining multiple decision trees, including random forest (RF), extra trees (ET), and extreme gradient boosting (XGBoost) models. RF [57] is an ensemble learning method that uses the bagging method to combine multiple decision trees. In RF, basic DTs are built on many randomly-generated subsets, and the class with the majority voting will be selected to be the final classification result [58]. ET [59] is another tree-based ensemble learning method that is similar to RF, but it uses all samples to build DTs and randomly selects the feature sets. In addition, RF optimizes splits on DTs while ET randomly makes the splits. XGBoost [32] is a popular tree-based ensemble model designed for speed and performance improvement, which uses the boosting and gradient descent methods to combine basic DTs. In XGBoost, the next input sample of a new DT will be related to the results of previous DTs. XGBoost aims to minimize the following objective function [55]:

$$Obj = -\frac{1}{2} \sum_{j=1}^t \frac{G_j^2}{H_j + \lambda} + \gamma t, \quad (20)$$

where t is the number of leaves in a decision tree, G and H are the sums of the first and second order gradient statistics of the cost function, γ and λ are the penalty coefficients.

Since tree-based ensemble models are built with decision trees as base learners, they have the same hyper-parameters as DT models, as described in this subsection. Apart from these hyper-parameters, RF, ET, and XGBoost all have another crucial hyper-parameter to be tuned, which is the number of

decision trees to be combined, denoted by 'n_estimators' in sklearn. XGBoost has several additional hyper-parameters, including [60]: 'min_child_weight' which means the minimum sum of weights in a child node; 'subsample' and 'colsample_bytree' used to control the subsampling ratio of instances and features, respectively; and four continuous hyper-parameters — 'gamma', 'alpha', 'lambda', and 'learning_rate' — indicating the minimum loss reduction for a split, L_1 , and L_2 regularization term on weights, and the learning rate, respectively.

3.1.6. Ensemble Learning Algorithms

Apart from tree-based ensemble models, there are several other generic ensemble learning methods that combine multiple singular ML models to achieve better model performance than any singular algorithms alone. Three common ensemble learning models — voting, bagging, and AdaBoost — are introduced in this subsection [61].

Voting [61] is a basic ensemble learning algorithm that uses the majority voting rule to combine singular estimators and generate a comprehensive estimator with improved accuracy. In sklearn, the voting method can be set to be 'hard' or 'soft', indicating whether to use majority voting or averaged predicted probabilities to determine the classification result. The list of selected single ML estimators and their weights can also be tuned in certain cases. For instance, a higher weight can be assigned to a better-performing singular ML model in a voting model.

Bootstrap aggregating [61], also named bagging, trains multiple base estimators on different randomly-extracted subsets to construct a final predictor [62]. When using bagging methods, the first consideration should be the type and number of base estimators in the ensemble, denoted by 'base_estimator' and 'n_estimators', respectively. Then, the 'max_samples' and 'max_features', indicating the sample size and feature size to generate different subsets, can also be tuned.

AdaBoost [61], short for adaptive boosting, is an ensemble learning method that trains multiple base learners consecutively (weak learners), and later learners emphasize the mis-classified samples of previous learners; ultimately, a final strong learner is obtained. During this process, incorrectly-classified instances are retrained with other new instances, and their weights are adjusted so that the subsequent classifiers focus more on difficult cases, thereby gradually building a stronger classifier. In AdaBoost, the type of base estimator, 'base_estimator', can be set to a decision tree or other methods. In

addition, the maximum number of estimators at which boosting is terminated, 'n_estimators', and the learning rate that shrinks the contribution of each classifier, should also be tuned to achieve a trade-off between these two hyper-parameters.

3.1.7. Deep Learning Models

Deep learning (DL) algorithms are widely applied to various areas — like computer vision, natural language processing, and machine translation — since they have had great success solving many types of problems. DL models are based on the theory of artificial neural networks (ANNs). Common types of DL architectures include deep neural networks (DNNs), feedforward neural networks (FFNNs), deep belief networks (DBNs), convolutional neural networks (CNNs), recurrent neural networks (RNNs) and many more [63]. All these DL models have similar hyper-parameters since they have similar underlying neural network architecture. Compared with other ML models, DL models benefit more from HPO since they often have many hyper-parameters that require tuning.

The first set of hyper-parameters is related to the construction of a DL model; hence, named model design hyper-parameters. Since all neural network models have an input layer and an output layer, the complexity of a deep learning model mainly depends on the number of hidden layers and the number of neurons in each layer, which are two main hyper-parameters to build DL models [64]. These two hyper-parameters are set and tuned according to the complexity of the datasets or the problems. DL models need to have enough capacity to model objective functions (or prediction tasks) while avoiding over-fitting. At the next stage, certain function types need to be set or tuned. The first function type to configure is the loss function type, which is chosen mainly based on the problem type (*e.g.*, binary cross-entropy for binary classification problems, multi-class cross-entropy for multi-classification problems, and RMSE for regression problems). Another important hyper-parameter is the activation function type used to model non-linear functions, which be set to 'softmax', 'rectified linear unit (ReLU)', 'sigmoid', 'tanh', or 'softsign'. Lastly, the optimizer type can be set to stochastic gradient descent (SGD), adaptive moment estimation (Adam), root mean square propagation (RMSprop), etc. [65].

On the other hand, some other hyper-parameters are related to the optimization and training process of DL models; hence, categorized as optimizer hyper-parameters. The learning rate is one of the most important hyper-

parameters in DL models [66]. It determines the step size at each iteration, which enables the objective function to converge. A large learning rate speeds up the learning process, but the gradient may oscillate around a local minimum value or even cannot converge. On the other hand, a small learning rate converges smoothly, but will largely increase model training time by requiring more training epochs. An appropriate learning rate should enable the objective function to converge to a global minimum in a reasonable amount of time. Another common hyper-parameter is the drop-out rate. Drop-out is a standard regularization method for DL models proposed to reduce over-fitting. In drop-out, a proportion of neurons are randomly removed, and the percentage of neurons to be removed should be tuned.

Mini-batch size and the number of epochs are the other two DL hyper-parameters that represent the number of processed samples before updating the model, and the number of complete passes through the entire training set, respectively [67]. Mini-batch size is affected by the resource requirements of the training process and the number of iterations. The number of epochs depends on the size of the training set and should be tuned by slowly increasing its value until validation accuracy starts to decrease, which indicates over-fitting. On the other hand, DL models often converge within a few epochs, and the following epochs may lead to unnecessary additional execution time and over-fitting, which can be avoided by the early stopping method. Early stopping is a form of regularization whereby model training stops in advance when validation accuracy does not increase after a certain number of consecutive epochs. The number of waiting epochs, called early stop patience, can also be tuned to reduce model training time.

Apart from traditional DL models, transfer learning (TL) is a technology that obtains a pre-trained model on the data in a related domain and transfers it to other target tasks [68]. To transfer a DL model from one problem to another problem, a certain number of top layers are frozen, and only the remaining layers are retrained to fit the new problem. Therefore, the number of frozen layers is a vital hyper-parameter to tune if TL is used.

3.2. *Unsupervised Learning Algorithms*

Unsupervised learning algorithms are a set of ML algorithms used to identify unknown patterns in unlabeled datasets. Clustering and dimensionality-reduction algorithms are the two main types of unsupervised learning methods. Clustering methods include k-means, DBSCAN, EM, hierarchical clustering, etc.; while PCA and LDA are two commonly-used dimensionality

reduction algorithms [30].

3.2.1. Clustering Algorithms

In most clustering algorithms — including k-means, EM, and hierarchical clustering — the number of clusters is the most important hyper-parameter to tune [69].

The k-means algorithm [70] uses k prototypes, indicating the centroids of clusters, to cluster data. In k-means algorithms, the number of clusters, 'n_clusters', must be specified, and is determined by minimizing the sum of squared errors [70]:

$$\sum_{i=0}^{n_k} \min_{u_j \in C_k} (\mathbf{x}_i - u_j)^2, \quad (21)$$

where $(\mathbf{x}_1, \dots, \mathbf{x}_n)$ is the data matrix; u_j , also called the centroid of the cluster C_k , is the mean of the samples in the cluster; and n_k is the number of sample points in the cluster C_k .

To tune k-means, 'n_clusters' is the most crucial hyper-parameter. Besides this, the method for centroid initialization, 'init', could be set to 'k-means++', 'random' or a human-defined array, which slightly affects model performance. In addition, 'n_init', denoting the number of times that the k-means algorithm will be executed with different centroid seeds, and the 'max_iter', the maximum number of iterations in a single execution of k-means, also have slight impacts on model performance [31].

The expectation-maximization (EM) algorithm [71] is an iterative algorithm used to detect the maximum likelihood estimation of parameters. Gaussian Mixture model is a clustering method that uses a mixture of Gaussian distributions to model data by implementing the EM method. Similar to k-means, its major hyper-parameter to be tuned is 'n_components', indicating the number of clusters or Gaussian distributions. Additionally, different methods can be chosen to constrain the covariance of the estimated classes in Gaussian mixture models, including 'full covariance', 'tied', 'diagonal' or 'spherical' [72]. Other hyper-parameters could also be tuned, including 'max_iter' and 'tol', representing the number of EM iterations to perform and the convergence threshold, respectively [31].

Hierarchical clustering [73] methods build clusters by continuously merging or splitting the built-in clusters. The hierarchy of clusters is represented by a tree-structure; its root indicates the unique cluster gathering all samples, and its leaves represent the clusters with only one sample [73]. In sklearn, the

function 'AgglomerativeClustering' is a common type of hierarchical clustering. In agglomerative clustering, the linkage criteria, 'linkage', determines the distance between sets of observations and can be set to 'ward', 'complete', 'average', or 'single', indicating whether to minimize the variance of the all clusters, or use the maximum, average, or minimum distance between every two clusters, respectively. Like other clustering methods, its main hyper-parameter is the number of clusters, 'n_clusters'. However, 'n_clusters' cannot be set if we choose to set the 'distance_threshold', the linkage distance threshold for merging clusters, since if so, 'n_clusters' will be determined automatically.

DBSCAN [74] is a density-based clustering method that determines the clusters by dividing data into clusters with sufficiently high density. Unlike other clustering models, the number of clusters does not need to be configured before training. Instead, DBSCAN has two significant conditional hyper-parameters — the scan radius represented by 'eps', and the minimum number of considered neighbor points represented by 'min_samples' — which define the cluster density together [75]. DBSCAN works by starting with an unvisited point and detecting all its neighbor points within a pre-defined distance 'eps'. If the number of neighbor points reaches the value of 'min_samples', this unvisited point and all its neighbors are defined as a cluster. The procedures are executed recursively until all data points have been visited. A higher 'min_samples' or a lower 'eps' indicates a higher density to form a cluster.

3.2.2. Dimensionality Reduction Algorithms

The increasing amount of collected data provides ample information, but also increases problem complexity. In real-world applications, many features are irrelevant or redundant to predict target variables. Dimensionality reduction algorithms often serve as feature engineering methods to extract important features and eliminate insignificant or redundant features. Two common dimensionality-reduction algorithms are principal component analysis (PCA) and linear discriminant analysis (LDA). In PCA and LDA, the number of features to be extracted, represented by 'n_components' in sklearn, is the main hyper-parameter to be tuned.

Principal component analysis (PCA) [76] is a widely used linear dimensionality reduction method. PCA is based on the concept of mapping the original n -dimensional features into k -dimension features as the new orthogonal features, also called the principal components. PCA works by calculat-

ing the covariance matrix of the data matrix to obtain the eigenvectors of the covariance matrix. The matrix comprises the eigenvectors of k features with the largest eigenvalues (*i.e.*, the largest variance). Consequently, the data matrix can be transformed into a new space with reduced dimensionality. Singular value decomposition (SVD) [77] is a popular method used to obtain the eigenvalues and eigenvectors of the covariance matrix of PCA. Therefore, in addition to 'n_components', the SVD solver type is another hyper-parameter of PCA to be tuned, which can be assigned to 'auto', 'full', 'arpack' or 'randomized' [31].

Linear discriminant analysis (LDA) [78] is another common dimensionality reduction method that projects the features onto the most discriminative directions. Unlike PCA, which obtains the direction with the largest variance as the principal component, LDA optimizes the feature subspace of classification. The objective of LDA is to minimize the variance inside each class and maximize the variance between different classes after projection. Thus, the projection points in each class should be as close as possible, and the distance between the center points of different classes should be as large as possible. Similar to PCA, the number of features to be extracted, 'n_components', should be tuned in LDA models. Additionally, the solver type of LDA can also be set to 'svd' for SVD, 'lsqr' for least-squares solution, or 'eigen' for eigenvalue decomposition [79]. LDA also has a conditional hyper-parameter, the shrinkage parameter, 'shrinkage', which can be set to a float value along with 'lsqr' and 'eigen' solvers.

4. Hyper-parameter Optimization Techniques

4.1. Model-free Algorithms

4.1.1. Babysitting

Babysitting, also called 'Trial and Error' or grad student descent (GSD), is a basic hyper-parameter tuning method [8]. This method is implemented by 100% manual tuning and widely used by students and researchers. The workflow is simple: after building a ML model, a student tests many possible hyper-parameter values based on experience, guessing, or the analysis of previously-evaluated results; the process is repeated until this student runs out of time (often reaching a deadline) or is satisfied with the results. As such, this approach requires a sufficient amount of prior knowledge and experience to identify optimal hyper-parameter values with limited time.

Manual tuning is infeasible for many problems due to several factors, like a large number of hyper-parameters, complex models, time-consuming model evaluations, and non-linear hyper-parameter interactions [9]. These factors inspired increased research into techniques for the automatic optimization of hyper-parameters [80].

4.1.2. Grid Search

Grid search (GS) is one of the most commonly-used methods to explore hyper-parameter configuration space [81]. GS can be considered an exhaustive search or a brute-force method that evaluates all the hyper-parameter combinations given to the grid of configurations [82]. GS works by evaluating the Cartesian product of a user-specified finite set of values [6].

GS cannot exploit the well-performing regions further by itself. Therefore, to identify the global optimum, the following procedure needs to be performed manually [2]:

1. Start with a large search space and step size.
2. Narrow the search space and step size based on the previous results of well-performing hyper-parameter configurations.
3. Repeat step 2 multiple times until an optimum is reached.

GS can be easily implemented and parallelized. However, the main drawback of GS is its inefficiency for high-dimensionality hyper-parameter configuration space, since the number of evaluations increases exponentially as the number of hyper-parameters grows. This exponential growth is referred to as the curse of dimensionality [83]. For GS, assuming that there are k parameters, and each of them has n distinct values, its computational complexity increases exponentially at a rate of $O(n^k)$ [19]. Thus, only when the hyper-parameter configuration space is small can GS be an effective HPO method.

4.1.3. Random Search

To overcome certain limitations of GS, random search (RS) was proposed in [13]. RS is similar to GS; but, instead of testing all values in the search space, RS randomly selects a pre-defined number of samples between the upper and lower bounds as candidate hyper-parameter values, and then trains these candidates until the defined budget is exhausted. The theoretical basis of RS is that if the configuration space is large enough, then the global

optimums, or at least their approximations, can be detected. With a limited budget, RS is able to explore a larger search space than GS [13].

The main advantage of RS is that it is easily parallelized and resource-allocated since each evaluation is independent. Unlike GS, RS samples a fixed number of parameter combinations from the specified distribution, which improves system efficiency by reducing the probability of wasting much time on a small poor-performing region. Since the number of total evaluations in RS is set to a fixed value n before the optimization process starts, the computational complexity of RS is $O(n)$ [84]. In addition, RS can detect the global optimum or the near-global optimum when given enough budgets [6].

Although RS is more efficient than GS for large search spaces, there are still a large number of unnecessary function evaluations since it does not exploit the previously well-performing regions [2].

To conclude, the main limitation of both RS and GS is that every evaluation in their iterations is independent of previous evaluations; thus, they waste massive time evaluating poorly-performing areas of the search space. This issue can be solved by other optimization methods, like Bayesian optimization that uses previous evaluation records to determine the next evaluation [14].

4.2. Gradient-based Optimization

Gradient descent [85] is a traditional optimization technique that calculates the gradient of variables to identify the promising direction and moves towards the optimum. After randomly selecting a data point, the technique moves towards the opposite direction of the largest gradient to locate the next data point. Therefore, a local optimum can be reached after convergence. The local optimum is also the global optimum for convex functions. Gradient-based algorithms have a time complexity of $O(n^k)$ for optimizing k hyper-parameters [86].

For specific machine learning algorithms, the gradient of certain hyper-parameters can be calculated, and then the gradient descent can be used to optimize these hyper-parameters. Although gradient-based algorithms have a faster convergence speed to reach local optimum than the previously-presented methods in Section 4.1, they have several limitations. Firstly, they can only be used to optimize continuous hyper-parameters because other types of hyper-parameters, like categorical hyper-parameters, do not have gradient directions. Secondly, they are only efficient for convex functions because the local instead of a global optimum may be reached for non-convex

functions [2]. Therefore, the gradient-based algorithms can only be used in some cases where it is possible to obtain the gradient of hyper-parameters; *e.g.*, optimizing the learning rate in neural networks (NN) [11]. Still, it is not guaranteed for ML algorithms to identify global optimums using gradient-based optimization techniques.

4.3. Bayesian Optimization

Bayesian optimization (BO) [87] is an iterative algorithm that is popularly used for HPO problems. Unlike GS and RS, BO determines the future evaluation points based on the previously-obtained results. To determine the next hyper-parameter configuration, BO uses two key components: a surrogate model and an acquisition function [57]. The surrogate model aims to fit all the currently-observed points into the objective function. After obtaining the predictive distribution of the probabilistic surrogate model, the acquisition function determines the usage of different points by balancing the trade-off between exploration and exploitation. Exploration is to sample the instances in the areas that have not been sampled, while exploitation is to sample in the currently promising regions where the global optimum is most likely to occur, based on the posterior distribution. BO models balance the exploration and the exploitation processes to detect the current most likely optimal regions and avoid missing better configurations in the unexplored areas [88].

The basic procedures of BO are as follows [87]:

1. Build a probabilistic surrogate model of the objective function.
2. Detect the optimal hyper-parameter values on the surrogate model.
3. Apply these hyper-parameter values to the real objective function to evaluate them.
4. Update the surrogate model with new results.
5. Repeat steps 2 - 4 until the maximum number of iterations is reached.

Thus, BO works by updating the surrogate model after each evaluation on the objective function. BO is more efficient than GS and RS since it can detect the optimal hyper-parameter combinations by analyzing the previously-tested values, and running a surrogate model is often much cheaper than running the entire objective function.

However, since Bayesian optimization models are executed based on the previously-tested values, they belong to sequential methods that are diffi-

cult to parallelize; but they can usually detect near-optimal hyper-parameter combinations within a few iterations [7].

Common surrogate models for BO include Gaussian process (GP) [89], random forest (RF) [90], and the tree Parzen estimator (TPE) [12]. Therefore, there are three main types of BO algorithms based on their surrogate models: BO-GP, BO-RF, BO-TPE. An alternative name for BO-RF is sequential model-based algorithm configuration (SMAC) [90].

4.3.1. BO-GP

Gaussian process (GP) is a standard surrogate model for objective function modeling in BO [87]. Assuming that the function f with a mean μ and a covariance σ^2 is a realization of a GP, the predictions follow a normal distribution [91]:

$$p(y|x, D) = N(y|\hat{\mu}, \hat{\sigma}^2), \quad (22)$$

where D is the configuration space of hyper-parameters, and $y = f(x)$ is the evaluation result of each hyper-parameter value x . After obtaining a set of predictions, the points to be evaluated next are then selected from the confidence intervals generated by the BO-GP model. Each newly-tested data point is added to the sample records, and the BO-GP model is re-built with the new information. This procedure is repeated until termination.

Applying a BO-GP to a size n dataset has a time complexity of $O(n^3)$ and space complexity of $O(n^2)$ [92]. One main limitation of BO-GP is that the cubic complexity to the number of instances limits the capacity for parallelization [3]. Additionally, it is mainly used to optimize continuous variables.

4.3.2. SMAC

Random forest (RF) is another popular surrogate function for BO to model the objective function using an ensemble of regression trees. BO using RF as the surrogate model is also called SMAC [90].

Assuming that there is a Gaussian model $N(y|\hat{\mu}, \hat{\sigma}^2)$, and $\hat{\mu}$ and $\hat{\sigma}^2$ are the mean and variance of the regression function $r(x)$, respectively, then [90]:

$$\hat{\mu} = \frac{1}{|B|} \sum_{r \in B} r(x), \quad (23)$$

$$\hat{\sigma}^2 = \frac{1}{|B| - 1} \sum_{r \in B} (r(x) - \hat{\mu})^2, \quad (24)$$

where B is a set of regression trees in the forest. The major procedures of SMAC are as follows [3]:

1. RF starts with building B regression trees, each constructed by sampling n instances from the training set with replacement.
2. A split node is selected from d hyper-parameters for each tree.
3. To maintain a low computational cost, both the minimum number of instances considered for further split and the number of trees to grow are set to a certain value.
4. Finally, the mean and variance for each new configuration are estimated by RF.

Compared with BO-GP, the main advantage of SMAC is its support for all types of variables, including continuous, discrete, categorical, and conditional hyper-parameters [91]. The time complexities of using SMAC to fit and predict variances are $O(n \log n)$ and $O(\log n)$, respectively, which are much lower than the complexities of BO-GP [3].

4.3.3. BO-TPE

Tree-structured Parzen estimator (TPE) [12] is another common surrogate model for BO. Instead of defining a predictive distribution used in BO-GP, BO-TPE creates two density functions, $l(x)$ and $g(x)$, to act as the generative models for all domain variables [3]. To apply TPE, the observation results are divided into good results and poor results by a pre-defined percentile y^* , and the two sets of results are modeled by simple Parzen windows [12]:

$$p(x|y, D) = \begin{cases} l(x), & \text{if } y < y^* \\ g(x), & \text{if } y > y^* \end{cases} . \quad (25)$$

After that, the expected improvement in the acquisition function is reflected by the ratio between the two density functions, which is used to determine the new configurations for evaluation. The Parzen estimators are organized in a tree structure, so the specified conditional dependencies are retained. Therefore, TPE naturally supports specified conditional hyper-parameters [91]. The time complexity of BO-TPE is $O(n \log n)$, which is lower than the complexity of BO-GP [3].

BO methods are effective for many HPO problems, even if the objective function f is stochastic, non-convex, or non-continuous. However, the main drawback of BO models is that, if they fail to achieve the balance between

exploration and exploitation, they might only reach a local instead of a global optimum. RS does not have this limitation since it does not focus on any specific area. Additionally, it is difficult to parallelize BO models since their intermediate results are dependent on each other [7].

4.4. Multi-fidelity Optimization Algorithms

One major issue with HPO is the long execution time, which increases with a larger hyper-parameter configuration space and larger datasets. The execution time may be several hours, several days, or even more [93]. Multi-fidelity optimization techniques are common approaches to solve the constraint of limited time and resources. To save time, people can use a subset of the original dataset or a subset of the features [94]. Multi-fidelity involves low-fidelity and high-fidelity evaluations and combines them for practical applications [95]. In low-fidelity evaluations, a relatively small subset is evaluated at a low cost but with poor generalization performance. In high-fidelity evaluations, a relatively large subset is evaluated with better generalization performance but at a higher cost than low-fidelity evaluations. In multi-fidelity optimization algorithms, poorly-performing configurations are discarded after each round of hyper-parameter evaluation on generated subsets, and only well-performing hyper-parameter configurations will be evaluated on the entire training set.

Bandit-based algorithms categorized to multi-fidelity optimization algorithms have shown success dealing with deep learning optimization problems [3]. Two common bandit-based techniques are successive halving [96] and Hyperband [16].

4.4.1. Successive Halving

Theoretically speaking, exhaustive methods are able to identify the optimal hyper-parameter combination by evaluating all the given combinations. However, many factors, including limited time and resources, should be considered in practical applications. These factors are called budgets (B). To overcome the limitations of GS and RS and to improve efficiency, successive halving algorithms were proposed in [96].

The main process of using successive halving algorithms for HPO is as follows. Firstly, it is presumed that there are n sets of hyper-parameter combinations, and that they are evaluated with uniformly-allocated budgets ($b = B/n$). Then, according to the evaluation results for each iteration, half of the poorly-performing hyper-parameter configurations are eliminated, and

the better-performing half is passed to the next iteration with double budgets ($b_{i+1} = 2 * b_i$). The above process is repeated until the final optimal hyper-parameter combination is detected.

Successive halving is more efficient than RS, but is affected by the trade-off between the number of hyper-parameter configurations and the budgets allocated to each configuration [6]. Thus, the main concern of successive halving is how to allocate the budget and how to determine whether to test fewer configurations with a higher budget for each or to test more configurations with a lower budget for each [2].

4.4.2. Hyperband

Hyperband [16] is then proposed to solve the dilemma of successive halving algorithms by dynamically choosing a reasonable number of configurations. It aims to achieve a trade-off between the number of hyper-parameter configurations (n) and their allocated budgets by dividing the total budgets (B) into n pieces and allocating these pieces to each configuration ($b = B/n$). Successive halving serves as a subroutine on each set of random configurations to eliminate the poorly-performing hyper-parameter configurations and improve efficiency. The main steps of Hyperband algorithms are shown in Algorithm 1 [2].

Algorithm 1 Hyperband

Input: $b_{\max}, b_{\min}$

- 1: $s_{\max} = \log \left(\frac{b_{\max}}{b_{\min}} \right)$
- 2: **for** $s \in \{b_{\max}, b_{\min} - 1, \dots, 0\}$ **do**
- 3: $n = \text{DetermineBudget}(s)$
- 4: $\gamma = \text{SampleConfigurations}(n)$
- 5: $\text{SuccessiveHalving}(\gamma)$
- 6: **end for**
- 7: **return** *The best configuration so far.*

Firstly, the budget constraints $b_{\min}$ and $b_{\max}$ are determined by the total number of data points, the minimum number of instances required to train a sensible model, and the available budgets. After that, the number of configurations n and the budget size allocated to each configuration are calculated based on $b_{\min}$ and $b_{\max}$ in steps 2-3 of Algorithm 1. The configurations are sampled based on n and b , and then passed to the successive halving model

demonstrated in steps 4-5. The successive halving algorithm discards the identified poorly-performing configurations and passes the well-performing configurations on to the next iteration. This process is repeated until the final optimal hyper-parameter configuration is identified. By involving the successive halving searching method, Hyperband has a computational complexity of $O(n \log n)$ [16].

4.4.3. BOHB

Bayesian Optimization HyperBand (BOHB) [97] is a state-of-the-art HPO technique that combines Bayesian optimization and Hyperband to incorporate the advantages of both while avoiding their drawbacks. The original Hyperband uses a random search to search the hyper-parameter configuration space, which has a low efficiency. BOHB replaces the RS method by BO to achieve both high performance as well as low execution time by effectively using parallel resources to optimize all types of hyper-parameters. In BOHB, TPE is the standard surrogate model for BO, but it uses multidimensional kernel density estimators. Therefore, the complexity of BOHB is also $O(n \log n)$ [97].

It has been shown that BOHB outperforms many other optimization techniques when tuning SVM and DL models [97]. The only limitation of BOHB is that it requires the evaluations on subsets with small budgets to be representative of evaluations on the entire training set; otherwise, BOHB may have a slower convergence speed than standard BO models.

4.5. Metaheuristic Algorithms

Metaheuristic algorithms [98] are a set of algorithms mainly inspired by biological theories and widely used for optimization problems. Unlike many traditional optimization methods, metaheuristics have the capacity to solve non-convex, non-continuous, and non-smooth optimization problems.

Population-based optimization algorithms (POAs) are a major type of metaheuristic algorithm, including genetic algorithms (GAs), evolutionary algorithms, evolutionary strategies, and particle swarm optimization (PSO). POAs start by creating and updating a population as each generation; each individual in every generation is then evaluated until the global optimum is identified [14]. The main differences between different POAs are the methods used to generate and select populations [17]. POAs can be easily parallelized since a population of N individuals can be evaluated on at most N threads or

machines in parallel [6]. Genetic algorithms and particle swarm optimization are the two main POAs that are popularly-used for HPO problems.

4.5.1. Genetic Algorithm

Genetic algorithm (GA) [18] is one of the common metaheuristic algorithms based on the evolutionary theory that individuals with the best survival capability and adaptability to the environment are more likely to survive and pass on their capabilities to future generations. The next generation will also inherit their parents' characteristics and may involve better and worse individuals. Better individuals will be more likely to survive and have more capable offspring, while the worse individuals will gradually disappear. After several generations, the individual with the best adaptability will be identified as the global optimum [99].

To apply GA to HPO problems, each chromosome or individual represents a hyper-parameter, and its decimal value is the actual input value of the hyper-parameter in each evaluation. Every chromosome has several genes, which are binary digits; and then crossover and mutation operations are performed on the genes of this chromosome. The population involves all possible values within the initialized chromosome/parameter ranges, while the fitness function characterizes the evaluation metrics of the parameters [99].

Since the randomly-initialized parameter values often do not include the optimal parameter values, several operations, including selection, crossover, and mutation operations, must be performed on the well-performing chromosomes to identify the optimums [18]. Chromosome selection is implemented by selecting those chromosomes with good fitness function values. To keep the population size unchanged, the chromosomes with good fitness function values are passed to the next generation with higher probability, where they generate new chromosomes with the parents' best characteristics. Chromosome selection ensures that good characteristics of each generation can be passed to later generations. Crossover is used to generate new chromosomes by exchanging a proportion of genes in different chromosomes. Mutation operations are also used to generate new chromosomes by randomly altering one or more genes of a chromosome. Crossover and mutation operations enable later generations to have different characteristics and reduce the chance of missing good characteristics [3].

The main procedures of GA are as follows [98]:

1. Randomly initialize the population, chromosomes, and genes, which

represent the entire search space, hyper-parameters, and hyper-parameter values, respectively.

2. Evaluate the performance of each individual in the current generation by calculating the fitness function, which indicates the objective function of a ML model.
3. Perform selection, crossover, and mutation operations on the chromosomes to produce a new generation involving the next hyper-parameter configurations to be evaluated.
4. Repeat steps 2 & 3 until the termination condition is met.
5. Terminate and output the optimal hyper-parameter configuration.

Among the above steps, the population initialization step is an important step of GA and PSO since it provides an initial guess of the optimal values. Although the initialized values will be iteratively improved in the optimization process, a suitable population initialization method can significantly improve the convergence speed and performance of POAs. A good initial population of hyper-parameters should involve individuals that are close to global optimums by covering the promising regions and should not be localized to an unpromising region of the search space [100].

To generate hyper-parameter configuration candidates for the initial population, random initialization that simply creates the initial population with random values in the given search space is often used in GA [101]. Thus, GA is easily implemented and does not necessitate good initializations, because its selection, crossover, and mutation operations lower the possibility of missing the global optimum.

Hence, it is useful when the data analyst does not have much experience determining a potential appropriate initial search space for the hyper-parameters. The main limitation of GA is that the algorithm itself introduces additional hyper-parameters to be configured, including the fitness function type, population size, crossover rate, and mutation rate. Moreover, GA is a sequential execution algorithm, making it difficult to parallelize. The time complexity of GA is $O(n^2)$ [102]. As a result, sometimes, GA may be inefficient due to its low convergence speed.

4.5.2. Particle Swarm Optimization

Particle swarm optimization (PSO) [103] is another set of evolutionary algorithms that are commonly used for optimization problems. PSO algorithms are inspired by biological populations that exhibit both individual and

social behaviors [17]. PSO works by enabling a group of particles (swarm) to traverse the search space in a semi-random manner [9]. PSO algorithms identify the optimal solution through cooperation and information sharing among individual particles in a group.

In PSO, there are a group of n particles in a swarm $\mathbf{S}$ [2]:

$$\mathbf{S} = (S_1, S_2, \dots, S_n), \quad (26)$$

and each particle S_i is represented by a vector:

$$S_i = \langle \vec{x}_i, \vec{v}_i, \vec{p}_i \rangle, \quad (27)$$

where $\vec{x}_i$ is the current position, $\vec{v}_i$ is the current velocity, and $\vec{p}_i$ is the known best position of the particle so far.

After initializing the position and velocity of each particle, every particle evaluates the current position and records the position with its performance score. In the next iteration, the velocity $\vec{v}_i$ of each particle is changed based on the previous position $\vec{p}_i$ and the current global optimal position $\vec{p}$:

$$\vec{v}_i := \vec{v}_i + U(0, \varphi_1)(\vec{p}_i - \vec{x}_i) + U(0, \varphi_2)(\vec{p} - \vec{x}_i), \quad (28)$$

where $U(0, \varphi)$ is the continuous uniform distributions based on the acceleration constants φ_1 and φ_2 .

After that, the particles move based on their new velocity vectors:

$$\vec{x}_i := \vec{x}_i + \vec{v}_i. \quad (29)$$

The above procedures are repeated until convergence or termination constraints are reached.

Compared with GA, it is easier to implement PSO, since PSO does not have certain additional operations like crossover and mutation. In GA, all chromosomes share information with each other, so the entire population moves uniformly toward the optimal region; while in PSO, only information on the individual best particle and the global best particle is transmitted to others, which is a one-way flow of information sharing, and the entire search process follows the direction of the current optimal solution [2]. The computational complexity of PSO algorithm is $O(n \log n)$ [104]. In most cases, the convergence speed of PSO is faster than of GA. In addition, particles in PSO operate independently and only need to share information with each

other after each iteration, so this process is easily parallelized to improve model efficiency [9].

The main limitation of PSO is that it requires proper population initialization; otherwise, it might only reach a local instead of a global optimum, especially for discrete hyper-parameters [105]. Proper population initialization requires developers’ prior experience or using population initialization techniques. Many population initialization techniques have been proposed to improve the performance of evolutionary algorithms, like the opposition-based optimization algorithm [101] and the space transformation search method [106]. Involving additional population initialization techniques will require more execution time and resources.

5. Applying Optimization Techniques to Machine Learning Algorithms

5.1. Optimization Techniques Analysis

Grid search (GS) is a simple method, its major limitation being that it is time-consuming and impacted by the curse of dimensionality [83]. Thus, it is unsuitable for a large number of hyper-parameters. Moreover, GS is often not able to detect the global optimum of continuous parameters, since it requires a pre-defined, finite set of hyper-parameter values. It is also unrealistic for GS to be used to identify integer and continuous hyper-parameter optimums with limited time and resources. Therefore, compared with other techniques, GS is only efficient for a small number of categorical hyper-parameters.

Random search is more efficient than GS and supports all types of hyper-parameters. In practical applications, using RS to evaluate the randomly-selected hyper-parameter values helps analysts to explore a large search space. However, since RS does not consider previously-tested results, it may involve many unnecessary evaluations, which decrease its efficiency [13].

Hyperband can be considered an improved version of RS, and they both support parallel executions. Hyperband balances model performance and resource usage, so it is more efficient than RS, especially with limited time and resources [15]. However, GS, RS, and Hyperband all have a major constraint in that they treat each hyper-parameter independently and do not consider hyper-parameter correlations [107]. Thus, they will be inefficient for ML algorithms with conditional hyper-parameters, like SVM, DBSCAN, and logistic regression.

Gradient-based algorithms are not a prevalent choice for hyper-parameter optimization, since they only support continuous hyper-parameters and can only detect a local instead of a global optimum for non-convex HPO problems [2]. Therefore, gradient-based algorithms can only be used to optimize certain hyper-parameters, like the learning rate in DL models.

Bayesian optimization models are divided into three different models — BO-GP, SMAC, and BO-TPE — based on their surrogate models. BO algorithms determine the next hyper-parameter value based on the previously-evaluated results to reduce unnecessary evaluations and improve efficiency. BO-GP mainly supports continuous and discrete hyper-parameters (by rounding them), but does not support conditional hyper-parameters [14]; while SMAC and BO-TPE are both able to handle categorical, discrete, continuous, and conditional hyper-parameters. SMAC performs better when there are many categorical and conditional parameters, or cross-validation is used, while BO-GP performs better for only a few continuous parameters [15]. BO-TPE preserves the specified conditional relationships, so one advantage of BO-TPE over BO-GP is its innate support for specified conditional hyper-parameters [14].

Metaheuristic algorithms, including GA and PSO, are more complicated than many other HPO algorithms, but often perform well for complex optimization problems. They support all types of hyper-parameters and are particularly efficient for large configuration spaces, since they can obtain the near-optimal solutions even within very few iterations. However, GA and PSO have their own advantages and disadvantages in practical use. PSO is able to support large-scale parallelization, and is particularly suitable for continuous and conditional HPO problems [19]; on the other hand, GA is executed sequentially, making it difficult to be parallelized. Therefore, PSO often executes faster than GA, especially for large configuration spaces and large datasets. However, an appropriate population initialization is crucial for PSO; otherwise, it may converge slowly or only identify a local instead of a global optimum. Yet, the impact of proper population initialization is not as significant for GA as for PSO [108]. Another limitation of GA is that it introduces additional hyper-parameters, like its crossover and mutation rates [18].

The strengths and limitations of the hyper-parameter optimization algorithms involved in this paper are summarized in Table 1.

Table 1: The comparison of common HPO algorithms (n is the number of hyper-parameter values and k is the number of hyper-parameters)

HPO Method	Strengths	Limitations	Time Complexity
GS	· Simple.	· Time-consuming, · Only efficient with categorical HPs.	$O(n^k)$
RS	· More efficient than GS. · Enable parallelization.	· Not consider previous results. · Not efficient with conditional HPs.	$O(n)$
Gradient-based models	· Fast convergence speed for continuous HPs.	· Only support continuous HPs. · May only detect local optimums.	$O(n^k)$
BO-GP	· Fast convergence speed for continuous HPs.	· Poor capacity for parallelization. · Not efficient with conditional HPs.	$O(n^3)$
SMAC	· Efficient with all types of HPs.	· Poor capacity for parallelization.	$O(n \log n)$
BO-TPE	· Efficient with all types of HPs. · Keep conditional dependencies.	· Poor capacity for parallelization.	$O(n \log n)$
Hyperband	· Enable parallelization.	· Not efficient with conditional HPs. · Require subsets with small budgets to be representative.	$O(n \log n)$
BOHB	· Efficient with all types of HPs. · Enable parallelization.	· Require subsets with small budgets to be representative.	$O(n \log n)$
GA	· Efficient with all types of HPs. · Not require good initialization.	· Poor capacity for parallelization.	$O(n^2)$
PSO	· Efficient with all types of HPs. · Enable parallelization.	· Require proper initialization.	$O(n \log n)$

5.2. Apply HPO Algorithms to ML Models

Since there are many different HPO methods for different use cases, it is crucial to select the appropriate optimization techniques for different ML models.

Firstly, if we have access to multiple fidelities, which means that it is able to define meaningful budgets: the performance rankings of hyper-parameter configurations evaluated on small budgets should be the same as or similar to the configuration rankings on the full budget (the original dataset); BOHB would be the best choice, since it has the advantages of both BO and Hyperband [6] [97].

On the other hand, if multiple fidelities are not applicable, which means that using the subsets of the original dataset or the subsets of original features is misleading or too noisy to reflect the performance of the entire dataset, BOHB may perform poorly with higher time complexity than standard BO

models, then choosing other HPO algorithms would be more efficient [97].

ML algorithms can be classified by the characteristics of their hyper-parameter configurations. Appropriate optimization algorithms can be chosen to optimize the hyper-parameters based on these characteristics.

5.2.1. One Discrete Hyper-parameter

Commonly for some ML algorithms, like certain neighbor-based, clustering, and dimensionality reduction algorithms, only one discrete hyper-parameter needs to be tuned. For KNN, the major hyper-parameter is k , the number of considered neighbors. The most essential hyper-parameter of k-means, hierarchical clustering, and EM is the number of clusters. Similarly, for dimensionality reduction algorithms, including PCA and LDA, their basic hyper-parameter is 'n_components', the number of features to be extracted.

In these situations, Bayesian optimization is the best choice, and the three surrogates could be tested to find the best one. Hyperband is another good choice, which may have a fast execution speed due to its capacity for parallelization. In some cases, people may want to fine-tune the ML model by considering other less important hyper-parameters, like the distance metric of KNN and the SVD solver type of PCA; so BO-TPE, GA, or PSO could be chosen for these situations.

5.2.2. One Continuous Hyper-parameter

Some linear models, including ridge and lasso algorithms, and some naïve Bayes algorithms, involving multinomial NB, Bernoulli NB, and complement NB, generally only have one vital continuous hyper-parameter to be tuned. In ridge and lasso algorithms, the continuous hyper-parameter is 'alpha', the regularization strength. In the three NB algorithms mentioned above, the critical hyper-parameter is also named 'alpha', but it represents the additive (Laplace/Lidstone) smoothing parameter. In terms of these ML algorithms, BO-GP is the best choice, since it is good at optimizing a small number of continuous hyper-parameters. Gradient-based algorithms can also be used, but might only detect local optimums, so they are less effective than BO-GP.

5.2.3. A Few Conditional Hyper-parameters

It is noticeable that many ML algorithms have conditional hyper-parameters, like SVM, LR, and DBSCAN. LR has three correlated hyper-parameters, 'penalty', 'C', and the solver type. Similarly, DBSCAN has 'eps' and 'min_samples' that must be tuned in conjunction. SVM is more complex, since after setting

a different kernel type, there is a separate set of conditional hyper-parameters that need to be tuned next, as described in Section 3.1.3. Hence, some HPO methods that cannot effectively optimize conditional hyper-parameters, including GS, RS, BO-GP, and Hyperband, are not suitable for ML models with conditional hyper-parameters. For these ML methods, BO-TPE is the best choice if we have pre-defined relationships among the hyper-parameters. SMAC is also a good choice, since it also performs well for tuning conditional hyper-parameters. GA and PSO can be used, as well.

5.2.4. A Large Hyper-parameter Configuration Space with Multiple Types of Hyper-parameters

Tree-based algorithms, including DT, RF, ET, and XGBoost, as well as DL algorithms, like DNN, CNN, RNN, are the most complex types of ML algorithms to be tuned, since they have many hyper-parameters with various, different types. For these ML models, PSO is the best choice since it enables parallel executions to improve efficiency, particularly for DL models that often require massive training time. Some other techniques, like GA, BO-TPE, and SMAC can also be used, but they may cost more time than PSO, since it is difficult to parallelize these techniques.

5.2.5. Categorical Hyper-parameters

This category of hyper-parameters is mainly for ensemble learning algorithms, since their major hyper-parameter is a categorical hyper-parameter. For bagging and AdaBoost, the categorical hyper-parameter is 'base_estimator', which is set to be a singular ML model. For voting, it is 'estimators', indicating a list of ML singular models to be combined. The voting method has another categorical hyper-parameter, 'voting', which is used to choose whether to use a hard or soft voting method. If we only consider these categorical hyper-parameters, GS would be sufficient to detect their suitable base machine learners. On the other hand, in many cases, other hyper-parameters need to be considered, like 'n_estimators', 'max_samples', and 'max_features' in bagging, as well as 'n_estimators' and 'learning_rate' in AdaBoost; consequently, BO algorithms would be a better choice to optimize these continuous or discrete hyper-parameters.

In conclusion, when tuning a ML model to achieve high model performance and low computational costs, the most suitable HPO algorithm should be selected based on the properties of its hyper-parameters.

6. Existing HPO Frameworks

To tackle HPO problems, many open-source libraries exist to apply theory into practice and lower the threshold for ML developers. In this section, we provide a brief introduction to some popular open-source HPO libraries or frameworks mainly for Python programming. The principles behind the involved optimization algorithms are provided in Section 4.

6.1. *Sklearn*

In sklearn [31], 'GridSearchCV' can be implemented to detect the optimal hyper-parameters using the GS algorithm. Each hyper-parameter value in the human-defined configuration space is evaluated by the program, with its performance evaluated using cross-validation. When all the instances in the configuration space have been evaluated, the optimal hyper-parameter combination in the defined search space with its performance score will be returned.

'RandomizedSearchCV' is also provided in sklearn to implement a RS method. It evaluates a pre-defined number of randomly-selected hyper-parameter values in parallel. Cross-validation is conducted to effectively evaluate the performance of each configuration.

6.2. *Spearmint*

Spearmint [87] is a library using Bayesian optimization with the Gaussian process as the surrogate model. Spearmint's primary deficiency is that it is not very efficient for categorical and conditional hyper-parameters.

6.3. *BayesOpt*

Bayesian Optimization (BayesOpt) [109] is a Python library employed to solve HPO problems using BO. BayesOpt uses a Gaussian process as its surrogate model to calculate the objective function based on past evaluations and utilizes an acquisition function to determine the next values.

6.4. *Hyperopt*

Hyperopt [110] is a HPO framework that involves RS and BO-TPE as the optimization algorithms. Unlike some of the other libraries that only support a single model, Hyperopt is able to use multiple models to model hierarchical hyper-parameters. In addition, Hyperopt is parallelizable since it uses MongoDB as the central database to store the hyper-parameter combinations. Hyperopt-sklearn [111] and hyperas [112] are the two libraries that can apply Hyperopt to scikit-learn and Keras libraries.

6.5. SMAC

SMAC [90][113] is another library that uses BO with random forest as the surrogate model. It supports categorical, continuous, and discrete variables.

6.6. BOHB

BOHB framework [97] is a combination of Bayesian optimization and Hyperband [15]. It overcomes one limitation of Hyperband, in that it randomly generates the test configurations, by replacing this procedure by BO. TPE is used as the surrogate model to store and model function evaluations. Using BOHB to evaluate the instance can achieve a trade-off between model performance and the current budget.

6.7. Optunity

Optunity [83] is a popular HPO framework that provides several optimization techniques, including GS, RS, PSO, and BO-TPE. In Optunity, categorical hyper-parameters are converted to discrete hyper-parameters by indexing, and discrete hyper-parameters are processed as continuous hyper-parameters by rounding them; as such, it supports all types of hyper-parameters.

6.8. Skopt

Skopt (scikit-optimize) [114] is a HPO library that is built on top of the scikit-learn [31] library. It implements several sequential model-based optimization models, including RS and BO-GP. The methods exhibit good performance with small search space and proper initialization.

6.9. GpFlowOpt

GpFlowOpt [115] is a Python library for BO using GP as the surrogate model. It supports running BO-GP on GPU using the Tensorflow library. Therefore, GpFlowOpt is a good choice if BO is used in deep learning models with GPU resources available.

6.10. Talos

Talos [116] is a Python package designed for hyper-parameter optimization with Keras models. Talos can be fully deployed into any Keras models and implemented easily without learning any new syntax. Several optimization techniques, including GS, RS, and probabilistic reduction, can be implemented using Talos.

6.11. *Sherpa*

Sherpa [117] is a Python package used for HPO problems. It can be used with other ML libraries, including sklearn [31], Tensorflow [118], and Keras [33]. It supports parallel computations and has several optimization methods, including GS, RS, BO-GP (via GPyOpt), Hyperband, and population-based training (PBT).

6.12. *Osprey*

Osprey [119] is a Python library designed to optimize hyper-parameters. Several HPO strategies are available in Osprey, including GS, RS, BO-TPE (via Hyperopt), and BO-GP (via GPyOpt).

6.13. *FAR-HO*

FAR-HO [120] is a hyper-parameter optimization package that employs gradient-based algorithms with TensorFlow. FAR-HO contains a few gradient-based optimizers, like reverse hyper-gradient and forward hyper-gradient methods. This library is designed to build access to the gradient-based hyper-parameter optimizers in TensorFlow, allowing deep learning model training and hyper-parameter optimization in GPU or other tensor-optimized computing environments.

6.14. *Hyperband*

Hyperband [16] is a Python package for tuning hyper-parameters by Hyperband, a bandit-based approach. Similar to 'GridSearchCV' and 'RandomizedSearchCV' in scikit-learn, there is a class named 'HyperbandSearchCV' in Hyperband that can be combined with sklearn and used for HPO problems. In 'HyperbandSearchCV' method, cross-validation is used for evaluation.

6.15. *DEAP*

DEAP [121] is a novel evolutionary computation package for Python that contains several evolutionary algorithms like GA and PSO. It integrates with parallelization mechanisms like multiprocessing, and machine learning packages like sklearn.

6.16. TPOT

TPOT [122] is a Python tool for auto-ML that uses genetic programming to optimize ML pipelines. TPOT is built on top of sklearn, so it is easy to implement TPOT on ML models. 'TPOTClassifier' is its principal function, and several additional hyper-parameters of GA must be set to fit specific problems.

6.17. Nevergrad

Nevergrad [123] is an open-source Python library that includes a wide range of optimizers, like fast-GA and PSO. In ML, Nevergrad can be used to tune all types of hyper-parameters, including discrete, continuous, and categorical hyper-parameters, by choosing different optimizers.

7. Experiments

To summarize the content of Sections 3 to 6, a comprehensive overview of applying hyper-parameter optimization techniques to ML models is shown in Table 2. It provides a summary of common ML algorithms, their hyper-parameters, suitable optimization methods, and available Python libraries; thus, data analysts and researchers can look up this table and select suitable optimization algorithms as well as libraries for practical use.

To put theory into practice, several experiments have been conducted based on Table 2. This section provides the experiments of applying eight different HPO techniques to three common and representative ML algorithms on two benchmark datasets. In the first part of this section, the experimental setup and the main process of HPO are discussed. In the second part, the results of utilizing different HPO methods are compared and analyzed. The sample code of the experiments has been published in [124] to illustrate the process of applying hyper-parameter optimization to ML models.

7.1. Experimental Setup

Based on the steps to optimize hyper-parameters discussed in Section 2.2, several steps were completed before the actual optimization experiments start.

Firstly, two standard benchmarking datasets provided by the sklearn library [31], namely, the Modified National Institute of Standards and Technology dataset (MNIST) and the Boston housing dataset, are selected as

Table 2: A comprehensive overview of common ML models, their hyper-parameters, suitable optimization techniques, and available Python libraries

ML Algorithm	Main HPs	Optional HPs	HPO methods	Libraries
Linear regression	-	-	-	-
Ridge & lasso	alpha	-	BO-GP	Skpot
Logistic regression	penalty, c, solver	-	BO-TPE, SMAC	Hyperopt, SMAC
KNN	n_neighbors	weights, p, algorithm	BOs, Hyperband	Skpot, Hyperopt, SMAC, Hyperband
SVM	C, kernel, epsilon (for SVR)	gamma, coef0, degree	BO-TPE, SMAC, BOHB	Hyperopt, SMAC, BOHB
NB	alpha	-	BO-GP	Skpot
DT	criterion, max_depth, min_samples_split, min_samples_leaf, max_features	splitter, min_weight_fraction_leaf, max_leaf_nodes	GA, PSO, BO-TPE, SMAC, BOHB	TPOT, Optunity, SMAC, BOHB
RF & ET	n_estimators max_depth, criterion, min_samples_split, min_samples_leaf, max_features	splitter, min_weight_fraction_leaf, max_leaf_nodes	GA, PSO, BO-TPE, SMAC, BOHB	TPOT, Optunity, SMAC, BOHB
XGBoost	n_estimators, max_depth, learning_rate, subsample, colsample_bytree,	min_child_weight, gamma, alpha, lambda	GA, PSO, BO-TPE, SMAC, BOHB	TPOT, Optunity, SMAC, BOHB
Voting	estimators, voting	weights	GS	sklearn
Bagging	base_estimator, n_estimators	max_samples, max_features	GS, BOs	sklearn, Skpot, Hyperopt, SMAC
AdaBoost	base_estimator, n_estimators, learning_rate	-	BO-TPE, SMAC	Hyperopt, SMAC
Deep learning	number of hidden layers, 'units' per layer, loss, optimizer, Activation, learning_rate, dropout rate, epochs, batch_size, early stop patience	number of frozen layers (if transfer learning is used)	PSO, BOHB	Optunity, BOHB
K-means	n_clusters	init, n_init, max_iter	BOs, Hyperband	Skpot, Hyperopt, SMAC, Hyperband
Hierarchical clustering	n_clusters, distance_threshold	linkage	BOs, Hyperband	Skpot, Hyperopt, SMAC, Hyperband
DBSCAN	eps, min_samples	-	BO-TPE, SMAC, BOHB	Hyperopt, SMAC, BOHB
Gaussian mixture	n_components	covariance_type, max_iter, tol	BO-GP	Skpot
PCA	n_components	svd_solver	BOs, Hyperband	Skpot, Hyperopt, SMAC, Hyperband
LDA	n_components	solver, shrinkage	BOs, Hyperband	Skpot, Hyperopt, SMAC, Hyperband

the benchmark datasets for HPO method evaluation on data analytics problems. MNIST is a hand-written digit recognition dataset used as a multi-classification problem, while the Boston housing dataset contains information about the price of houses in various places in the city of Boston and can be used as a regression dataset to predict the housing prices.

At the next stage, the ML models with their objective function need to be configured. In Section 5.2, all common ML models are divided into five categories based on their hyper-parameter types. Among those ML categories, "one discrete hyper-parameter", "a few conditional hyper-parameters", and "a large hyper-parameter configuration space with multiple types of hyper-parameters" are the three most common cases. Thus, three ML algorithms, KNN, SVM, and RF, are selected as the target models to be optimized, since their hyper-parameter types represent the three most common HPO cases: KNN has one important hyper-parameter, the number of considered nearest neighbors for each sample; SVM has a few conditional hyper-parameters, like the kernel type and the penalty parameter C ; RF has multiple hyper-parameters of different types, as discussed in Section 3.1.5. Moreover, KNN, SVM, and RF can all be applied to solve both classification and regression problems.

In the next step, the performance metrics and evaluation methods are configured. For each experiment on the selected two datasets, 3-fold cross validation is implemented to evaluate the involved HPO methods. The two most commonly-used performance metrics are used in our experiments. For classification models, accuracy is used as the classifier performance metric, which is the proportion of correctly classified data; while for regression models, the mean squared error (MSE) is used as the regressor performance metric, which measures the average squared difference between the predicted values and the actual values. Additionally, the computational time (CT), the total time needed to complete a HPO process with 3-fold cross-validation, is also used as the model efficiency metric [55]. In each experiment, the optimized ML model architecture that has the highest accuracy or the lowest MSE and the optimal hyper-parameter configuration will be returned.

After that, to fairly compare different optimization algorithms and frameworks, certain constraints should be satisfied. Firstly, we compare different HPO methods using the same hyper-parameter configuration space. For KNN, the only hyper-parameter to be optimized, 'n_neighbors', is set to be in the same range of 1 to 20 for each optimization method evaluation. The hyper-parameters of SVM and RF models for classification and regression

Table 3: Configuration space for the hyper-parameters of tested ML models

ML Model	Hyper-parameter	Type	Search Space
RF Classifier	n_estimators	Discrete	[10,100]
	max_depth	Discrete	[5,50]
	min_samples_split	Discrete	[2,11]
	min_samples_leaf	Discrete	[1,11]
	criterion	Categorical	['gini', 'entropy']
	max_features	Discrete	[1,64]
SVM Classifier	C	Continuous	[0.1,50]
	kernel	Categorical	['linear', 'poly', 'rbf', 'sigmoid']
KNN Classifier	n_neighbors	Discrete	[1,20]
RF Regressor	n_estimators	Discrete	[10,100]
	max_depth	Discrete	[5,50]
	min_samples_split	Discrete	[2,11]
	min_samples_leaf	Discrete	[1,11]
	criterion	Categorical	['mse', 'mae']
	max_features	Discrete	[1,13]
SVM Regressor	C	Continuous	[0.1,50]
	kernel	Categorical	['linear', 'poly', 'rbf', 'sigmoid']
	epsilon	Continuous	[0.001,1]
KNN Regressor	n_neighbors	Discrete	[1,20]

problems are also set to be in the same configuration space for each type of problem. The specifics of the configuration space for ML models are shown in Table 3. The selected hyper-parameters and their search space are determined based on the concepts in Section 3, domain knowledge, and manual testings [81]. The hyper-parameter types of each ML algorithm are also summarized in Table 3.

On the other hand, to fairly compare the performance metrics of optimization techniques, the maximum number of iterations for all HPO methods is set to 50 for RF and SVM model optimizations, and 10 for KNN model optimization based on manual testings and domain knowledge. Moreover, to avoid the impacts of randomness, all experiments are repeated ten times with different random seeds, and results are averaged for regression problems or given the majority vote for classification problems.

In Section 4, more than ten HPO methods are introduced. In our experiments, eight representative HPO approaches are selected for performance comparison, including GS, RS, BO-GP, BO-TPE, Hyperband, BOHB, GA, and PSO. After setting up the fair experimental environments for each HPO method, the HPO experiments are implemented based on the steps discussed in Section 2.2.

All experiments were conducted using Python 3.5 on a machine with

Table 4: Performance evaluation of applying HPO methods to the RF classifier on the MNIST dataset

Optimization Algorithm	Accuracy (%)	CT (s)
Default HPs	90.65	0.09
GS	93.32	48.62
RS	93.38	16.73
BO-GP	93.38	20.60
BO-TPE	93.88	12.58
Hyperband	93.38	8.89
BOHB	93.38	9.45
GA	93.83	19.19
PSO	93.73	12.43

6 Core i7-8700 processor and 16 gigabytes (GB) of memory. The involved ML and HPO algorithms are evaluated using multiple open-source Python libraries and frameworks introduced in Section 6, including sklearn [31], Skopt [114], Hyperopt [110], Optunity [83], Hyperband [16], BOHB [97], and TPOT [122].

7.2. Performance Comparison

The experiments of applying eight different HPO methods to ML models are summarized in Tables 4 to 9. Tables 4 to 6 provide the performance of each optimization algorithm when applied to RF, SVM, and KNN classifiers evaluated on the MNIST dataset after a complete optimization process; while Tables 7 to 9 demonstrate the performance of each HPO method when applied to RF, SVM, and KNN regressors evaluated on the Boston-housing dataset. In the first step, each ML model with its default hyper-parameter configuration is trained and evaluated as the baseline model. After that, each HPO algorithm is implemented on the ML models to evaluate and compare their accuracies for classification problems, or MSEs for regression problems, as well as their computational time (CT).

From Tables 4 to 9, we can see that using the default HP configurations do not yield the best model performance in our experiments, which emphasizes the importance of utilizing HPO methods. GS and RS can be seen as baseline models for HPO problems. From the results in Tables 4 to 9, it is shown that the computational time of GS is often much higher than other optimization methods. With the same search space size, RS is faster than GS, but both of them cannot guarantee to detect the near-optimal hyper-parameter configurations of ML models, especially for RF and SVM models

Table 5: Performance evaluation of applying HPO methods to the SVM classifier on the MNIST dataset

Optimization Algorithm	Accuracy (%)	CT (s)
Default HPs	97.05	0.29
GS	97.44	32.90
RS	97.35	12.48
BO-GP	97.50	17.56
BO-TPE	97.44	3.02
Hyperband	97.44	11.37
BOHB	97.44	8.18
GA	97.44	16.89
PSO	97.44	8.33

Table 6: Performance evaluation of applying HPO methods to the KNN classifier on the MNIST dataset

Optimization Algorithm	Accuracy (%)	CT (s)
Default HPs	96.27	0.24
GS	96.22	7.86
RS	96.33	6.44
BO-GP	96.83	1.12
BO-TPE	96.83	2.33
Hyperband	96.22	4.54
BOHB	97.44	3.84
GA	96.83	2.34
PSO	96.83	1.73

Table 7: Performance evaluation of applying HPO methods to the RF regressor on the Boston-housing dataset

Optimization Algorithm	MSE	CT (s)
Default HPs	31.26	0.08
GS	29.02	4.64
RS	27.92	3.42
BO-GP	26.79	17.94
BO-TPE	25.42	1.53
Hyperband	26.14	2.56
BOHB	25.56	1.88
GA	26.95	4.73
PSO	25.69	3.20

Table 8: Performance evaluation of applying HPO methods to the SVM regressor on the Boston-housing dataset

Optimization Algorithm	MSE	CT (s)
Default HPs	77.43	0.02
GS	67.07	1.33
RS	61.40	0.48
BO-GP	61.27	5.87
BO-TPE	59.40	0.33
Hyperband	73.44	0.32
BOHB	59.67	0.31
GA	60.17	1.12
PSO	58.72	0.53

Table 9: Performance evaluation of applying HPO methods to the KNN regressor on the Boston-housing dataset

Optimization Algorithm	MSE	CT (s)
Default HPs	81.48	0.004
GS	81.53	0.12
RS	80.77	0.11
BO-GP	80.77	0.49
BO-TPE	80.83	0.08
Hyperband	80.87	0.10
BOHB	80.77	0.09
GA	80.77	0.33
PSO	80.74	0.19

which have a larger search space than KNN.

The performance of BO and multi-fidelity models is much better than GS and RS. The computation time of BO-GP is often higher than other HPO methods due to its cubic time complexity, but it can obtain better performance metrics for ML models with small-size continuous hyper-parameter space, like KNN. Conversely, hyperband is often not able to obtain the highest accuracy or the lowest MSE among the optimization methods, but their computational time is low because it works on the small-sized subsets. The performance of BO-TPE and BOHB is often better than others, since they can detect the optimal or near-optimal hyper-parameter configurations within a short computational time.

For metaheuristics methods, GA and PSO, their accuracies are often higher than other HPO methods for classification problems, and their MSEs are often lower than other optimization techniques. However, their computational time is often higher than BO-TPE and multi-fidelity models, especially for GA, which does not support parallel executions.

To summarize, it is simple to implement GS and RS, but they often cannot detect the optimal hyper-parameter configurations or cost much computational time. BO-GP and GA also cost more computational time than many other HPO methods, but BO-GP works well on small configuration space, while GA is effective for large configuration space. Hyperband’s computational time is low, but it cannot guarantee to detect the global optimum. For ML models with large configuration space, BO-TPE, BOHB, and PSO often work well.

8. Open Issues, Challenges, and Future Research Directions

Although there have been many existing HPO algorithms and practical frameworks, some issues still need to be addressed, and several aspects in this domain could be improved. In this section, we discuss the open challenges, current research questions, and potential research directions in the future. They can be classified as model complexity challenges and model performance challenges, as summarized in Table [10](#).

8.1. Model Complexity

8.1.1. Costly Objective Function Evaluations

To evaluate the performance of a ML model with different hyper-parameter configurations, its objective function must be minimized in each evaluation.

Table 10: The open challenges and future directions of HPO research

Category	Challenges & Future Requirements	Brief Description
Model complexity	Costly objective function evaluations	HPO methods should reduce evaluation time on large datasets.
	Complex search space	HPO methods should reduce execution time on high dimensionalities (large hyper-parameter search space).
Model performance	Strong anytime performance	HPO methods should be able to detect the optimal or near-optimal HPs even with a very limited budget.
	Strong final performance	HPO methods should be able to detect the global optimum when given a sufficient budget.
	Comparability	There should exist a standard set of benchmarks to fairly evaluate and compare different optimization algorithms.
	Over-fitting and generalization	The optimal HPs detected by HPO methods should have generalizability to build efficient models on unseen data.
	Randomness	HPO methods should reduce randomness on the obtained results.
	Scalability	HPO methods should be scalable to multiple libraries or platforms (<i>e.g.</i> , distributed ML platforms).
	Continuous updating capability	HPO methods should consider their capacity to detect and update optimal HP combinations on continuously-updated data.

Depending on the scale of data, the model complexity, and available computational resources, the evaluation of each hyper-parameter configuration may cost several minutes, hours, days, or even more [93]. Additionally, the values of certain hyper-parameters have a direct impact on the execution time, like the number of considered neighbors in KNN, the number of basic decision trees in RF, and the number of hidden layers in deep neural networks [125].

To solve this problem by HPO algorithms, BO models reduce the total number of evaluations by spending time choosing the next evaluating point instead of simply evaluating all possible hyper-parameter configurations; however, they still require much execution time due to their poor capacity for parallelization. On the other hand, although multi-fidelity optimization methods, like Hyperband, have had some success dealing with HPO problems with limited budgets, there are still some problems that cannot be effectively solved by HPO due to the complexity of models or the scale of datasets [6]. For example, the ImageNet [126] challenge is a very popular problem in the image processing domain, but there has not been any re-

search or work on efficiently optimizing hyper-parameters for the ImageNet challenge yet, due to its huge scale and the complexity of CNN models used on ImageNet.

8.1.2. Complex Search Space

In many problems to which ML algorithms are applied, only a few hyper-parameters have significant effects on model performance, and they are the main hyper-parameters that require tuning. However, certain other unimportant hyper-parameters may still affect the performance slightly and may be considered to optimize the ML model further, which increases the dimensionality of hyper-parameter search space. As the number of hyper-parameters and configurations increase, they exponentially increase the dimensionality of the search space and the complexity of the problems, and the total objective function evaluation time will also increase exponentially [7]. Therefore, it is necessary to reduce the influence of large search spaces on execution time by improving existing HPO methods.

8.2. Model Performance

8.2.1. Strong Anytime Performance and Final Performance

HPO techniques are often expensive and sometimes require extreme resources, especially for massive datasets or complex ML models. One example of a resource-intensive model is deep learning models, since they view objective function evaluations as black-box functions and do not consider their complexity. However, the overall budget is often very limited for most practical situations, so HPO algorithms should be able to prioritize objective function evaluations and have a strong anytime performance, which indicates the capacity to detect optimal or near-optimal configurations even with a very limited budget [97]. For instance, an efficient HPO method should have a high convergence speed so that there would not be a huge difference between the results before and after model convergence, and should avoid random results even if time and resources are limited, like RS methods cannot.

On the other hand, if conditions permit and an adequate budget is given, HPO approaches should be able to identify the global optimal hyper-parameter configuration, named a strong final performance [97].

8.2.2. Comparability of HPO Methods

To optimize the hyper-parameters of ML models, different optimization algorithms can be applied to each ML framework. Different optimization

techniques have their own strengths and drawbacks in different cases, and currently, there is no single optimization approach that outperforms all other approaches when processing different datasets with various metrics and hyper-parameter types [3]. In this paper, we have analyzed the strengths and weaknesses of common hyper-parameter optimization techniques based on their principles and their performance in practical applications; but this topic could be extended more comprehensively.

To solve this problem, a standard set of benchmarks could be designed and agreed on by the community for a better comparison of different HPO algorithms. For example, there is a platform called COCO (Comparing Continuous Optimizers) [127] that provides benchmarks and analyzes common continuous optimizers. However, there is, to date, not any reliable platform that provides benchmarks and analysis of all common hyper-parameter optimization approaches. It would be easier for people to choose HPO algorithms in practical applications if a platform like COCO exists for HPO problems. In addition, a unified metric can also improve the comparability of different HPO algorithms, since different metrics are currently used in different practical problems [6].

On the other hand, based on the comparison of different HPO algorithms, a way to further improve HPO is to combine existing models or propose new models that contain as many benefits as possible and are more suitable for practical problems than existing singular models. For example, the BOHB method [97] has had some success dealing with HPO problems by combining Bayesian optimization and Hyperband. In addition, future research should consider both model performance and time budgets to develop HPO algorithms that suit real-world applications.

8.2.3. *Over-fitting and Generalization*

Generalization is another issue with HPO models. Since hyper-parameter evaluations are done with a finite number of evaluations in datasets, the optimal hyper-parameter values detected by HPO approaches might not be the same optimums on previously-unseen data. This is similar to over-fitting issues with ML models that occur when a model is closely fit to a finite number of known data points but is unfit to unseen data [128]. Generalization is also a common concern for multi-fidelity algorithms, like Hyperband and BOHB, since they need to extract subsets to represent the entire dataset.

One solution to reduce or avoid over-fitting is to use cross-validation to identify a stable optimum that performs best in all or most of the subsets

instead of a sharp optimum that only performs well in a singular validation set [6]. However, cross-validation increases the execution time several-fold. It would be beneficial if methods can better deal with overfitting and improve generalization in future research.

8.2.4. *Randomness*

There are stochastic components in the objective function of ML algorithms; thus, in some cases, the optimal hyper-parameter configuration might be different after each run. This randomness could be due to various procedures of certain ML models, like neural network initialization, or different sampled subsets in a bagging model [93]; or due to certain procedures of HPO algorithms, like crossover and mutation operations in GA. In addition, it is often difficult for HPO methods to identify the global optimum, due to the fact that HPO problems are mainly NP-hard problems. Many existing HPO algorithms can only collect several different near-optimal values, which is caused by randomness. Thus, the existing HPO models can be further improved to reduce the impact of randomness. One possible solution is to run a HPO method multiple times and select the hyper-parameter value that occurs most as the final optimum.

8.2.5. *Scalability*

In practice, one main limitation of many existing HPO frameworks is that they are tightly integrated with one or a couple of machine learning libraries, like sklearn and Keras, which restricts them to only work with a single node instead of large data volumes [3]. To tackle large datasets, some distributed machine learning platforms, like Apache SystemML [129] and Spark MLlib [130], have been developed; however, only very few HPO frameworks exist that support distributed ML. Therefore, more research efforts and scalable HPO frameworks, like the ones supporting distributed ML platforms, should be developed to support more libraries.

On the other hand, future practical HPO algorithms should have the scalability to efficiently optimize hyper-parameters from a small size to a large size, irrespective of whether they are continuous, discrete, categorical, or conditional hyper-parameters.

8.2.6. *Continuous Updating Capability*

In practice, many datasets are not stationary and are constantly updated by adding new data and deleting old data. Correspondingly, the optimal

hyper-parameter values or combinations may also change with the changes in data. Currently, developing HPO methods with the capacity to continuously tune hyper-parameter values as the data changes has not drawn much attention, since researchers and data analysts often do not alter the ML model after achieving a currently optimal performance [3]. However, since their optimal hyper-parameter values would change as data changes, proper approaches should be proposed to achieve continuous updating capability.

9. Conclusion

Machine learning has become the primary strategy for tackling data-related problems and has been widely used in various applications. To apply ML models to practical problems, their hyper-parameters need to be tuned to fit specific datasets. However, since the scale of produced data is greatly increased in real-life, and manually tuning hyper-parameters is extremely computationally expensive, it has become crucial to optimize hyper-parameters by an automatic process. In this survey paper, we have comprehensively discussed the state-of-the-art research into the domain of hyper-parameter optimization as well as how to apply them to different ML models by theory and practical experiments. To apply optimization methods to ML models, the hyper-parameter types in a ML model is the main concern for HPO method selection. To summarize, BOHB is the recommended choice for optimizing a ML model, if randomly selected subsets are highly-representative of the given dataset, since it can efficiently optimize all types of hyper-parameters; otherwise, BO models are recommended for small hyper-parameter configuration space, while PSO is usually the best choice for large configuration space. Moreover, some existing useful HPO tools and frameworks, open challenges, and potential research directions are also provided and highlighted for practical use and future research purposes. We hope that our survey paper serves as a useful resource for ML users, developers, data analysts, and researchers to use and tune ML models utilizing proper HPO techniques and frameworks. We also hope that it helps to enhance understanding of the challenges that still exist within the HPO domain, and thereby further advancing HPO and ML applications in future research.

References

- [1] M.I. Jordan, T.M. Mitchell, Machine learning: Trends, perspectives, and prospects, *Science* 349 (2015) 255–260.

<https://doi.org/10.1126/science.aaa8415>.

- [2] M.-A. Zöller and M. F. Huber, Benchmark and Survey of Automated Machine Learning Frameworks, arXiv preprint [arXiv:1904.12054](https://arxiv.org/abs/1904.12054), (2019). <https://arxiv.org/abs/1904.12054>.
- [3] R. E. Shawi, M. Maher, S. Sakr, Automated machine learning: State-of-the-art and open challenges, arXiv preprint [arXiv:1906.02287](https://arxiv.org/abs/1906.02287), (2019). <http://arxiv.org/abs/1906.02287>.
- [4] M. Kuhn and K. Johnson, Applied Predictive Modeling, Springer (2013) ISBN: 9781461468493.
- [5] G.I. Diaz, A. Fokoue-Nkoutche, G. Nannicini, H. Samulowitz, An effective algorithm for hyperparameter optimization of neural networks, IBM J. Res. Dev. 61 (2017) 1–20. <https://doi.org/10.1147/JRD.2017.2709578>.
- [6] F. Hutter, L. Kotthoff, and J. Vanschoren, Eds., Automatic Machine Learning: Methods, Systems, Challenges, Springer (2019) ISBN: 9783030053185.
- [7] N. Decastro-García, Á. L. Muñoz Castañeda, D. Escudero García, and M. V. Carriegos, Effect of the Sampling of a Dataset in the Hyperparameter Optimization Phase over the Efficiency of a Machine Learning Algorithm, Complexity 2019 (2019). <https://doi.org/10.1155/2019/6278908>.
- [8] S. Abreu, Automated Architecture Design for Deep Neural Networks, arXiv preprint [arXiv:1908.10714](https://arxiv.org/abs/1908.10714), (2019). <http://arxiv.org/abs/1908.10714>.
- [9] O. S. Steinholtz, A Comparative Study of Black-box Optimization Algorithms for Tuning of Hyper-parameters in Deep Neural Networks, M.S. thesis, Dept. Elect. Eng., Luleå Univ. Technol., (2018).
- [10] G. Luo, A review of automatic selection methods for machine learning algorithms and hyper-parameter values, Netw. Model. Anal. Heal. Informatics Bioinforma. 5 (2016) 1–16. <https://doi.org/10.1007/s13721-016-0125-6>.

- [11] D. Maclaurin, D. Duvenaud, R.P. Adams, Gradient-based Hyperparameter Optimization through Reversible Learning, arXiv preprint [arXiv:1502.03492](https://arxiv.org/abs/1502.03492), (2015). [http://arxiv.org/abs/1502.03492](https://arxiv.org/abs/1502.03492).
- [12] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, Algorithms for hyperparameter optimization, Proc. Adv. Neural Inf. Process. Syst., (2011) 2546–2554.
- [13] B. James and B. Yoshua, Random Search for Hyper-Parameter Optimization, J. Mach. Learn. Res. 13 (1) (2012) 281–305.
- [14] K. Eggenberger, M. Feurer, F. Hutter, J. Bergstra, J. Snoek, H. Hoos, K. Leyton-Brown, Towards an Empirical Foundation for Assessing Bayesian Optimization of Hyperparameters, BayesOpt Work. (2013) 1–5.
- [15] K. Eggenberger, F. Hutter, H.H. Hoos, K. Leyton-Brown, Efficient benchmarking of hyperparameter optimizers via surrogates, Proc. Natl. Conf. Artif. Intell. 2 (2015) 1114–1120.
- [16] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, Hyperband: A novel bandit-based approach to hyperparameter optimization, J. Mach. Learn. Res. 18 (2012) 1–52.
- [17] Q. Yao *et al.*, Taking Human out of Learning Applications: A Survey on Automated Machine Learning, arXiv preprint [arXiv:1810.13306](https://arxiv.org/abs/1810.13306), (2018). [http://arxiv.org/abs/1810.13306](https://arxiv.org/abs/1810.13306).
- [18] S. Lessmann, R. Stahlbock, S.F. Crone, Optimizing hyperparameters of support vector machines by genetic algorithms, Proc. 2005 Int. Conf. Artif. Intell. ICAI’05. 1 (2005) 74–80.
- [19] P. R. Lorenzo, J. Nalepa, M. Kawulok, L. S. Ramos, and J. R. Paster, Particle swarm optimization for hyper-parameter selection in deep neural networks, Proc. ACM Int. Conf. Genet. Evol. Comput., (2017) 481–488.
- [20] S. Sun, Z. Cao, H. Zhu, J. Zhao, A Survey of Optimization Methods from a Machine Learning Perspective, arXiv preprint [arXiv:1906.06821](https://arxiv.org/abs/1906.06821), (2019). <https://arxiv.org/abs/1906.06821>.

- [21] T.M. S. Bradley, A. Hax, Applied Mathematical Programming, Addison-Wesley, Reading, Massachusetts. (1977).
- [22] S. Bubeck, Convex optimization: Algorithms and complexity, Found. Trends Mach. Learn. 8 (2015) 231–357. <https://doi.org/10.1561/22000000050>.
- [23] B. Shahriari, A. Bouchard-Côté, and N. de Freitas, “Unbounded Bayesian optimization via regularization,” Proc. Artif. Intell. Statist., (2016) 1168–1176.
- [24] G.I. Diaz, A. Fokoue-Nkoutche, G. Nannicini, H. Samulowitz, An effective algorithm for hyperparameter optimization of neural networks, IBM J. Res. Dev. 61 (2017) 1–20. <https://doi.org/10.1147/JRD.2017.2709578>.
- [25] C. Gambella, B. Ghaddar, and J. Naoum-Sawaya, Optimization Models for Machine Learning: A Survey, arXiv preprint [arXiv:1901.05331](https://arxiv.org/abs/1901.05331), (2019). <http://arxiv.org/abs/1901.05331>.
- [26] E. R. Sparks, A. Talwalkar, D. Haas, M. J. Franklin, M. I. Jordan, and T. Kraska, Automating model search for large scale machine learning, Proc. 6th ACM Symp. Cloud Comput., (2015) 368–380.
- [27] J. Nocedal and S. Wright, Numerical Optimization, (2006) Springer-Verlag, ISBN: 978-0-387-40065-5.
- [28] A. Moubayed, M. Injadat, A. Shami, H. Lutfiyya, DNS Typo-Squatting Domain Detection: A Data Analytics & Machine Learning Based Approach, 2018 IEEE Glob. Commun. Conf. GLOBECOM 2018 - Proc. (2018). <https://doi.org/10.1109/GLOCOM.2018.8647679>.
- [29] R. Caruana, A. Niculescu-Mizil, An empirical comparison of supervised learning algorithms, ACM Int. Conf. Proceeding Ser. 148 (2006) 161–168. <https://doi.org/10.1145/1143844.1143865>.
- [30] O. Kramer, Scikit-Learn, in Machine Learning for Evolution Strategies. Cham, Switzerland: Springer International Publishing, (2016) 45–53.
- [31] F. Pedregosa *et al.*, Scikit-learn: Machine learning in Python, J. Mach. Learn. Res., 12 (2011) 2825–2830.

- [32] T.Chen, C.Guestrin, XGBoost: a scalable tree boosting system, arXiv preprint [arXiv:1603.02754](https://arxiv.org/abs/1603.02754), (2016). [http://arxiv.org/abs/1603.02754](https://arxiv.org/abs/1603.02754).
- [33] F. Chollet, Keras, 2015. <https://github.com/fchollet/keras>.
- [34] C. Gambella, B. Ghaddar, J. Naoum-Sawaya, Optimization Models for Machine Learning: A Survey, (2019) 1–40. <http://arxiv.org/abs/1901.05331>
- [35] C.M. Bishop, Pattern Recognition and Machine Learning. (2006) Springer, ISBN: 978-0-387-31073-2.
- [36] A.E. Hoerl, R.W. Kennard, Ridge Regression: Applications to Nonorthogonal Problems, *Technometrics*. 12 (1970) 69–82. <https://doi.org/10.1080/00401706.1970.10488635>.
- [37] L.E. Melkumova, S.Y. Shatskikh, Comparing Ridge and LASSO estimators for data analysis, *Procedia Eng.* 201 (2017) 746–755. <https://doi.org/10.1016/j.proeng.2017.09.615>.
- [38] R. Tibshirani, Regression Shrinkage and Selection Via the Lasso, *J. R. Stat. Soc. Ser. B.* 58 (1996) 267–288. <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>.
- [39] D.W. Hosmer Jr, S. Lemeshow, Applied logistic regression, *Technometrics*, 34 (1) (2013), 358–359.
- [40] J.O. Ogutu, T. Schulz-Streeck, H.P. Piepho, Genomic selection using regularized linear regression models: ridge regression, lasso, elastic net and their extensions, *BMC Proceedings. BioMed Cent.* 6 (2012).
- [41] J.M. Keller, M.R. Gray, A Fuzzy K-Nearest Neighbor Algorithm, *IEEE Trans. Syst. Man Cybern.* SMC-15 (1985) 580–585. <https://doi.org/10.1109/TSMC.1985.6313426>.
- [42] W. Zuo, D. Zhang, K. Wang, On kernel difference-weighted k-nearest neighbor classification, *Pattern Anal. Appl.* 11 (2008) 247–257. <https://doi.org/10.1007/s10044-007-0100-z>.
- [43] A. Smola, V. Vapnik, Support vector regression machines, *Adv. Neural Inf. Process. Syst.* 9 (1997) 155–161.

- [44] L. Yang, R. Muresan, A. Al-Dweik, L.J. Hadjileontiadis, Image-Based Visibility Estimation Algorithm for Intelligent Transportation Systems, *IEEE Access*. 6 (2018) 76728–76740. <https://doi.org/10.1109/ACCESS.2018.2884225>.
- [45] L. Yang, Comprehensive Visibility Indicator Algorithm for Adaptable Speed Limit Control in Intelligent Transportation Systems, M.A.Sc. thesis, University of Guelph, 2018.
- [46] O.S. Soliman, A.S. Mahmoud, A classification system for remote sensing satellite images using support vector machine with non-linear kernel functions, 2012 8th Int. Conf. Informatics Syst. INFOS 2012. (2012) BIO-181-BIO-187.
- [47] I. Rish, An empirical study of the naive Bayes classifier, *IJCAI 2001 Work. Empir. methods Artif. Intell.*, (2001), 41-46.
- [48] J.N. Sulzmann, J. Fürnkranz, E. Hüllermeier, On pairwise naive bayes classifiers, *Lect. Notes Comput. Sci. (Including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)*. 4701 LNAI (2007) 371–381. https://doi.org/10.1007/978-3-540-74958-5_35.
- [49] C. Bustamante, L. Garrido, R. Soto, Comparing fuzzy Naive Bayes and Gaussian Naive Bayes for decision making in RoboCup 3D, *Lect. Notes Comput. Sci. (Including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)*. 4293 LNAI (2006) 237–247. https://doi.org/10.1007/11925231_23.
- [50] A.M. Kibriya, E. Frank, B. Pfahringer, G. Holmes, Multinomial naive bayes for text categorization revisited, *Lect. Notes Artif. Intell. (Sub-series Lect. Notes Comput. Sci)*. 3339 (2004) 488–499.
- [51] J.D.M. Rennie, L. Shih, J. Teevan, D.R. Karger Tackling the poor assumptions of Naive Bayes text classifiers, *Proc. Twent. Int. Conf. Mach. Learn. ICML* (2003), 616-623.
- [52] V. Narayanan, I. Arora, and A. Bhatia, Fast and accurate sentiment classification using an enhanced naïve Bayes model, *arXiv preprint [arXiv:1305.6143](https://arxiv.org/abs/1305.6143)*, (2013). <https://arxiv.org/abs/1305.6143>.

- [53] S. Rasoul, L. David, A Survey of Decision Tree Classifier Methodology, *IEEE Trans. Syst. Man. Cybern.* 21 (1991) 660–674.
- [54] D.M. Manias, M. Jammal, H. Hawilo, A. Shami, P. Heidari, A. Larabi, R. Brunner, Machine Learning for Performance-aware Virtual Network Function Placement, 2019 IEEE Glob. Commun. Conf. GLOBECOM 2019 - Proc. (2019) 12–17. <https://doi.org/10.1109/GLOBECOM38437.2019.9013246>.
- [55] L. Yang, A. Moubayed, I. Hamieh, A. Shami, Tree-based intelligent intrusion detection system in internet of vehicles, 2019 IEEE Glob. Commun. Conf. GLOBECOM 2019 - Proc. (2019). <https://doi.org/10.1109/GLOBECOM38437.2019.9013892>.
- [56] S. Sanders, C. Giraud-Carrier, Informing the use of hyper-parameter optimization through metalearning, *Proc. - IEEE Int. Conf. Data Mining, ICDM.* 2017-Novem (2017) 1051–1056. <https://doi.org/10.1109/ICDM.2017.137>.
- [57] M. Injadat, F. Salo, A.B. Nassif, A. Essex, A. Shami, Bayesian Optimization with Machine Learning Algorithms Towards Anomaly Detection, 2018 IEEE Glob. Commun. Conf. (2018) 1–6. <https://doi.org/10.1109/glocom.2018.8647714>.
- [58] F. Salo, M.N. Injadat, A. Moubayed, A.B. Nassif, A. Essex, Clustering Enabled Classification using Ensemble Feature Selection for Intrusion Detection, 2019 Int. Conf. Comput. Netw. Commun. ICNC 2019. (2019) 276–281. <https://doi.org/10.1109/ICCNC.2019.8685636>.
- [59] K. Arjunan, C.N. Modi, An enhanced intrusion detection framework for securing network layer of cloud computing, *ISEA Asia Secur. Priv. Conf. 2017, ISEASP 2017.* (2017) 1–10. <https://doi.org/10.1109/ISEASP.2017.7976988>.
- [60] Y. Xia, C. Liu, Y.Y. Li, N. Liu, A boosted decision tree approach using Bayesian hyper-parameter optimization for credit scoring, *Expert Syst. Appl.* 78 (2017) 225–241. <https://doi.org/10.1016/j.eswa.2017.02.017>.
- [61] T. G. Dietterich, Ensemble methods in machine learning, *Mult. Classif. Syst.*, 1857 (2000), 1–15.

- [62] A. Moubayed, E. Aqeeli, A. Shami, Ensemble-based Feature Selection and Classification Model for DNS Typo-squatting Detection, in: 2020 IEEE Can. Conf. Electr. Comput. Eng., 2020.
- [63] W. Yin, K. Kann, M. Yu, and H. Schütze, Comparative Study of CNN and RNN for Natural Language Processing, arXiv preprint [arXiv:1702.01923](https://arxiv.org/abs/1702.01923), (2017). <https://arxiv.org/abs/1702.01923>
- [64] A. Koutsoukas, K.J. Monaghan, X. Li, J. Huan, Deep-learning: Investigating deep neural networks hyper-parameters and comparison of performance to shallow methods for modeling bioactivity data, J. Chem-inform. 9 (2017) 1–13. <https://doi.org/10.1186/s13321-017-0226-y>.
- [65] T. Domhan, J.T. Springenberg, F. Hutter, Speeding up automatic hyperparameter optimization of deep neural networks by extrapolation of learning curves, IJCAI Int. Jt. Conf. Artif. Intell. 2015-January (2015) 3460–3468.
- [66] Y. Ozaki, M. Yano, M. Onishi, Effective hyperparameter optimization using Nelder-Mead method in deep learning, IPSJ Trans. Comput. Vis. Appl. 9 (2017). <https://doi.org/10.1186/s41074-017-0030-7>.
- [67] F.C. Soon, H.Y. Khaw, J.H. Chuah, J. Kanesan, Hyper-parameters optimisation of deep CNN architecture for vehicle logo recognition, IET Intell. Transp. Syst. 12 (2018) 939–946. <https://doi.org/10.1049/iet-its.2018.5127>.
- [68] D. Han, Q. Liu, W. Fan, A new image classification method using CNN transfer learning and web data augmentation, Expert Syst. Appl. 95 (2018) 43–56. <https://doi.org/10.1016/j.eswa.2017.11.028>.
- [69] C. Di Francescomarino, M. Dumas, M. Federici, C. Ghidini, F.M. Maggi, W. Rizzi, L. Simonetto, Genetic algorithms for hyperparameter optimization in predictive business process monitoring, Inf. Syst. 74 (2018) 67–83. <https://doi.org/10.1016/j.is.2018.01.003>.
- [70] A. Moubayed, M. Injadat, A. Shami, H. Lutfiyya, Student Engagement Level in e-Learning Environment: Clustering Using K-means, Am. J. Distance Educ. 34 (2020) 1–20. <https://doi.org/10.1080/08923647.2020.1696140>.

- [71] T. K. Moon, The expectation-maximization algorithm, *IEEE Signal Process. Mag.* 13 (6) (1996) 47–60.
- [72] S. Brahim-Belhouari, A. Bermak, M. Shi, P.C.H. Chan, Fast and Robust gas identification system using an integrated gas sensor technology and Gaussian mixture models, *IEEE Sens. J.* 5 (2005) 1433–1444. <https://doi.org/10.1109/JSEN.2005.858926>.
- [73] Z. Y., K. G., Hierarchical Clustering Algorithms for Document Dataset, *Data Min. Knowl. Discov.* 10 (2005) 141–168.
- [74] K. Khan, S.U. Rehman, K. Aziz, S. Fong, S. Sarasvady, A. Vishwa, DBSCAN: Past, present and future, 5th Int. Conf. Appl. Digit. Inf. Web Technol. ICADIWT 2014. (2014) 232–238. <https://doi.org/10.1109/ICADIWT.2014.6814687>.
- [75] H. Zhou, P. Wang, H. Li, Research on adaptive parameters determination in DBSCAN algorithm, *J. Inf. Comput. Sci.* 9 (2012) 1967–1973.
- [76] J. Shlens, A Tutorial on Principal Component Analysis, arXiv preprint [arXiv:1404.1100](https://arxiv.org/abs/1404.1100), (2014). <https://arxiv.org/abs/1404.1100>
- [77] N. Halko, P. Martinsson, J. Tropp, Finding structure with randomness: probabilistic algorithms for constructing approximate matrix decompositions, *SIAM Rev.* 53 (2) (2011), pp. 217–288
- [78] M. Loog, Conditional linear discriminant analysis, *Proc. - Int. Conf. Pattern Recognit.* 2 (2006) 387–390. <https://doi.org/10.1109/ICPR.2006.402>.
- [79] P. Howland, J. Wang, H. Park, Solving the small sample size problem in face recognition using generalized discriminant analysis, *Pattern Recognit.* 39 (2006) 277–287. <https://doi.org/10.1016/j.patcog.2005.06.013>.
- [80] I. Ilievski, T. Akhtar, J. Feng, C.A. Shoemaker, Efficient hyperparameter optimization of deep learning algorithms using deterministic RBF surrogates, 31st AAAI Conf. Artif. Intell. AAAI 2017. (2017) 822–829.
- [81] M.N. Injadat, A. Moubayed, A.B. Nassif, A. Shami, Systematic Ensemble Model Selection Approach for Educational Data Mining, *Knowledge-Based Syst.* 200 (2020) 105992. <https://doi.org/10.1016/j.knosys.2020.105992>.

- [82] M. Injadat, A. Moubayed, A.B. Nassif, A. Shami, Multi-split Optimized Bagging Ensemble Model Selection for Multi-class Educational Data Mining, Springer’s Appl. Intell. (2020).
- [83] M. Claesen, J. Simm, D. Popovic, Y. Moreau, and B. De Moor, Easy Hyperparameter Search Using Optunity, arXiv preprint [arXiv:1412.1114](https://arxiv.org/abs/1412.1114), (2014). <https://arxiv.org/abs/1412.1114>.
- [84] C. Witt, Worst-case and average-case approximations by simple randomized search heuristics, in: Proceedings of the 22nd Annual Symposium on Theoretical Aspects of Computer Science, STACS’05, Stuttgart, Germany, 2005, pp. 44–56.
- [85] Y. Bengio, Gradient-based optimization of hyperparameters, Neural Comput. 12 (8) (2000) 1889-1900.
- [86] H. H. Yang and S. I. Amari, Complexity Issues in Natural Gradient Descent Method for Training Multilayer Perceptrons, Neural Comput. 10 (8) (1998) 2137–2157.
- [87] J. Snoek, H. Larochelle, R. Adams Practical Bayesian optimization of machine learning algorithms Adv. Neural Inf. Process. Syst. 4 (2012), 2951-2959.
- [88] E. Hazan, A. Klivans, and Y. Yuan, Hyperparameter optimization: a spectral approach, arXiv preprint [arXiv:1706.00764](https://arxiv.org/abs/1706.00764), (2017). <https://arxiv.org/abs/1706.00764>.
- [89] M. Seeger, Gaussian processes for machine learning, Int. J. Neural Syst., 14 (2004), 69-106.
- [90] F. Hutter, H. H. Hoos, and K. Leyton-Brown, Sequential model-based optimization for general algorithm configuration, Proc. LION 5, (2011) 507-523.
- [91] I. Dewancker, M. McCourt, S. Clark, Bayesian Optimization Primer, (2015) URL: [https://sigopt.com/static/pdf/SigOpt Bayesian Optimization Primer.pdf](https://sigopt.com/static/pdf/SigOpt%20Bayesian%20Optimization%20Primer.pdf)
- [92] J. Hensman, N. Fusi, and N. D. Lawrence, Gaussian processes for big data, arXiv preprint [arXiv:1309.6835](https://arxiv.org/abs/1309.6835), (2013). <https://arxiv.org/abs/1309.6835>.

- [93] M. Claesen and B. De Moor, Hyperparameter Search in Machine Learning, arXiv preprint [arXiv:1502.02127](https://arxiv.org/abs/1502.02127), (2015). <https://arxiv.org/abs/1502.02127>.
- [94] L. Bottou, Large-scale machine learning with stochastic gradient descent, Proceedings of the COMPSTAT, Springer (2010) 177-186.
- [95] S. Zhang, J. Xu, E. Huang, C.H. Chen, A new optimal sampling rule for multi-fidelity optimization via ordinal transformation, IEEE Int. Conf. Autom. Sci. Eng. 2016-Novem (2016) 670–674. <https://doi.org/10.1109/COASE.2016.7743467>.
- [96] Z. Karnin, T. Koren, O. Somekh, Almost optimal exploration in multi-armed bandits, 30th Int. Conf. Mach. Learn. ICML 2013. 28 (2013) 2275–2283.
- [97] S. Falkner, A. Klein, F. Hutter, BOHB: Robust and Efficient Hyperparameter Optimization at Scale, 35th Int. Conf. Mach. Learn. ICML 2018. 4 (2018) 2323–2341.
- [98] A. Gogna, A. Tayal, Metaheuristics: Review and application, J. Exp. Theor. Artif. Intell. 25 (2013) 503–526. <https://doi.org/10.1080/0952813X.2013.782347>.
- [99] F. Itano, M.A. De Abreu De Sousa, E. Del-Moral-Hernandez, Extending MLP ANN hyper-parameters Optimization by using Genetic Algorithm, Proc. Int. Jt. Conf. Neural Networks. 2018-July (2018) 1–8. <https://doi.org/10.1109/IJCNN.2018.8489520>.
- [100] B. Kazimipour, X. Li, A.K. Qin, A Review of Population Initialization Techniques for Evolutionary Algorithms, 2014 IEEE Congr. Evol. Comput. (2014) 2585–2592. <https://doi.org/10.1109/CEC.2014.6900618>.
- [101] S. Rahnamayan, H.R. Tizhoosh, M.M.A. Salama, A novel population initialization method for accelerating evolutionary algorithms, Comput. Math. with Appl. 53 (2007) 1605–1614. <https://doi.org/10.1016/j.camwa.2006.07.013>.
- [102] F. G. Lobo, D. E. Goldberg, and M. Pelikan, Time complexity of genetic algorithms on exponentially scaled problems, Proc. Genet. Evol. Comput. Conf., (2000) 151-158.

- [103] Y. Shi, R.C. Eberhart, Parameter Selection in Particle Swarm Optimization, *Evolutionary Programming VII*, Springer (1998) 591-600.
- [104] X. Yan, F. He, Y. Chen, A Novel Hardware / Software Partitioning Method Based on Position Disturbed Particle Swarm Optimization with Invasive Weed Optimization, 32 (2017) 340–355. <https://doi.org/10.1007/s11390-017-1714-2>.
- [105] M.Y. Cheng, K.Y. Huang, M. Hutomo, Multiobjective Dynamic-Guiding PSO for Optimizing Work Shift Schedules, *J. Constr. Eng. Manag.* 144 (2018) 1–7. [https://doi.org/10.1061/\(ASCE\)CO.1943-7862.0001548](https://doi.org/10.1061/(ASCE)CO.1943-7862.0001548).
- [106] H. Wang, Z. Wu, J. Wang, X. Dong, S. Yu, G. Chen, A new population initialization method based on space transformation search, *5th Int. Conf. Nat. Comput. ICNC 2009*. 5 (2009) 332–336. <https://doi.org/10.1109/ICNC.2009.371>.
- [107] J. Wang, J. Xu, and X. Wang, Combination of Hyperband and Bayesian Optimization for Hyperparameter Optimization in Deep Learning, *arXiv preprint [arXiv:1801.01596](https://arxiv.org/abs/1801.01596)*, (2018). <https://arxiv.org/abs/1801.01596>.
- [108] P. Cazzaniga, M.S. Nobile, D. Besozzi, The impact of particles initialization in PSO: Parameter estimation as a case in point, *2015 IEEE Conf. Comput. Intell. Bioinforma. Comput. Biol. CIBCB 2015*. (2015) 1–8. <https://doi.org/10.1109/CIBCB.2015.7300288>.
- [109] R. Martinez-Cantin, BayesOpt: A Bayesian optimization library for nonlinear optimization, experimental design and bandits, *J. Mach. Learn. Res.* 15 (2015) 3735–3739.
- [110] J. Bergstra, B. Komer, C. Eliasmith, D. Yamins, D.D. Cox, Hyperopt: A Python library for model selection and hyperparameter optimization, *Comput. Sci. Discov.* 8 (2015). <https://doi.org/10.1088/1749-4699/8/1/014008>.
- [111] B. Komer, J. Bergstra, and C. Eliasmith, Hyperopt-sklearn: Automatic hyperparameter configuration for scikit-learn, *Proc. ICML Workshop AutoML*, (2014) 34–40.

- [112] M. Pumperla, Hyperas, 2019. <http://maxpumperla.com/hyperas/>.
- [113] M. Lindauer, K. Eggensperger, M. Feurer, S. Falkner, A. Biedenkapp, and F. Hutter, Smac v3: Algorithm configuration in python, 2017. <https://github.com/automl/SMAC3>.
- [114] Tim Head, MechCoder, Gilles Louppe, *et al.*, scikitoptimize/scikit-optimize: v0.5.2, 2018. <https://doi.org/10.5281/zenodo.1207017>.
- [115] N. Knudde, J. van der Herten, T. Dhaene, and I. Couckuyt, GPflowOpt: A Bayesian Optimization Library using TensorFlow, arXiv preprint [arXiv:1711.03845](https://arxiv.org/abs/1711.03845), (2017). <https://arxiv.org/abs/1711.03845>.
- [116] Autonomio Talos [Computer software], 2019. <http://github.com/autonomio/talos>.
- [117] L. Hertel, P. Sadowski, J. Collado, P. Baldi, Sherpa: Hyperparameter Optimization for Machine Learning Models, Conf. Neural Inf. Process. Syst. (2018).
- [118] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, *et al.*, TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems, arXiv preprint [arXiv:1603.04467](https://arxiv.org/abs/1603.04467), (2016). <https://arxiv.org/abs/1603.04467>.
- [119] J. Grandgirard, D. Poinso, L. Krespi, J.P. Nénon, A.M. Cortesero, Osprey: Hyperparameter Optimization for Machine Learning, 103 (2002) 239–248. <https://doi.org/10.21105/joss.00034>.
- [120] L. Franceschi, M. Donini, P. Frasconi, and M. Pontil, Forward and reverse gradient-based hyperparameter optimization, 34th Int. Conf. Mach. Learn. ICML 2017, 70 (2017) 1165–1173.
- [121] F.A. Fortin, F.M. De Rainville, M.A. Gardner, M. Parizeau, C. Gagné, DEAP: Evolutionary algorithms made easy, J. Mach. Learn. Res. 13 (2012) 2171–2175.
- [122] R. S. Olson and J. H. Moore, TPOT: A tree-based pipeline optimization tool for automating machine learning, Auto Mach. Learn. (2019) 151–160. https://doi.org/10.1007/978-3-030-05318-5_8

- [123] J. Rapin and O. Teytaud, Nevergrad - A gradient-free optimization platform, 2018. <https://GitHub.com/FacebookResearch/Nevergrad>.
- [124] L. Yang and A. Shami, Hyperparameter Optimization of Machine Learning Algorithms, 2020. <https://github.com/LiYangHart/Hyperparameter-Optimization-of-Machine-Learning-Algorithms>.
- [125] C.M. Bishop, Neural Networks for Pattern Recognition, Oxford University Press (1995).
- [126] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, Adv. Neural Inf. Process. Syst. 25 (2012) 1097-1105
- [127] N. Hansen, A. Auger, O. Mersmann, T. Tutar, and D. Brockhoff, COCO: A Platform for Comparing Continuous Optimizers in a Black-Box Setting, arXiv preprint [arXiv:1603.08785](https://arxiv.org/abs/1603.08785), (2016). <https://arxiv.org/abs/1603.08785>.
- [128] G.C. Cawley, N.L.C. Talbot, On over-fitting in model selection and subsequent selection bias in performance evaluation, J. Mach. Learn. Res. 11 (2010) 2079–2107.
- [129] M. Boehm, A. Surve, S. Tatikonda, *et al.*, SystemML: declarative machine learning on spark, Proc. VLDB Endow. 9 (2016) 1425–1436. <https://doi.org/10.14778/3007263.3007279>.
- [130] X. Meng, J. Bradley, B. Yavuz, *et al.*, Mllib: machine learning in apache spark, J. Mach. Learn. Res. 17 (1) (2016) 1235-1241.



Li Yang received his Ph.D. in Electrical and Computer Engineering from Western University, London, Canada, in August 2022, his MASc. degree in Engineering from the University of Guelph, Guelph, Canada, 2018, and his B.E. degree in Computer Science from Wuhan University of Science and Technology, Wuhan, China, in 2016. Currently, he is a Postdoctoral Associate and Sessional Lecturer in the Optimized Computing and Communications (OC2) Lab at Western University. His research interests include cybersecurity, machine learning, AutoML, deep learning, network data analytics, Internet of Things (IoT), anomaly detection, online learning, concept drift, and time series data analytics.



Abdallah Shami is currently a Professor in the Electrical and Computer Engineering Department and the Acting Associate Dean (Research) of the Faculty of Engineering, Western University, London, ON, Canada, where he is also the Director of the Optimized Computing and Communications Laboratory. Dr. Shami has chaired key symposia for the IEEE GLOBECOM, IEEE International Conference on Communications, and IEEE International Conference on Computing, Networking and Communications. He was the elected Chair for the IEEE Communications Society Technical Committee on Communications Software from 2016 to 2017 and the IEEE London Ontario Section Chair from 2016 to 2018. He is currently an Associate Editor of the IEEE Transactions on Mobile Computing, IEEE Internet of Things, and IEEE Communications Surveys and Tutorials journals.

DEEP LEARNING FOR FINANCE

MODULE 1 | LESSON 4

Multilayer Perceptron: Revisiting market Timing of AAPL stock with Hyper-parameter Tuning

Reading Time 60 minutes

Prior Knowledge Deep Learning, MLP, Timing Strategy, Hyper-parameter

Keywords MLP for Regression, Hyper-parameter Tuning, Keras Tuner , Hyperband

In this lesson we revisit the timing our investments in AAPL stock under a regression problem using MLPs with Hyper-parameter tuning. This way, we will be able to observe first-hand the power of Hyper-parameter optimization (HPO). In order to complicate the predictability problem a bit over the classification example we went over in Lesson 2, and also to conveniently use the simplest form of the Keras Tuner, our predictive model MLP will be a regression one. As usual, we will first focus on the Deep Learning side of the task by building a predictive regression model with HPO, and later on check the performance of such a trading strategy in the test sample.

1. Data and sources

As we did in the example of Lesson 2, we will keep using the **yfinance** library to retrieve daily Apple stock price data from Yahoo! Finance:

```
In [1]: import numpy as np
import pandas as pd

# Read the CSV file
file_path = 'AAPL.csv'
df = pd.read_csv(file_path, delimiter=';', decimal=',', parse_dates=['Date'], da
df = df.sort_values('Date')
df = df.set_index('Date')

# Rename '% Return' column to 'Ret'
df = df.rename(columns={'% Return': 'Ret'})

# Define the name variable
name = 'Ret'

# Set 'Date' as index and keep only 'Ret' column
df = df.dropna()
```

```
df.head()
```

Out[1]:

	Price	CVol	Change	% Change	Ret	Total Return (Gross)	Cumulative Return %	Open	High	L
Date										
1991-12-02	0.46	118899320.0	0.01	1.97	1.97	0.48	276,80	0.45	0.46	C
1991-12-03	0.45	102860910.0	-0.01	-2.42	-2.42	0.47	267,70	0.46	0.46	C
1991-12-05	0.45	99198490.0	0.00	-0.99	-0.99	0.47	264,06	0.45	0.46	C
1991-12-06	0.44	196985780.0	-0.01	-2.50	-2.50	0.46	254,96	0.44	0.44	C
1991-12-09	0.44	97832080.0	0.00	0.77	0.77	0.46	257,69	0.44	0.45	C

2. Timing Apple stock with Multilayer Perceptron (MLPs)

Now, let's start building our model in a similar fashion to what we have done in previous examples, starting with inputs and output variables:

2.1 Inputs and outputs

For, among others, comparability reasons, we will consider the very same inputs for the regression model as we did for the classification example of Lesson 2. Specifically, those are AAPL stock returns from the past 25, 60, 90, 120 and 240 days:

```
In [2]: df = df[['Ret']]

df["Ret25_i"] = df[name].rolling(25).apply(lambda x: 100 * (np.prod(1 + x / 100)
df["Ret60_i"] = df[name].rolling(60).apply(lambda x: 100 * (np.prod(1 + x / 100)
df["Ret90_i"] = df[name].rolling(90).apply(lambda x: 100 * (np.prod(1 + x / 100)
df["Ret120_i"] = df[name].rolling(120).apply(lambda x: 100 * (np.prod(1 + x / 100)
df["Ret240_i"] = df[name].rolling(240).apply(lambda x: 100 * (np.prod(1 + x / 100)

df = df.dropna()
df.tail(10)
```

Out[2]:

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i
Date						
2025-04-23	2.43	-3.795072	-10.881083	-17.324816	-11.375229	13.166308
2025-04-24	1.84	-3.186662	-12.437332	-15.363482	-10.514112	14.812281
2025-04-25	0.44	-2.242569	-12.454764	-15.498093	-10.228100	15.098768
2025-04-28	0.41	-3.719238	-11.440489	-15.210988	-8.459465	14.426408
2025-04-29	0.51	-4.309509	-10.388438	-15.764123	-6.287033	15.809066
2025-04-30	0.61	-5.026928	-6.678198	-16.064459	-4.444495	14.500296
2025-05-01	0.39	-3.703195	-8.241178	-13.894452	-3.686575	14.238568
2025-05-02	-3.74	-8.267882	-11.549127	-17.690963	-7.887429	8.640630
2025-05-05	-3.15	-8.729653	-14.608582	-21.754709	-10.493604	5.155357
2025-05-06	-0.19	-10.636714	-12.675026	-22.144726	-12.535408	4.934575

- **Defining the output: Regression**

Then, we need to define our output variable. As we have already mentioned, we will face a regression problem here so our output is actually a continuous variable containing the returns of AAPL stock for some future time span.

In this case, our model will aim to predict the return of AAPL stock over the next **25 days**. Please note that this choice is completely subjective (the model may even work better for some other time span!). Once you have gone through the entire notebook and understood the basics of the process, please go ahead and play around with all these features!

```
In [3]: df["Ret25"] = df["Ret25_i"].shift(-25)
df = df.dropna()
df.tail(10)
```

Out[3]:

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i	Ret25
Date							
2025-03-18	-0.61	-6.554621	-15.996749	-4.367412	-6.574936	25.694524	-3.795072
2025-03-19	1.20	-7.450848	-13.129686	-2.831145	-4.729781	28.099556	-3.186662
2025-03-20	-0.53	-9.595756	-14.190763	-3.970333	-5.612264	26.811931	-2.242569
2025-03-21	1.95	-9.613488	-14.131805	-1.773607	-3.346428	29.921378	-3.719238
2025-03-24	1.13	-9.738442	-13.429862	-2.744908	-2.750216	30.800886	-4.309509
2025-03-25	1.37	-8.456087	-13.241573	-1.294066	-1.536051	33.487222	-5.026928
2025-03-26	-0.99	-9.507160	-14.374483	-1.084266	-4.693366	31.220908	-3.703195
2025-03-27	1.05	-8.912227	-12.318013	-0.443875	-0.806104	34.087094	-8.267882
2025-03-28	-2.66	-11.237523	-13.499902	-4.411194	-3.685448	25.103400	-8.729653
2025-03-31	1.94	-10.082015	-11.191258	-1.163172	-1.333480	26.442997	-10.636714

In [4]: `df.describe()`

Out[4]:

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i	
count	8010.000000	8010.000000	8010.000000	8010.000000	8010.000000	8010.000000	8
mean	0.115211	2.919923	7.325607	11.229095	15.196986	32.624228	
std	2.691975	13.389784	21.602310	27.567421	33.122047	56.101787	
min	-51.870000	-69.598132	-77.074930	-72.483802	-74.729250	-75.007329	
25%	-1.160000	-4.791231	-4.843004	-4.884834	-3.924481	-6.063950	
50%	0.100000	2.894122	7.114339	10.713840	14.713342	29.238560	
75%	1.370000	10.587134	19.263231	26.127322	30.736367	59.176953	
max	33.230000	121.286374	116.442841	135.727260	167.513049	300.683525	



2.2 Train-Test samples and Scaling

The next important step, as you know, is in defining our train and test samples, together with the scaling of variables.

Now, as you may remember, we did not perform any scaling in the example from Lesson 2, and that is because all of the input and output variables are essentially of the same nature (i.e., stock returns), but note that it is common (and good) practice to perform the scaling. You can incorporate the scaling here by relying on what we did in the last module of ML. Indeed, you can actually try check whether different scaling methods matter for the predictions.

As for the definition of test and train samples, we will devote the usual first 60% of examples in that data (in chronological order) to training and the remaining 40% to testing:

```
In [5]: # Train-test split
ts = int(0.4 * len(df)) # Number of observations in test sample (20%)
split_time = len(df) - ts # Start of test sample
test_time = df.index[split_time:] # Extract Date index for test period
Ret_vector = df['Ret'][split_time:].values # Extract Ret values for test period
df.tail()
```

```
Out[5]:
```

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i	Ret25
Date							
2025-03-25	1.37	-8.456087	-13.241573	-1.294066	-1.536051	33.487222	-5.026928
2025-03-26	-0.99	-9.507160	-14.374483	-1.084266	-4.693366	31.220908	-3.703195
2025-03-27	1.05	-8.912227	-12.318013	-0.443875	-0.806104	34.087094	-8.267882
2025-03-28	-2.66	-11.237523	-13.499902	-4.411194	-3.685448	25.103400	-8.729653
2025-03-31	1.94	-10.082015	-11.191258	-1.163172	-1.333480	26.442997	-10.636714

So let's formally split our data in train and test samples for input and output variables:

```
In [6]: from sklearn.model_selection import train_test_split

Xdf, ydf = df.iloc[:, 1:-1], df.iloc[:, -1]
X = Xdf.astype("float32")
y = ydf.astype("float32")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=ts, shuffle=False
) # It is important to keep "shuffle=False"
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
(4806, 4) (3204, 4) (4806,) (3204,)
```

2.3 Model and Training

Finally, it is time to set up our model. This is where the new things in the lesson come, which mainly sum up to one: **Keras Tuner**.

- **Keras Tuner**

Keras Tuner is a library that helps us in Hyper-parameter tuning for our TensorFlow programs. The functioning of the library is fairly easy and intuitive, with some specific syntax that you should be aware of. Despite that we will be using the Keras Tuner in this and future notebooks, we strongly recommend that you take a look at some extra resources:

- This simple tutorial from the TensorFlow website:
https://www.tensorflow.org/tutorials/keras/keras_tuner?hl=en
- There are multiple online examples and tutorials on how to use the Keras Tuner, here you have one that you can consult for clarification:
<https://www.youtube.com/watch?v=6Nf1x7qThR8>
- Keras Tuner **library documentation**: https://keras.io/api/keras_tuner/

Now the Keras Tuners works in a rather simple way, but it requires us to perform tasks in an organized way. Let's see how.

First we will need to import the library, of course:

```
In [7]: !pip install keras_tuner
```

Requirement already satisfied: keras_tuner in /usr/local/lib/python3.11/site-packages (1.4.7)

Requirement already satisfied: keras in /usr/local/lib/python3.11/site-packages (from keras_tuner) (3.9.2)

Requirement already satisfied: packaging in /usr/local/lib/python3.11/site-packages (from keras_tuner) (23.1)

Requirement already satisfied: requests in /usr/local/lib/python3.11/site-packages (from keras_tuner) (2.32.3)

Requirement already satisfied: kt-legacy in /usr/local/lib/python3.11/site-packages (from keras_tuner) (1.0.5)

Requirement already satisfied: absl-py in /usr/local/lib/python3.11/site-packages (from keras->keras_tuner) (2.2.2)

Requirement already satisfied: numpy in /usr/local/lib/python3.11/site-packages (from keras->keras_tuner) (1.26.2)

Requirement already satisfied: rich in /usr/local/lib/python3.11/site-packages (from keras->keras_tuner) (14.0.0)

Requirement already satisfied: namex in /usr/local/lib/python3.11/site-packages (from keras->keras_tuner) (0.0.9)

Requirement already satisfied: h5py in /usr/local/lib/python3.11/site-packages (from keras->keras_tuner) (3.13.0)

Requirement already satisfied: optree in /usr/local/lib/python3.11/site-packages (from keras->keras_tuner) (0.15.0)

Requirement already satisfied: ml-dtypes in /usr/local/lib/python3.11/site-packages (from keras->keras_tuner) (0.5.1)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/site-packages (from requests->keras_tuner) (3.1.0)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/site-packages (from requests->keras_tuner) (3.4)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/site-packages (from requests->keras_tuner) (1.26.15)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/site-packages (from requests->keras_tuner) (2022.12.7)

Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/lib/python3.11/site-packages (from optree->keras->keras_tuner) (4.13.2)

Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.11/site-packages (from rich->keras->keras_tuner) (3.0.0)

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.11/site-packages (from rich->keras->keras_tuner) (2.15.1)

Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.11/site-packages (from markdown-it-py>=2.2.0->rich->keras->keras_tuner) (0.1.2)

WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>

[notice] A new release of pip available: 22.3.1 -> 25.1

[notice] To update, run: `pip install --upgrade pip`

```
In [8]: import keras_tuner as kt
import tensorflow as tf
from keras_tuner import HyperModel
```

```
2025-05-07 07:46:50.146413: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-05-07 07:46:50.195517: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2025-05-07 07:46:51.312736: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
```

Then, in addition to setting the random seed for replicability and the validation split for the use of early-stopping later on, we need to define a function **build()** within the **MLP_model** class.

This build function contains the different 'choices' we are going to ask the Keras Tuner to decide for us. In this first and simple example we are simply tuning 3 hyper-parameters:

- **Dropout:** we are letting the tuner choose between a value for dropout of 0.2 or 0.3.
- **Number of layers in the model:** in this case, we will let the tuner choose between 1 and 5 hidden layers in the model. We do this using the **hp.Int()** method.
- **Number of units in each hidden layer:** the tuner will choose between 1 (min_value) and 25 (max_value) units in each layer, with a step size of 5.

```
In [9]: tf.keras.backend.clear_session()
tf.random.set_seed(1234)
val_split = 0.2

class MLP_model(HyperModel):
    def build(self, hp):
        # We define a constant activation function of ReLU form. We will not be
        act_fun = "relu"

        # We do ask the Keras Tuner to choose whether is best to have a dropout
        n_dropout = hp.Choice("n_dropout", values=[0.20, 0.30])

        model = tf.keras.models.Sequential()

        # Now, we will use a loop to let the tuner choose the number of layers to
        for i in range(1, hp.Int("num_layers", 1, 5)):
            # Within this loop, we will also ask the tuner to decide the optimal
            model.add(
                tf.keras.layers.Dense(
                    units=hp.Int(
                        "units_dense_" + str(i), min_value=1, max_value=25, step
                    ),
                    activation=act_fun,
                )
            )
```

```

    )
    model.add(tf.keras.layers.Dropout(n_dropout))

    model.add(tf.keras.layers.Dense(units=1))

    # As was the case with the activation function, there is no tuning on th
    hp_lr = 1e-5
    adam = tf.keras.optimizers.Adam(learning_rate=hp_lr)
    model.compile(optimizer=adam, loss="mean_absolute_error")

    return model

```

Now, once we have defined our HyperModel (i.e., **MLP_model()** class), we can perform the tuning of the hyper-parameters. This is, essentially, another optimization algorithm (**Tuner**), for which we would also need to make some decisions:

- **Hypermodel:** we have already defined our hypermodel: `MLP_model()`
- **Objective:** what is the objective function to optimize? In this case we will minimize the loss function in the validation set.
- **Other parameters:** these include whether we overwrite results, fixing the random seed, or storage of results (convenient for large complex models).

But what about the **Tuner**? In this case, we will select the Hyperband tuner embedded in Keras, for which you can consult the documentation here:

https://keras.io/api/keras_tuner/tuners/hyperband/#hyperband-class

Arguably, you could easily have decided to use another kind of optimizer from the ones available in the Keras Tuner (https://keras.io/api/keras_tuner/tuners/). Feel free to come back here once you have completed the notebook and see what would happen under another tuner!

```

In [10]: import os

from keras import backend as K

# First, we clear the session just to make sure our seeds are correctly working
K.clear_session()

# Then, we call the model and perform the tuning:
hypermodel = MLP_model()
tuner = kt.Hyperband(
    hypermodel,
    objective=kt.Objective("val_loss", direction="min"),
    overwrite=True,
    max_epochs=10,
    seed=1234,
    directory=os.path.normpath("C:/"),
)

# Let's run the tuner! (Warning: this could take time)

```

```
tuner.search(X_train, y_train, validation_split=val_split)
best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]
```

Trial 30 Complete [00h 00m 05s]
val_loss: 8.497085571289062

Best val_loss So Far: 7.930953025817871
Total elapsed time: 00h 01m 14s

In [11]: `best_hps.values`

```
Out[11]: {'n_dropout': 0.2,
          'num_layers': 3,
          'units_dense_1': 11,
          'units_dense_2': 21,
          'units_dense_3': 11,
          'units_dense_4': 21,
          'tuner/epochs': 10,
          'tuner/initial_epoch': 4,
          'tuner/bracket': 1,
          'tuner/round': 1,
          'tuner/trial_id': '0021'}
```

Now, we can use these tuned hyper-parameter values to train our model, together with a usual early-stopping criteria:

```
In [12]: model = tuner.hypermodel.build(best_hps)
         es = tf.keras.callbacks.EarlyStopping(
             monitor="val_loss", mode="min", verbose=1, patience=20, restore_best_weights
         )

         # fit the model
         model.fit(
             X_train,
             y_train,
             validation_split=0.2,
             epochs=200,
             batch_size=32,
             verbose=2,
             callbacks=[es],
         )
```

Epoch 1/200
121/121 - 1s - 8ms/step - loss: 60.7589 - val_loss: 55.2892
Epoch 2/200
121/121 - 1s - 4ms/step - loss: 60.5621 - val_loss: 54.1300
Epoch 3/200
121/121 - 0s - 2ms/step - loss: 57.7094 - val_loss: 53.0120
Epoch 4/200
121/121 - 0s - 3ms/step - loss: 57.6723 - val_loss: 51.8763
Epoch 5/200
121/121 - 0s - 2ms/step - loss: 56.7486 - val_loss: 50.7455
Epoch 6/200
121/121 - 0s - 3ms/step - loss: 54.7629 - val_loss: 49.6327
Epoch 7/200
121/121 - 0s - 2ms/step - loss: 54.0818 - val_loss: 48.5352
Epoch 8/200
121/121 - 0s - 3ms/step - loss: 52.7764 - val_loss: 47.4433
Epoch 9/200
121/121 - 0s - 3ms/step - loss: 52.4463 - val_loss: 46.3664
Epoch 10/200
121/121 - 0s - 2ms/step - loss: 51.0419 - val_loss: 45.2981
Epoch 11/200
121/121 - 0s - 2ms/step - loss: 49.1633 - val_loss: 44.2710
Epoch 12/200
121/121 - 0s - 2ms/step - loss: 48.2504 - val_loss: 43.2420
Epoch 13/200
121/121 - 0s - 3ms/step - loss: 47.6171 - val_loss: 42.2085
Epoch 14/200
121/121 - 0s - 2ms/step - loss: 46.6572 - val_loss: 41.1772
Epoch 15/200
121/121 - 0s - 3ms/step - loss: 46.3400 - val_loss: 40.1405
Epoch 16/200
121/121 - 0s - 2ms/step - loss: 44.4603 - val_loss: 39.1390
Epoch 17/200
121/121 - 0s - 2ms/step - loss: 43.9083 - val_loss: 38.1496
Epoch 18/200
121/121 - 0s - 2ms/step - loss: 42.4401 - val_loss: 37.1640
Epoch 19/200
121/121 - 0s - 3ms/step - loss: 42.3213 - val_loss: 36.1823
Epoch 20/200
121/121 - 0s - 2ms/step - loss: 41.1723 - val_loss: 35.2069
Epoch 21/200
121/121 - 0s - 3ms/step - loss: 40.4357 - val_loss: 34.2402
Epoch 22/200
121/121 - 0s - 2ms/step - loss: 39.6546 - val_loss: 33.2788
Epoch 23/200
121/121 - 0s - 3ms/step - loss: 37.9864 - val_loss: 32.3380
Epoch 24/200
121/121 - 0s - 2ms/step - loss: 36.7197 - val_loss: 31.4274
Epoch 25/200
121/121 - 0s - 3ms/step - loss: 36.6431 - val_loss: 30.4911
Epoch 26/200
121/121 - 0s - 2ms/step - loss: 35.4619 - val_loss: 29.5744
Epoch 27/200
121/121 - 0s - 2ms/step - loss: 35.1836 - val_loss: 28.6601
Epoch 28/200
121/121 - 0s - 3ms/step - loss: 33.9963 - val_loss: 27.7613
Epoch 29/200
121/121 - 0s - 2ms/step - loss: 33.8394 - val_loss: 26.8533
Epoch 30/200
121/121 - 0s - 2ms/step - loss: 32.4806 - val_loss: 25.9754

Epoch 31/200
121/121 - 0s - 2ms/step - loss: 32.4247 - val_loss: 25.0806
Epoch 32/200
121/121 - 0s - 3ms/step - loss: 30.8789 - val_loss: 24.2257
Epoch 33/200
121/121 - 0s - 2ms/step - loss: 30.3947 - val_loss: 23.3857
Epoch 34/200
121/121 - 0s - 2ms/step - loss: 29.3155 - val_loss: 22.5609
Epoch 35/200
121/121 - 0s - 2ms/step - loss: 29.9390 - val_loss: 21.7183
Epoch 36/200
121/121 - 0s - 2ms/step - loss: 28.6318 - val_loss: 20.9313
Epoch 37/200
121/121 - 0s - 3ms/step - loss: 28.5308 - val_loss: 20.1763
Epoch 38/200
121/121 - 0s - 2ms/step - loss: 27.4630 - val_loss: 19.4093
Epoch 39/200
121/121 - 0s - 2ms/step - loss: 27.7033 - val_loss: 18.6465
Epoch 40/200
121/121 - 0s - 3ms/step - loss: 26.6301 - val_loss: 17.8917
Epoch 41/200
121/121 - 0s - 2ms/step - loss: 26.6684 - val_loss: 17.1859
Epoch 42/200
121/121 - 0s - 2ms/step - loss: 25.9235 - val_loss: 16.4922
Epoch 43/200
121/121 - 0s - 3ms/step - loss: 25.6605 - val_loss: 15.8231
Epoch 44/200
121/121 - 0s - 2ms/step - loss: 25.1218 - val_loss: 15.2005
Epoch 45/200
121/121 - 0s - 2ms/step - loss: 24.3488 - val_loss: 14.6080
Epoch 46/200
121/121 - 0s - 2ms/step - loss: 24.7702 - val_loss: 14.0242
Epoch 47/200
121/121 - 0s - 3ms/step - loss: 24.3320 - val_loss: 13.4639
Epoch 48/200
121/121 - 0s - 2ms/step - loss: 24.0687 - val_loss: 12.9288
Epoch 49/200
121/121 - 0s - 3ms/step - loss: 23.4684 - val_loss: 12.5103
Epoch 50/200
121/121 - 0s - 2ms/step - loss: 23.1510 - val_loss: 12.1158
Epoch 51/200
121/121 - 0s - 3ms/step - loss: 22.9878 - val_loss: 11.7210
Epoch 52/200
121/121 - 0s - 2ms/step - loss: 22.4297 - val_loss: 11.4079
Epoch 53/200
121/121 - 0s - 3ms/step - loss: 22.7052 - val_loss: 11.1035
Epoch 54/200
121/121 - 0s - 2ms/step - loss: 22.3456 - val_loss: 10.7939
Epoch 55/200
121/121 - 0s - 2ms/step - loss: 22.7060 - val_loss: 10.5122
Epoch 56/200
121/121 - 0s - 2ms/step - loss: 21.8393 - val_loss: 10.2943
Epoch 57/200
121/121 - 0s - 3ms/step - loss: 21.8695 - val_loss: 10.1145
Epoch 58/200
121/121 - 0s - 2ms/step - loss: 21.8387 - val_loss: 9.9440
Epoch 59/200
121/121 - 0s - 2ms/step - loss: 21.2471 - val_loss: 9.7854
Epoch 60/200
121/121 - 0s - 3ms/step - loss: 21.5242 - val_loss: 9.6590

Epoch 61/200
121/121 - 0s - 2ms/step - loss: 21.3108 - val_loss: 9.5461
Epoch 62/200
121/121 - 0s - 3ms/step - loss: 20.8671 - val_loss: 9.4506
Epoch 63/200
121/121 - 0s - 2ms/step - loss: 20.9935 - val_loss: 9.3523
Epoch 64/200
121/121 - 0s - 2ms/step - loss: 20.7324 - val_loss: 9.2702
Epoch 65/200
121/121 - 0s - 3ms/step - loss: 21.1063 - val_loss: 9.2183
Epoch 66/200
121/121 - 0s - 2ms/step - loss: 20.9318 - val_loss: 9.1708
Epoch 67/200
121/121 - 0s - 2ms/step - loss: 20.6103 - val_loss: 9.1311
Epoch 68/200
121/121 - 0s - 3ms/step - loss: 20.2585 - val_loss: 9.1028
Epoch 69/200
121/121 - 0s - 2ms/step - loss: 20.4676 - val_loss: 9.0806
Epoch 70/200
121/121 - 0s - 2ms/step - loss: 20.4251 - val_loss: 9.0507
Epoch 71/200
121/121 - 0s - 3ms/step - loss: 20.2054 - val_loss: 9.0271
Epoch 72/200
121/121 - 0s - 2ms/step - loss: 20.3319 - val_loss: 9.0065
Epoch 73/200
121/121 - 0s - 2ms/step - loss: 20.0951 - val_loss: 8.9848
Epoch 74/200
121/121 - 0s - 2ms/step - loss: 19.8333 - val_loss: 8.9672
Epoch 75/200
121/121 - 0s - 3ms/step - loss: 19.8877 - val_loss: 8.9504
Epoch 76/200
121/121 - 0s - 2ms/step - loss: 20.1308 - val_loss: 8.9287
Epoch 77/200
121/121 - 0s - 2ms/step - loss: 20.1989 - val_loss: 8.9140
Epoch 78/200
121/121 - 0s - 2ms/step - loss: 19.7295 - val_loss: 8.9034
Epoch 79/200
121/121 - 0s - 2ms/step - loss: 19.5249 - val_loss: 8.8879
Epoch 80/200
121/121 - 0s - 2ms/step - loss: 19.8580 - val_loss: 8.8795
Epoch 81/200
121/121 - 0s - 2ms/step - loss: 19.3913 - val_loss: 8.8655
Epoch 82/200
121/121 - 0s - 2ms/step - loss: 19.5088 - val_loss: 8.8486
Epoch 83/200
121/121 - 0s - 3ms/step - loss: 19.7102 - val_loss: 8.8317
Epoch 84/200
121/121 - 0s - 2ms/step - loss: 19.0490 - val_loss: 8.8178
Epoch 85/200
121/121 - 0s - 2ms/step - loss: 19.2517 - val_loss: 8.8017
Epoch 86/200
121/121 - 0s - 2ms/step - loss: 19.4366 - val_loss: 8.7883
Epoch 87/200
121/121 - 0s - 2ms/step - loss: 19.2342 - val_loss: 8.7759
Epoch 88/200
121/121 - 0s - 2ms/step - loss: 19.0456 - val_loss: 8.7625
Epoch 89/200
121/121 - 0s - 3ms/step - loss: 19.2636 - val_loss: 8.7499
Epoch 90/200
121/121 - 0s - 3ms/step - loss: 18.8381 - val_loss: 8.7439

Epoch 91/200
121/121 - 0s - 2ms/step - loss: 18.7859 - val_loss: 8.7276
Epoch 92/200
121/121 - 0s - 3ms/step - loss: 18.7102 - val_loss: 8.7135
Epoch 93/200
121/121 - 0s - 2ms/step - loss: 18.8509 - val_loss: 8.7022
Epoch 94/200
121/121 - 0s - 3ms/step - loss: 18.4496 - val_loss: 8.6888
Epoch 95/200
121/121 - 0s - 2ms/step - loss: 18.6287 - val_loss: 8.6800
Epoch 96/200
121/121 - 0s - 3ms/step - loss: 18.4114 - val_loss: 8.6630
Epoch 97/200
121/121 - 0s - 2ms/step - loss: 18.7350 - val_loss: 8.6539
Epoch 98/200
121/121 - 0s - 3ms/step - loss: 18.5229 - val_loss: 8.6417
Epoch 99/200
121/121 - 0s - 3ms/step - loss: 18.1560 - val_loss: 8.6340
Epoch 100/200
121/121 - 0s - 3ms/step - loss: 18.3762 - val_loss: 8.6195
Epoch 101/200
121/121 - 0s - 2ms/step - loss: 18.4043 - val_loss: 8.6087
Epoch 102/200
121/121 - 0s - 2ms/step - loss: 18.0133 - val_loss: 8.5992
Epoch 103/200
121/121 - 0s - 2ms/step - loss: 18.4696 - val_loss: 8.5874
Epoch 104/200
121/121 - 0s - 3ms/step - loss: 18.5018 - val_loss: 8.5749
Epoch 105/200
121/121 - 0s - 3ms/step - loss: 18.1270 - val_loss: 8.5667
Epoch 106/200
121/121 - 0s - 2ms/step - loss: 18.0545 - val_loss: 8.5586
Epoch 107/200
121/121 - 0s - 2ms/step - loss: 18.1155 - val_loss: 8.5503
Epoch 108/200
121/121 - 0s - 2ms/step - loss: 17.6787 - val_loss: 8.5401
Epoch 109/200
121/121 - 0s - 2ms/step - loss: 17.8990 - val_loss: 8.5294
Epoch 110/200
121/121 - 0s - 2ms/step - loss: 17.8791 - val_loss: 8.5178
Epoch 111/200
121/121 - 0s - 2ms/step - loss: 17.7268 - val_loss: 8.5094
Epoch 112/200
121/121 - 0s - 2ms/step - loss: 17.8260 - val_loss: 8.4995
Epoch 113/200
121/121 - 0s - 2ms/step - loss: 17.5905 - val_loss: 8.4873
Epoch 114/200
121/121 - 0s - 2ms/step - loss: 17.6476 - val_loss: 8.4740
Epoch 115/200
121/121 - 0s - 2ms/step - loss: 17.5049 - val_loss: 8.4667
Epoch 116/200
121/121 - 0s - 2ms/step - loss: 17.4171 - val_loss: 8.4619
Epoch 117/200
121/121 - 0s - 3ms/step - loss: 17.4891 - val_loss: 8.4548
Epoch 118/200
121/121 - 0s - 3ms/step - loss: 17.2399 - val_loss: 8.4461
Epoch 119/200
121/121 - 0s - 2ms/step - loss: 17.7266 - val_loss: 8.4394
Epoch 120/200
121/121 - 0s - 2ms/step - loss: 17.1003 - val_loss: 8.4281

Epoch 121/200
121/121 - 0s - 2ms/step - loss: 17.1450 - val_loss: 8.4250
Epoch 122/200
121/121 - 0s - 3ms/step - loss: 17.2077 - val_loss: 8.4173
Epoch 123/200
121/121 - 0s - 2ms/step - loss: 17.1851 - val_loss: 8.4081
Epoch 124/200
121/121 - 0s - 2ms/step - loss: 17.0245 - val_loss: 8.4005
Epoch 125/200
121/121 - 0s - 2ms/step - loss: 17.0555 - val_loss: 8.3916
Epoch 126/200
121/121 - 0s - 2ms/step - loss: 17.3224 - val_loss: 8.3841
Epoch 127/200
121/121 - 0s - 2ms/step - loss: 17.0983 - val_loss: 8.3764
Epoch 128/200
121/121 - 0s - 3ms/step - loss: 17.1480 - val_loss: 8.3719
Epoch 129/200
121/121 - 0s - 2ms/step - loss: 16.9905 - val_loss: 8.3690
Epoch 130/200
121/121 - 0s - 2ms/step - loss: 17.1824 - val_loss: 8.3590
Epoch 131/200
121/121 - 0s - 3ms/step - loss: 16.8188 - val_loss: 8.3519
Epoch 132/200
121/121 - 0s - 2ms/step - loss: 16.7538 - val_loss: 8.3456
Epoch 133/200
121/121 - 0s - 2ms/step - loss: 16.7591 - val_loss: 8.3410
Epoch 134/200
121/121 - 0s - 3ms/step - loss: 16.6442 - val_loss: 8.3340
Epoch 135/200
121/121 - 0s - 2ms/step - loss: 16.8611 - val_loss: 8.3282
Epoch 136/200
121/121 - 0s - 2ms/step - loss: 16.4589 - val_loss: 8.3210
Epoch 137/200
121/121 - 0s - 2ms/step - loss: 16.5247 - val_loss: 8.3154
Epoch 138/200
121/121 - 0s - 2ms/step - loss: 16.1393 - val_loss: 8.3103
Epoch 139/200
121/121 - 0s - 2ms/step - loss: 16.7264 - val_loss: 8.3063
Epoch 140/200
121/121 - 0s - 3ms/step - loss: 16.4530 - val_loss: 8.3024
Epoch 141/200
121/121 - 0s - 2ms/step - loss: 16.5194 - val_loss: 8.2980
Epoch 142/200
121/121 - 0s - 2ms/step - loss: 16.5381 - val_loss: 8.2921
Epoch 143/200
121/121 - 0s - 2ms/step - loss: 16.4405 - val_loss: 8.2878
Epoch 144/200
121/121 - 0s - 2ms/step - loss: 16.3134 - val_loss: 8.2824
Epoch 145/200
121/121 - 0s - 3ms/step - loss: 16.5686 - val_loss: 8.2787
Epoch 146/200
121/121 - 0s - 2ms/step - loss: 16.0627 - val_loss: 8.2778
Epoch 147/200
121/121 - 0s - 2ms/step - loss: 16.2156 - val_loss: 8.2741
Epoch 148/200
121/121 - 0s - 3ms/step - loss: 16.4070 - val_loss: 8.2690
Epoch 149/200
121/121 - 0s - 3ms/step - loss: 16.3552 - val_loss: 8.2653
Epoch 150/200
121/121 - 0s - 2ms/step - loss: 16.4400 - val_loss: 8.2619

Epoch 151/200
121/121 - 0s - 2ms/step - loss: 16.3181 - val_loss: 8.2577
Epoch 152/200
121/121 - 0s - 2ms/step - loss: 15.9846 - val_loss: 8.2554
Epoch 153/200
121/121 - 0s - 2ms/step - loss: 15.9779 - val_loss: 8.2510
Epoch 154/200
121/121 - 0s - 2ms/step - loss: 15.9585 - val_loss: 8.2458
Epoch 155/200
121/121 - 0s - 2ms/step - loss: 15.8787 - val_loss: 8.2418
Epoch 156/200
121/121 - 0s - 2ms/step - loss: 15.7248 - val_loss: 8.2389
Epoch 157/200
121/121 - 0s - 2ms/step - loss: 15.9865 - val_loss: 8.2365
Epoch 158/200
121/121 - 0s - 3ms/step - loss: 15.9792 - val_loss: 8.2338
Epoch 159/200
121/121 - 0s - 2ms/step - loss: 15.9160 - val_loss: 8.2336
Epoch 160/200
121/121 - 0s - 2ms/step - loss: 15.8486 - val_loss: 8.2327
Epoch 161/200
121/121 - 0s - 3ms/step - loss: 15.9467 - val_loss: 8.2316
Epoch 162/200
121/121 - 0s - 2ms/step - loss: 15.6031 - val_loss: 8.2291
Epoch 163/200
121/121 - 0s - 2ms/step - loss: 15.5325 - val_loss: 8.2285
Epoch 164/200
121/121 - 0s - 3ms/step - loss: 15.6076 - val_loss: 8.2267
Epoch 165/200
121/121 - 0s - 2ms/step - loss: 15.5278 - val_loss: 8.2232
Epoch 166/200
121/121 - 0s - 3ms/step - loss: 15.5186 - val_loss: 8.2229
Epoch 167/200
121/121 - 0s - 2ms/step - loss: 15.5207 - val_loss: 8.2220
Epoch 168/200
121/121 - 0s - 2ms/step - loss: 15.6605 - val_loss: 8.2204
Epoch 169/200
121/121 - 0s - 2ms/step - loss: 15.6555 - val_loss: 8.2190
Epoch 170/200
121/121 - 0s - 3ms/step - loss: 15.3211 - val_loss: 8.2179
Epoch 171/200
121/121 - 0s - 2ms/step - loss: 15.5223 - val_loss: 8.2168
Epoch 172/200
121/121 - 0s - 3ms/step - loss: 15.7393 - val_loss: 8.2152
Epoch 173/200
121/121 - 0s - 2ms/step - loss: 15.5104 - val_loss: 8.2140
Epoch 174/200
121/121 - 0s - 2ms/step - loss: 15.7142 - val_loss: 8.2124
Epoch 175/200
121/121 - 0s - 2ms/step - loss: 15.3751 - val_loss: 8.2098
Epoch 176/200
121/121 - 0s - 2ms/step - loss: 15.4906 - val_loss: 8.2105
Epoch 177/200
121/121 - 0s - 3ms/step - loss: 15.3598 - val_loss: 8.2091
Epoch 178/200
121/121 - 0s - 3ms/step - loss: 15.1495 - val_loss: 8.2066
Epoch 179/200
121/121 - 0s - 2ms/step - loss: 15.3605 - val_loss: 8.2060
Epoch 180/200
121/121 - 0s - 3ms/step - loss: 15.3270 - val_loss: 8.2044

```

Epoch 181/200
121/121 - 0s - 2ms/step - loss: 15.1333 - val_loss: 8.2020
Epoch 182/200
121/121 - 0s - 2ms/step - loss: 15.4351 - val_loss: 8.2012
Epoch 183/200
121/121 - 0s - 2ms/step - loss: 15.2774 - val_loss: 8.1993
Epoch 184/200
121/121 - 0s - 3ms/step - loss: 15.1772 - val_loss: 8.1995
Epoch 185/200
121/121 - 0s - 3ms/step - loss: 15.2011 - val_loss: 8.1982
Epoch 186/200
121/121 - 0s - 3ms/step - loss: 15.3532 - val_loss: 8.1984
Epoch 187/200
121/121 - 1s - 5ms/step - loss: 15.0340 - val_loss: 8.1979
Epoch 188/200
121/121 - 0s - 2ms/step - loss: 15.0379 - val_loss: 8.1965
Epoch 189/200
121/121 - 0s - 3ms/step - loss: 15.0680 - val_loss: 8.1950
Epoch 190/200
121/121 - 0s - 2ms/step - loss: 14.9889 - val_loss: 8.1930
Epoch 191/200
121/121 - 0s - 3ms/step - loss: 15.1607 - val_loss: 8.1920
Epoch 192/200
121/121 - 0s - 2ms/step - loss: 15.0040 - val_loss: 8.1920
Epoch 193/200
121/121 - 0s - 2ms/step - loss: 14.9702 - val_loss: 8.1928
Epoch 194/200
121/121 - 0s - 3ms/step - loss: 15.0247 - val_loss: 8.1941
Epoch 195/200
121/121 - 0s - 2ms/step - loss: 14.7758 - val_loss: 8.1931
Epoch 196/200
121/121 - 0s - 2ms/step - loss: 14.8594 - val_loss: 8.1926
Epoch 197/200
121/121 - 0s - 2ms/step - loss: 15.0426 - val_loss: 8.1922
Epoch 198/200
121/121 - 0s - 3ms/step - loss: 14.9661 - val_loss: 8.1916
Epoch 199/200
121/121 - 0s - 2ms/step - loss: 14.9118 - val_loss: 8.1912
Epoch 200/200
121/121 - 0s - 3ms/step - loss: 14.7804 - val_loss: 8.1902
Restoring model weights from the end of the best epoch: 200.

```

```
Out[12]: <keras.src.callbacks.history.History at 0x7ef1a0703690>
```

This is how our 'tuned' model design will look, with a total of 2 hidden layers (with 6 and 21 units) and a final output layer:

```
In [13]: model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_5 (Dense)	(None, 11)	55
dropout_4 (Dropout)	(None, 11)	0
dense_6 (Dense)	(None, 21)	252
dropout_5 (Dropout)	(None, 21)	0
dense_7 (Dense)	(None, 1)	22

Total params: 989 (3.87 KB)

Trainable params: 329 (1.29 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 660 (2.58 KB)

And here we will end the deep learning part of our task today! We have built a somewhat simple model using hyper-parameter tuning. Now, let's see whether the predictions (out-of-sample, of course) of this model are actually useful in a trading strategy for the test period selected.

3. Financial performance of the model

Now, we will check if the predictions delivered in the test sample are valid for developing a trading strategy!

3.1 Backtesting out-of-sample (in test sample)

As usual, we will now perform a backtesting of a trading strategy based on the predictions of the model:

```
In [14]: y_pred = model.predict(X_test)
```

101/101 ————— 0s 925us/step

```
In [15]: df_predictions = pd.DataFrame(
    {
        'Date': test_time, # No flatten needed for DatetimeIndex
        'Pred': y_pred.flatten() if y_pred.ndim > 1 else y_pred, # Flatten if 2
        'Ret': Ret_vector.flatten() if Ret_vector.ndim > 1 else Ret_vector, # F
    }
)
df_predictions.tail()
df = df_predictions
df.tail()
```

Out[15]:

	Date	Pred	Ret
3199	2025-03-25	1.448421	1.37
3200	2025-03-26	1.211321	-0.99
3201	2025-03-27	1.418115	1.05
3202	2025-03-28	1.219985	-2.66
3203	2025-03-31	1.213160	1.94

As we have done other times before, we will compute the returns of 3 strategies in the test period:

- **Buy and hold:** simply buys the stock at the beginning of the test period and holds it until the end.
- **Long-only:** this strategy either goes long in the stock (if the predicted 25-day return is positive) or goes to cash (with a 0% return associated).
- **Long/Short** this strategy goes long (short) in the stock when the 25-day ahead return predicted by the model is positive (negative)

```
In [16]: df['Ret'] = df['Ret'] / 100

df["Positions"] = np.where(df["Pred"] > 0, 1, -1)
df["Strat_ret"] = df["Positions"].shift(1) * df["Ret"]
df["Positions_L"] = df["Positions"].shift(1)
df["Positions_L"][df["Positions_L"] == -1] = 0
df["Strat_ret_L"] = df["Positions_L"] * df["Ret"]
df["CumRet"] = df["Strat_ret"].expanding().apply(lambda x: np.prod(1 + x) - 1)
df["CumRet_L"] = df["Strat_ret_L"].expanding().apply(lambda x: np.prod(1 + x) - 1)
df["bhRet"] = df["Ret"].expanding().apply(lambda x: np.prod(1 + x) - 1)

Final_Return_L = np.prod(1 + df["Strat_ret_L"]) - 1
Final_Return = np.prod(1 + df["Strat_ret"]) - 1
Buy_Return = np.prod(1 + df["Ret"]) - 1

print("Strat Return Long Only =", Final_Return_L * 100, "%")
print("Strat Return =", Final_Return * 100, "%")
print("Buy and Hold Return =", Buy_Return * 100, "%")
```

/tmp/ipykernel_13387/2786902323.py:6: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df["Positions_L"][df["Positions_L"] == -1] = 0
```

Strat Return Long Only = 739.5384870082764 %

Strat Return = 407.82058285920135 %

Buy and Hold Return = 1177.106632396256 %

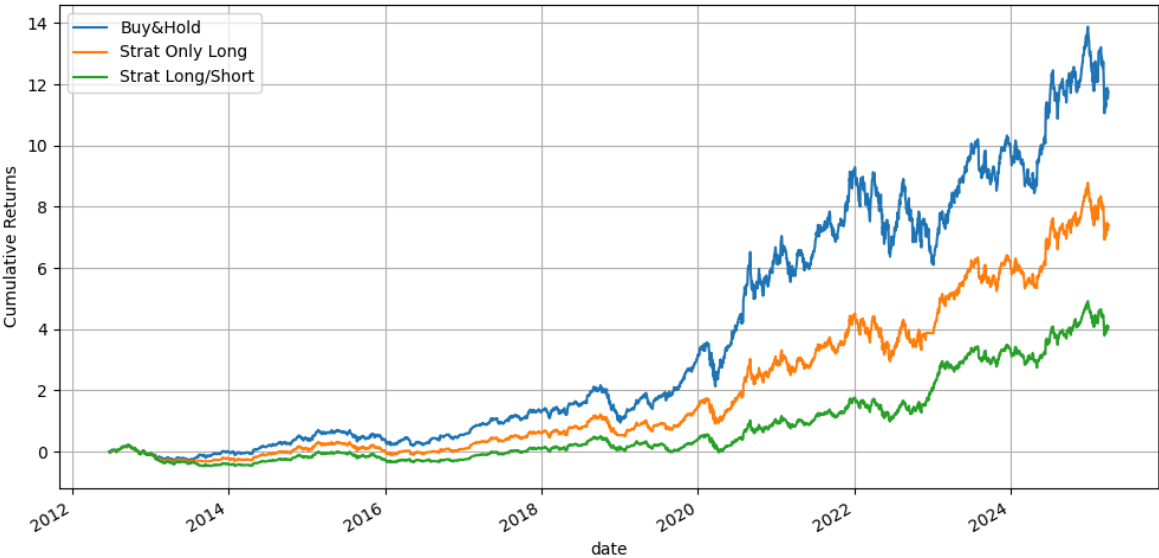
Judging by the numbers obtained here, it seems that, under these tuned hyper-parameters and, remember, for the testing period of our choice, our model does a fairly

decent job in timing the AAPL stock. Let's better see it graphically:

```
In [17]: import matplotlib.pyplot as plt

fig = plt.figure(figsize=(12, 6))
ax = plt.gca()
df.plot(x="Date", y="bhRet", label="Buy&Hold", ax=ax)
df.plot(x="Date", y="CumRet_L", label="Strat Only Long", ax=ax)
df.plot(x="Date", y="CumRet", label="Strat Long/Short", ax=ax)
plt.xlabel("date")
plt.ylabel("Cumulative Returns")
plt.grid()
plt.show()

df.describe()
```



Out[17]:

	Date	Pred	Ret	Positions	Strat_ret	Positior
count	3204	3204.000000	3204.000000	3204.000000	3203.000000	3203.000
mean	2018-11-08 16:05:23.595505664	1.702753	0.000952	0.859551	0.000664	0.929
min	2012-06-22 00:00:00	-1.788622	-0.128600	-1.000000	-0.128600	0.000
25%	2015-08-31 18:00:00	0.736493	-0.007500	1.000000	-0.007700	1.000
50%	2018-11-06 12:00:00	1.479906	0.001000	1.000000	0.000700	1.000
75%	2022-01-13 06:00:00	2.439269	0.010300	1.000000	0.010000	1.000
max	2025-03-31 00:00:00	8.799627	0.119800	1.000000	0.119800	1.000
std	NaN	1.450142	0.017669	0.511130	0.017684	0.255



4. Conclusion

In this last lesson of Module 1, we have implemented a basic tuning of model hyperparameters. Even though we have performed a somewhat simple tuning that focused on very few hyper-parameters, there are 2 things that we have likely come to realize: (i) tuning is computationally costly, and (ii) it can actually make a huge difference!. In the next module, we will continue to explore more advanced Deep Learning models, that will allow us to gradually increase the complexity of the tasks at hand and its implementation.

See you there!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Module 1: Lesson 1

Multilayer Perceptrons (MLPs)



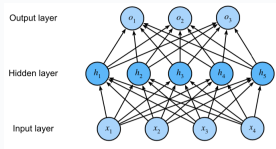
Outline

- ▶ Revisiting MLPs
- ▶ Initialization: vanishing and exploding gradients
- ▶ A quick note on regularization
- ▶ Dropout

Revisiting MLPs

During the last module of the Machine Learning course, we introduced a simple example of a deep Neural Network (NN): Multilayer Perceptrons (MLPs). This is precisely the point of departure for the Deep Learning for Finance course.

- Remember that we can express an MLP as a series of hidden layers with different units (neurons), each of which is activated (or not) given an activation function that incorporates the non-linearity into the process:



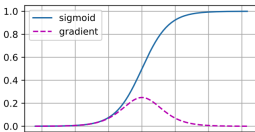
While we already know the basics about how these NNs work (e.g., forward and backprop), there are still many features to discuss that can help increase the efficiency and performance of our models. This is what we'll start doing in this lesson.

This lesson is based on Chapter 5 of Zhang et al.'s book *Dive into Deep Learning*.

Parameter initialization

Let's start discussing these by looking at the first task we need to perform when training: parameter initialization.

- ▶ So far, we have given relatively low importance to the way parameters (W and b) were initialized.
- ▶ We simply said that this was better done by selecting random numbers from a pre-specified distribution (e.g., uniform).
- ▶ However, the way that we initialize these parameters (e.g., the distribution we choose) can have significant effects on training.
- ▶ One of the most perilous effects of initialization choices is related to the (in)stability of gradients. The random choice of weights can lead to updates in parameters while training that are either:
 - Too large → Exploding gradient problem → difficult for gradient descent to converge
 - Too small → Vanishing gradient problem → difficult for algorithm to learn



How to initialize parameters

Is there any way to get rid of problems related to exploding/vanishing gradients?

Sadly, there is no universal way to get rid of the problems related to parameter initialization. There are, however, some things that we can do in order to mitigate their impact:

► **Default initialization in Keras:** One lazy option is to trust that Keras is taking care of the problem for us:

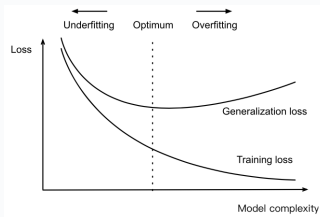
- Indeed, Keras has a default option for dense layers (the only ones we have seen so far) that relies on a uniform distribution called glorot uniform. Here, you have the link to the different default options when we set a dense layer: https://keras.io/api/layers/core_layers/dense/
- Here, you have the link to the Keras documentation for all the different initializers: <https://keras.io/api/layers/initializers/>

► **Xavier initialization:**

- Probably the one most commonly used in practice (actually glorot uniform is a type of Xavier)
- The typical Xavier initialization uses random weights from a Gaussian (or uniform) distribution with zero mean but conditioning the variance to avoid exploding/vanishing problems.

Regularization

Machine learning models, as we've already learned, are always subject to the trade-off between in- and out-of-sample performance. Deep learning models based on neural networks are not an exception to this problem:



When our models did overfit, we already worked out some solutions based on **regularization** of the loss function. These are also applicable in deep learning models, as we will see:

- ▶ Lasso (ℓ_1 -norm)
- ▶ Ridge (ℓ_2 -norm)
- ▶ Early Stopping

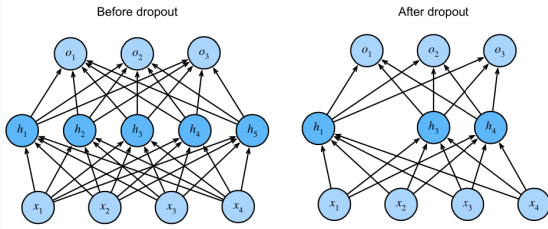
Dropout

A common form of regularization in deep learning that we have not yet talked about is **Dropout**.

Regularization norms are, in essence, ways to aim for a simpler model, which makes sense in light of the previous graph.

Dropout is a little different because it basically consists of adding noise to the model:

- ▶ It is called **dropout** because we literally drop out some neurons during model training.
- ▶ On each iteration during model training, we will zero-out (de-activate) a fraction of units in each layer.
- ▶ Typically, we will not use dropout in the test sample.



Summary of Lesson 1

In Lesson 1, we have:

- ▶ Revisited MLPs
- ▶ Discussed different options for parameter initialization
- ▶ Introduced dropout

⇒ **References for this lesson:**

Chapter 5, Sections 5.4 to 5.7 of [Zhang, Ashton, et al. *Dive into Deep Learning*](#).

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement all these novel things in practice in an increasingly deeper neural network based on MLPs.

Module 1: Lesson 2

MLP for classification: Timing AAPL stock



Outline

- ▶ MLP for classification
- ▶ Binary cross-entropy loss function
- ▶ Class weights
- ▶ Model predictive performance
- ▶ Trading strategy performance

MLP for classification

In the last module of Machine Learning, we dealt with an example of how to using neural networks, specifically MLPs, to time the momentum factor. Arguably, we used a somewhat simple model that did not really deliver an stellar performance when it came to predicting the forward-looking return of the factor.

Now, we'll insist on this challenging task with some changes:

- ▶ Perform a **classification** task: instead of focusing on predicting the forward-looking return of a financial series, we will merely try to predict whether this return will be positive ($y_i = 1$) or negative ($y_i = 0$).
- ▶ Focus on a **single stock (AAPL)**. Rather than looking at a factor that explains the cross-section of returns, we will limit our investigation to a single stock time series.
- ▶ Incorporate new features such as **dropout** and **class weights**.

Before jumping to a road map on the implementation you will see in detail in the notebook, a few caveats:

1. Focus on a single stock (AAPL). The model may not work as well/badly for others (feel free to try!)
2. Focus on a specific time period (1980–2022). The model may not work as well/badly for others (or future!).

Building our model (1/2)

We'll build our model in a very similar way to what we have done before:

- ▶ **Layers:** 3 hidden layers with 25, 15, and 10 units respectively + output layer
- ▶ **Dropout:** dropout layer of 20% after each hidden layer
- ▶ **Optimizer & Learning rate:** Adam & 10^{-6}
- ▶ **Activation function:** ReLU for all hidden layers + Sigmoid for output layer

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 25)	150
dropout (Dropout)	(None, 25)	0
dense_1 (Dense)	(None, 15)	390
dropout_1 (Dropout)	(None, 15)	0
dense_2 (Dense)	(None, 10)	160
dropout_2 (Dropout)	(None, 10)	0
dense_3 (Dense)	(None, 1)	11
=====		
Total params: 711		
Trainable params: 711		
Non-trainable params: 0		

Building our model (2/2)

More (new!) things about our model:

- **Loss function:** Binary cross-entropy

$$H(y_i) = -\frac{1}{N} \sum_{i=1}^N y_i \log(p(y_i)) + (1 - y_i) \log(1 - p(y_i))$$

- **Metric:** (binary) accuracy to evaluate our model performance

```
import tensorflow as tf

tf.keras.backend.clear_session()
tf.random.set_seed(1234)

act_fun = 'relu'
hp_units = 25
hp_units_2 = 15
hp_units_3 = 10
n_dropout = 0.2

model = tf.keras.models.Sequential()
model.add(tf.keras.layers.Dense(units = hp_units, activation=act_fun))
model.add( tf.keras.layers.Dropout(n_dropout))
model.add(tf.keras.layers.Dense(units = hp_units_2, activation=act_fun))
model.add( tf.keras.layers.Dropout(n_dropout))
model.add(tf.keras.layers.Dense(units = hp_units_3, activation=act_fun))
model.add( tf.keras.layers.Dropout(n_dropout))
model.add(tf.keras.layers.Dense(units = 1, activation='sigmoid'))

hp_lr = 1e-5

adam = tf.keras.optimizers.Adam(lr=hp_lr)

model.compile(optimizer = adam, loss='binary_crossentropy', metrics=['accuracy'])
```

Class weights

One problem that may always be present when we perform classification tasks is the under-representation of a certain class in the sample data. This can cause the model to overlook an important part of the data in the sample while training.

- ▶ For example, suppose we want to predict firm bankruptcy using imperfect historical data. Bankruptcy may not be a very frequent event in our sample but is truly our main variable of interest.
- ▶ To mitigate this problem, we can make the model pay more attention to certain classes. This can be done by introducing **class weights**, which simply consist of directing Keras to pay more attention to a certain class.
- ▶ In our case, we will try to overweight the negative classes in our sample (i.e., those instances where the 120-day-forward return on AAPL is negative).
- ▶ This is actually something we will introduce while training the model, similar to the EarlyStopping callback:

```
from sklearn.utils import class_weight

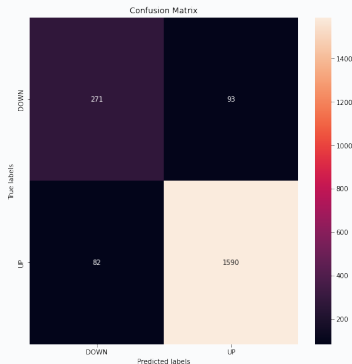
class_weight = {0: (np.mean(y_train)/0.5)*1.2, 1: 1.0}
print(class_weight)

history = model.fit(X_train, y_train, validation_split=0.2, epochs=500, batch_size=32,
                    verbose=2, callbacks=[es], class_weight=class_weight)
```

Model predictive performance

So how does our predictive model perform in the test sample?

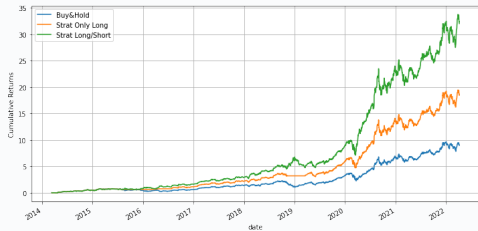
- To assess that, we will come back to an old friend: the confusion matrix!



- Model accuracy in Test: 91.40%

Trading strategy performance

Predictive power of the model in a test sample is important to assess the performance of our model. But if we want to investigate whether we could use this for trading, we better perform a backtest:



Usual **caveats** on backtesting:

- ▶ Specific stock (AAPL)
- ▶ Specific time period for training and testing (1980–2022)
- ▶ Based on historical information (what about things that never happened before?)

Summary of Lesson 2

In Lesson 2, we have:

- ▶ Developed an MLP model for classification to predict whether a certain stock will deliver a positive or negative return in the next few days
- ▶ Enhanced the performance of our MLP neural networks with features such as dropout or class weights.

⇒ **TO DO NEXT:** Go to the associated **Jupyter Notebook** to cover the full content of this lesson.

⇒ **In the next lesson**, we'll keep improving our neural network models by introducing hyperparameter tuning.

Introduction to the Keras Tuner

 Run in Google Colab (<https://colab.research.google.com/github/tensorflow/docs/blob/master/site/en/tutorials/keras/keras-tuner/keras-tuner-overview.ipynb>)

 View source on GitHub (<https://github.com/tensorflow/docs/blob/master/site/en/tutorials/keras/keras-tuner/keras-tuner-overview.ipynb>)

 Download notebook (https://storage.googleapis.com/tensorflow_docs/docs/site/en/tutorials/keras/keras-tuner/keras-tuner-overview.ipynb)

Overview

The Keras Tuner is a library that helps you pick the optimal set of hyperparameters for your TensorFlow program. The process of selecting the right set of hyperparameters for your machine learning (ML) application is called *hyperparameter tuning* or *hypertuning*.

Hyperparameters are the variables that govern the training process and the topology of an ML model. These variables remain constant over the training process and directly impact the performance of your ML program. Hyperparameters are of two types:

1. **Model hyperparameters** which influence model selection such as the number and width of hidden layers
2. **Algorithm hyperparameters** which influence the speed and quality of the learning algorithm such as the learning rate for Stochastic Gradient Descent (SGD) and the number of nearest neighbors for a k Nearest Neighbors (KNN) classifier

In this tutorial, you will use the Keras Tuner to perform hypertuning for an image classification application.

Setup

```
import tensorflow as tf
from tensorflow import keras
```

```
2024-08-16 01:25:04.811063: E external/local_xla/xla/stream_executor/cuda/cuda:
2024-08-16 01:25:04.832191: E external/local_xla/xla/stream_executor/cuda/cuda:
2024-08-16 01:25:04.838460: E external/local_xla/xla/stream_executor/cuda/cuda:
```

Install and import the Keras Tuner.

```
$ pip install -q -U keras-tuner
```

```
import keras_tuner as kt
```

Download and prepare the dataset

In this tutorial, you will use the Keras Tuner to find the best hyperparameters for a machine learning model that classifies images of clothing from the [Fashion MNIST dataset](https://github.com/zalandoresearch/fashion-mnist) (<https://github.com/zalandoresearch/fashion-mnist>).

Load the data.

```
(img_train, label_train), (img_test, label_test) = keras.datasets.fashion_mnist.load_data()
```

```
# Normalize pixel values between 0 and 1
img_train = img_train.astype('float32') / 255.0
img_test = img_test.astype('float32') / 255.0
```

Define the model

When you build a model for hypertuning, you also define the hyperparameter search space in addition to the model architecture. The model you set up for hypertuning is called a *hypermodel*.

You can define a hypermodel through two approaches:

- By using a model builder function
- By subclassing the `HyperModel` class of the Keras Tuner API

You can also use two pre-defined `HyperModel` (https://keras.io/api/keras_tuner/hypermodels/) classes - `HyperXception` (https://keras.io/api/keras_tuner/hypermodels/hyper_xception/) and `HyperResNet` (https://keras.io/api/keras_tuner/hypermodels/hyper_resnet/) for computer vision applications.

In this tutorial, you use a model builder function to define the image classification model. The model builder function returns a compiled model and uses hyperparameters you define inline to hypertune the model.

```
def model_builder(hp):
    model = keras.Sequential()
    model.add(keras.layers.Flatten(input_shape=(28, 28)))

    # Tune the number of units in the first Dense layer
    # Choose an optimal value between 32-512
    hp_units = hp.Int('units', min_value=32, max_value=512, step=32)
    model.add(keras.layers.Dense(units=hp_units, activation='relu'))
    model.add(keras.layers.Dense(10))

    # Tune the learning rate for the optimizer
    # Choose an optimal value from 0.01, 0.001, or 0.0001
    hp_learning_rate = hp.Choice('learning_rate', values=[1e-2, 1e-3, 1e-4])

    model.compile(optimizer=keras.optimizers.Adam(learning_rate=hp_learning_rate),
                  loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
                  metrics=['accuracy'])

    return model
```

Instantiate the tuner and perform hypertuning

Instantiate the tuner to perform the hypertuning. The Keras Tuner has four tuners available - `RandomSearch`, `Hyperband`, `BayesianOptimization`, and `Sklearn`. In this tutorial, you use the `Hyperband` (<https://arxiv.org/pdf/1603.06560.pdf>) tuner.

To instantiate the Hyperband tuner, you must specify the hypermodel, the `objective` to optimize and the maximum number of epochs to train (`max_epochs`).

```
tuner = kt.Hyperband(model_builder,
                     objective='val_accuracy',
                     max_epochs=10,
                     factor=3,
                     directory='my_dir',
                     project_name='intro_to_kt')
```

```
WARNING: All log messages before absl::InitializeLog() is called are written
I0000 00:00:1723771509.637777 14090 cuda_executor.cc:1015] successful NUMA
I0000 00:00:1723771509.641612 14090 cuda_executor.cc:1015] successful NUMA
I0000 00:00:1723771509.644868 14090 cuda_executor.cc:1015] successful NUMA
I0000 00:00:1723771509.648549 14090 cuda_executor.cc:1015] successful NUMA
I0000 00:00:1723771509.660168 14090 cuda_executor.cc:1015] successful NUMA
I0000 00:00:1723771509.663655 14090 cuda_executor.cc:1015] successful NUMA
I0000 00:00:1723771509.666633 14090 cuda_executor.cc:1015] successful NUMA
I0000 00:00:1723771509.670142 14090 cuda_executor.cc:1015] successful NUMA
I0000 00:00:1723771509.673591 14090 cuda_executor.cc:1015] successful NUMA
I0000 00:00:1723771509.677149 14090 cuda_executor.cc:1015] successful NUMA
I0000 00:00:1723771509.680118 14090 cuda_executor.cc:1015] successful NUMA
I0000 00:00:1723771509.683613 14090 cuda_executor.cc:1015] successful NUMA
```

The Hyperband tuning algorithm uses adaptive resource allocation and early-stopping to quickly converge on a high-performing model. This is done using a sports championship style bracket. The algorithm trains a large number of models for a few epochs and carries forward only the top-performing half of models to the next round. Hyperband determines the number of models to train in a bracket by computing $1 + \log_{\text{factor}}(\text{max_epochs})$ and rounding it up to the nearest integer.

Create a callback to stop training early after reaching a certain value for the validation loss.

```
stop_early = tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=5)
```

Run the hyperparameter search. The arguments for the search method are the same as those used for `tf.keras.model.fit` in addition to the callback above.

```
tuner.search(img_train, label_train, epochs=50, validation_split=0.2, callback

# Get the optimal hyperparameters
best_hps=tuner.get_best_hyperparameters(num_trials=1)[0]

print(f"""
The hyperparameter search is complete. The optimal number of units in the first
layer is {best_hps.get('units')} and the optimal learning rate for the optimizer
is {best_hps.get('learning_rate')}.
""")
```

```
Trial 30 Complete [00h 00m 25s]
val_accuracy: 0.8913333415985107
```

```
Best val_accuracy So Far: 0.8913333415985107
Total elapsed time: 00h 05m 37s
```

```
The hyperparameter search is complete. The optimal number of units in the first
layer is 416 and the optimal learning rate for the optimizer
is 0.001.
```

Train the model

Find the optimal number of epochs to train the model with the hyperparameters obtained from the search.

```
# Build the model with the optimal hyperparameters and train it on the data for
model = tuner.hypermodel.build(best_hps)
history = model.fit(img_train, label_train, epochs=50, validation_split=0.2)

val_acc_per_epoch = history.history['val_accuracy']
best_epoch = val_acc_per_epoch.index(max(val_acc_per_epoch)) + 1
print('Best epoch: %d' % (best_epoch,))
```

```

Epoch 1/50
1500/1500 _____ 4s 2ms/step - accuracy: 0.7774 - loss: 0.6344 -
Epoch 2/50
1500/1500 _____ 2s 1ms/step - accuracy: 0.8643 - loss: 0.3766 -
Epoch 3/50
1500/1500 _____ 2s 1ms/step - accuracy: 0.8790 - loss: 0.3291 -
Epoch 4/50
1500/1500 _____ 2s 1ms/step - accuracy: 0.8884 - loss: 0.3036 -
Epoch 5/50
1500/1500 _____ 2s 1ms/step - accuracy: 0.8955 - loss: 0.2840 -
Epoch 6/50
1500/1500 _____ 2s 1ms/step - accuracy: 0.8998 - loss: 0.2655 -
Epoch 7/50

```

Re-instantiate the hypermodel and train it with the optimal number of epochs from above.

```

hypermodel = tuner.hypermodel.build(best_hps)

# Retrain the model
hypermodel.fit(img_train, label_train, epochs=best_epoch, validation_split=0.1)

```

```

Epoch 1/32
1500/1500 _____ 3s 2ms/step - accuracy: 0.7789 - loss: 0.6225 -
Epoch 2/32
1500/1500 _____ 2s 1ms/step - accuracy: 0.8678 - loss: 0.3664 -
Epoch 3/32
1500/1500 _____ 2s 2ms/step - accuracy: 0.8781 - loss: 0.3343 -
Epoch 4/32
1500/1500 _____ 2s 2ms/step - accuracy: 0.8879 - loss: 0.3070 -
Epoch 5/32
1500/1500 _____ 2s 2ms/step - accuracy: 0.8908 - loss: 0.2902 -
Epoch 6/32
1500/1500 _____ 2s 2ms/step - accuracy: 0.9002 - loss: 0.2703 -
Epoch 7/32

```

To finish this tutorial, evaluate the hypermodel on the test data.

```

eval_result = hypermodel.evaluate(img_test, label_test)
print("[test loss, test accuracy]:", eval_result)

```

```
313/313 _____ 1s 2ms/step - accuracy: 0.8873 - loss: 0.4595  
[test loss, test accuracy]: [0.4649185538291931, 0.8881000280380249]
```

The `my_dir/intro_to_kt` directory contains detailed logs and checkpoints for every trial (model configuration) run during the hyperparameter search. If you re-run the hyperparameter search, the Keras Tuner uses the existing state from these logs to resume the search. To disable this behavior, pass an additional `overwrite=True` argument while instantiating the tuner.

Summary

In this tutorial, you learned how to use the Keras Tuner to tune hyperparameters for a model. To learn more about the Keras Tuner, check out these additional resources:

- [Keras Tuner on the TensorFlow blog](https://blog.tensorflow.org/2020/01/hyperparameter-tuning-with-keras-tuner.html)
(<https://blog.tensorflow.org/2020/01/hyperparameter-tuning-with-keras-tuner.html>)
- [Keras Tuner website](https://keras-team.github.io/keras-tuner/) (<https://keras-team.github.io/keras-tuner/>)

Also check out the [HPParams Dashboard](https://www.tensorflow.org/tensorboard/hyperparameter_tuning_with_hparams)

(https://www.tensorflow.org/tensorboard/hyperparameter_tuning_with_hparams) in TensorBoard to interactively tune your model hyperparameters.

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2024-08-16 UTC.

Module 1: Lesson 4

Hyperparameter Optimization on Keras Tuner



Outline

- ▶ Regression problem: Testing the Keras tuner
- ▶ Keras Tuner
- ▶ Timing AAPL stock using tuned model hyperparameters

Testing the Keras Tuner

In this practical lesson, we'll revisit the problem of timing AAPL stock (or any other stock/financial time series for that matter) that we faced in lesson 2 of this module. This time, however, we'll make it a regression (rather than classification) problem:

- ▶ To do this, we will use an MLP framework in a regression problem to predict the future 25-day of AAPL stock
- ▶ Different from previous examples, here, we will incorporate hyper-parameter tuning via the Keras Tuner.
- ▶ This whole exercise yields two main takeaways:
 - (i) Hyperparameter tuning is **computationally costly** (even in our simple case, it takes up to 5 minutes of computations).
 - (ii) Hyperparameter tuning can have a **substantial effect** on the predictive performance of our models (even simple tuning), especially in these first initial stages for us.
- ▶ For the complete documentation on the Keras tuner, please visit: https://keras.io/api/keras_tuner/

Keras Tuner

The HPO task is much facilitated by the Keras tuner: a library for hyperparameter tuning in TensorFlow.

- ▶ To properly use it, we first need to define our **hypermodel**: which parameters should we tune? what choices are available?
- ▶ In this case, we will tune: (i) # hidden layers, (ii) # units in hidden layers, and (iii) dropout rate:

```
class MLP_model(HyperModel):
    def build(self, hp):

        # We define a constant activation function of ReLU form. We will not be tuning the activation functions
        act_fun = 'relu'

        # We do ask the Keras Tuner to choose whether is best to have a dropout rate after each hidden layer of 0.2 or 0.3
        n_dropout = hp.Choice('n_dropout', values=[0.20, 0.30])

        model = tf.keras.models.Sequential()

        # Now, we will use a loop to let the tuner choose the number of layers that is best for the model between 1 and 5
        for i in range(1, hp.Int("num_layers", 1, 5)):
            # Within this loop, we will also ask the tuner to decide the optimal number of units that each of the selected layer should have.
            model.add(tf.keras.layers.Dense(units= hp.Int("units_dense_" + str(i), min_value=1, max_value=25, step=5), activation=act_fun))
            model.add( tf.keras.layers.Dropout(n_dropout))

        model.add(tf.keras.layers.Dense(units = 1))

        # As was the case with the activation function, there is no tuning on the learning rate, nor the optimizer, although we could
        # easily incorporate it.
        hp_lr = 1e-5
        adam = tf.keras.optimizers.Adam(learning_rate = hp_lr)
        model.compile(optimizer = adam, loss='mean_absolute_error')

        return model
```

Keras Tuner

Once we have built our hypermodel, it's time to put it to work:

- Our choice of optimizer (i.e., tuner) will be the **hyperband**:

Li, Lisha, et al. "Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization." *The Journal of Machine Learning Research*, vol. 18, no. 1, 2017, pp. 6765–6816, <https://www.jmlr.org/papers/volume18/16-558/16-558.pdf>

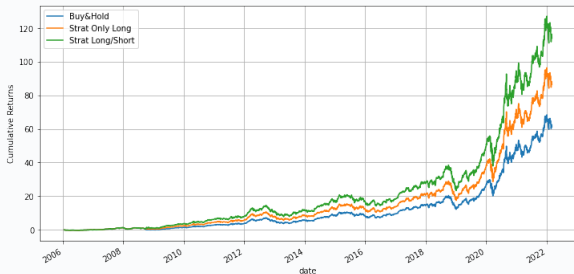
- Hyperband requires:
 - Hypermodel: just defined!
 - Objective: how to optimize? → Minimum loss in validation
 - Other parameters: setting random seed, storage of results, etc.

```
hypermodel=MLP_model()
tuner = kt.Hyperband(hypermodel,
                    objective = kt.Objective("val_loss", direction="min"),
                    overwrite = True,
                    max_epochs = 30,
                    seed = 1234,
                    directory=os.path.normpath('C:/'))

# Let's run the tuner! (Warning: this could take time)
tuner.search(X_train, y_train, validation_split=val_split)
best_hps = tuner.get_best_hyperparameters(num_trials = 1)[0]
```

Trading strategy based on tuned model predictions

Once we have tuned our model hyperparameters, we can check how a trading strategy based on model predictions in the test sample would have performed via **backtesting**:



One more time, please do remember the caveats associated with backtesting:

- It only validates the performance of the strategy for the test sample—no external validity from this!

Summary of Lesson 4

In Lesson 4, we have looked at:

- ▶ Keras tuner for hyperparameter optimization
- ▶ Performance of Keras tuner in a regression problem

⇒ **References for this lesson:**

Yu, Tong, and Hong Zhu. "Hyperparameter Optimization: A Review of Algorithms and Applications." *arXiv*, 2020, <https://arxiv.org/abs/2003.05689>

Introduction to Keras tuner example in TensorFlow:

https://www.tensorflow.org/tutorials/keras/keras_tuner?hl=en

TO DO NEXT: Go to the associated **Jupyter Notebook** to cover the full content of this lesson.

In the next module, we'll continue to dig deeper into the Deep Learning world by moving forward to more advanced and complex models and techniques.

Module 1: Lesson 3

Hyperparameter Optimization (HPO)

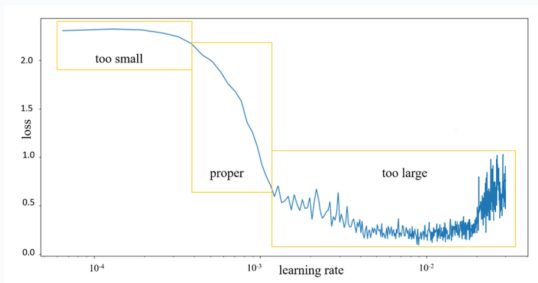


Outline

- ▶ Hyperparameter Optimization (HPO): why, what, and how?
- ▶ Which hyperparameters?
- ▶ Optimization algorithms

Hyperparameter Optimization (HPO)

- ▶ So far, we have used neural networks in a kind of **brute force** approach: simply by using trial and error in our models, we would see which combination of features (i.e., number of hidden layers, learning rate, optimizer, etc.) perform best.
- ▶ This is however quite inefficient because it requires spending a lot of time deciphering the best choices for a model.
- ▶ **Solution:** Hyperparameter Optimization (HPO)



Hyperparameter Optimization (HPO)

► **Solution:** Hyperparameter Optimization (HPO)

- Hyperparameters refer to parameters that cannot be updated during the training of the model.
- These hyperparameters can be **structure-related** (e.g., # of hidden layers), or **training-related** (e.g., learning rate or minibatch size)
- Final step of model design and first step of training a neural network

► **Purpose of HPO:**

1. Reduce costly menial work by the researcher
2. Improve accuracy and efficiency of training
3. Make the choice of hyper-parameters more convincing and reproducible

► **How?**

- We'll be using a series of optimization techniques
- These can be classified as **search algorithms** (e.g., grid search) and **trial algorithms** (e.g., curve fitting).

Which hyperparameters?

Given that HPO takes considerable computational resources, it is important to understand which hyperparameters take preferential treatment in the tuning. Here, we review major hyperparameters based on previous researchers' experience.

► Training-related hyperparameters:

- Learning rate (constant, linear/exponential decay)
- Optimizer (Mini-batch SGD, RMSprop, Adam)
- ...

► Model-design hyperparameters:

- Number of hidden layers
- Width (number of neurons) of hidden layers
- Regularization in cost function (ℓ_1 vs. ℓ_2 norms)
- Dropout (which rate?)
- Activation function (ReLU, Sigmoid, Tanh?)
- ...

Optimization algorithms

- Mathematically, HPO consists simply of finding a set of hyperparameters that achieve the minimum loss (or maximum accuracy) of a network/model.
- Computationally, HPO represents a complex computational problem to be solved by state-of-the-art algorithms of mainly two types: search and trial.

► Search algorithms

- Grid search: straightforward, but computationally costly (remember the curse of dimensionality!)
- Random search: similar to grid with randomized search (less time-consuming)
- Bayesian Optimization: selects next hyperparameter based on previous experience

► (Trial) Early-stopping algorithms

- Curve fitting: LPA algorithm (Learn, Predict, Assess)
- Successive Halving (SHA) and Hyperband (HB): random search sampling method + bandit-based early-stopping policy
- Extensions: Asynchronous Successive Halving (ASHA) and Bayesian Optimization Hyperband (BOHB)

Summary of Lesson 3

In Lesson 3, we have looked at:

- ▶ What is HPO, why we need it, and the basics of how to perform it
- ▶ Main hyper-parameters we can tune
- ▶ Basic review of algorithms for HPO

⇒ **References for this lesson:**

Yu, Tong, and Hong Zhu. "Hyperparameter Optimization: A Review of Algorithms and Applications." arXiv, 2020, <https://arxiv.org/abs/2003.05689>

. **TO-DO NEXT:** There is no associated Jupyter Notebook for this lesson, but it is extremely important that you go over the required readings for this lesson thoroughly.

In the next lesson, we will come back one more time to our stock timing example to see how a regression model based on a somewhat simple neural network can benefit from hyperparameter tuning using the Keras Tuner.